Course – Theoretical Foundations of Computations

Course Instructor

Dr. Umadevi V

Department of CSE, BMSCE

Webpage:https://sites.google.com/site/drvumadevi/

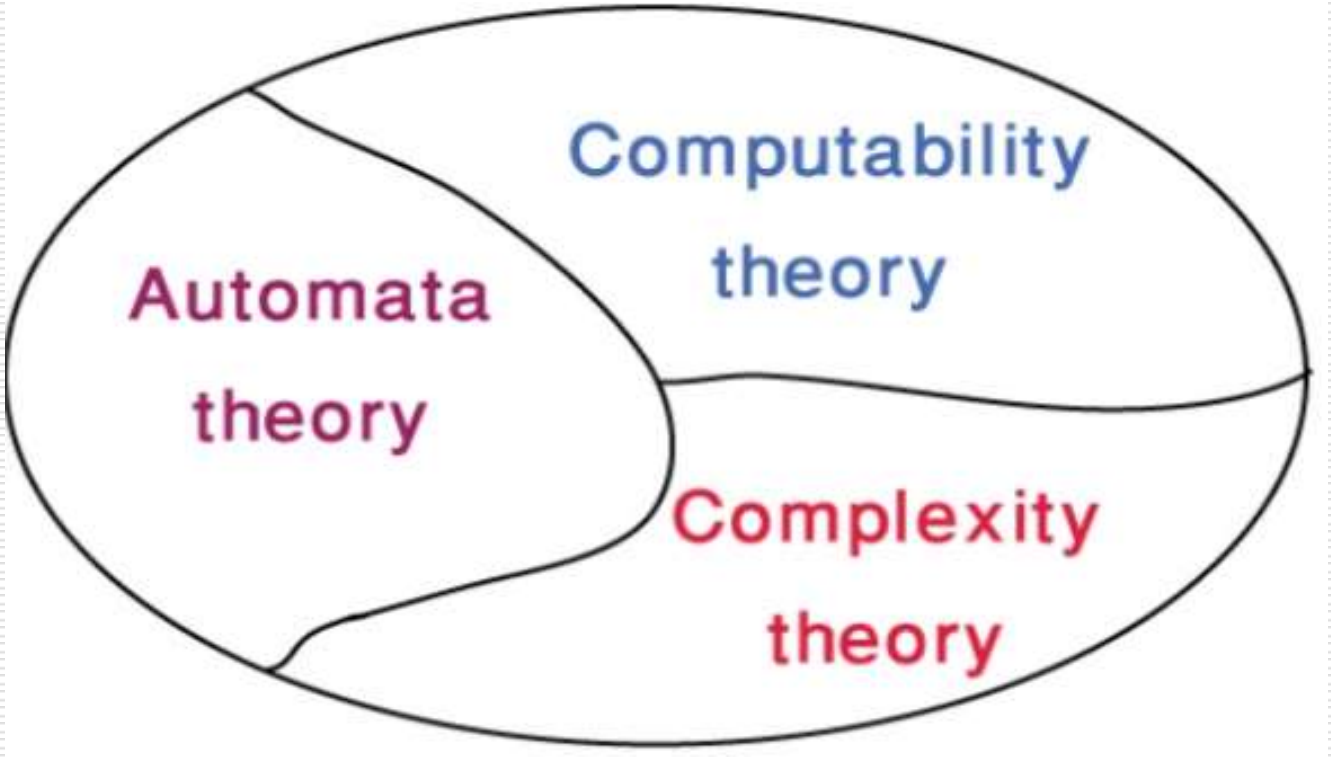
Unit-1

Introduction to Finite Automata

Introduction to Finite Automata, Central Concepts of Automata Theory, Deterministic Finite Automata (DFA), Nondeterministic Finite Automata (NFA), Finite Automata with Epsilon Transition, An Application Text Search.

Theory of Computation

The field is divided into Three major Branches



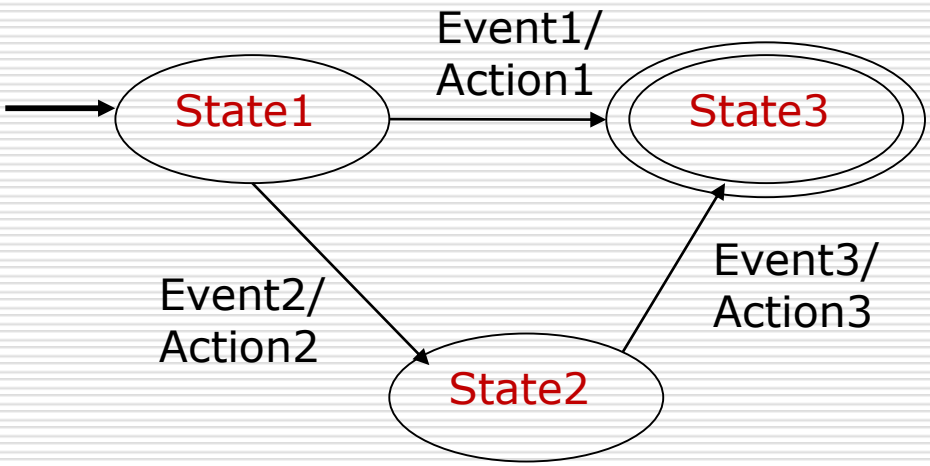
What is automata theory ?

- Automata theory is the study of abstract computational devices
- Abstract devices are (simplified) models of real computations
- Computations happen everywhere: On your laptop, on your cell phone, in nature, …
- Why do we need abstract models?

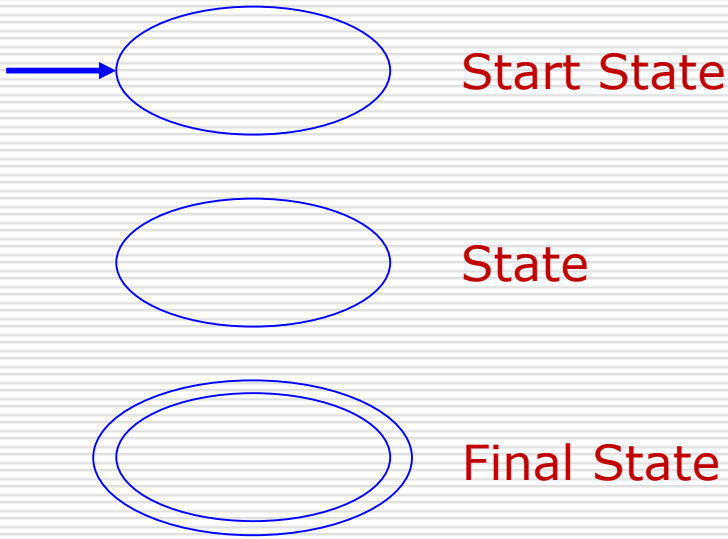
Finite State machine

Abstract machine or Automata or Finite State machine

Example:

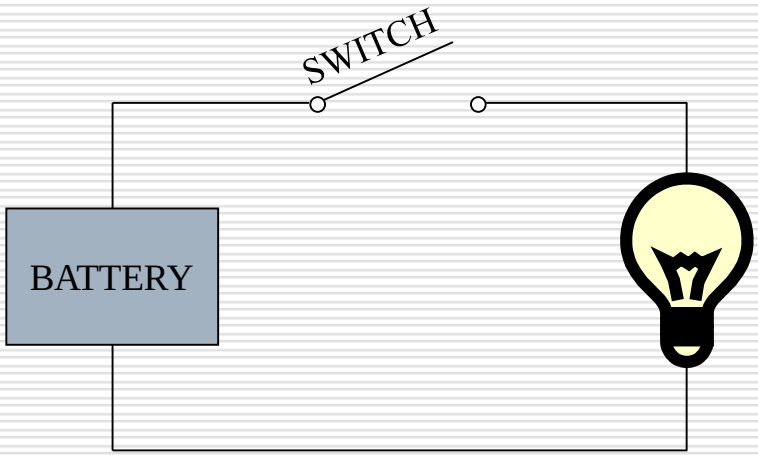


Notations



Nodes: State
Edges: Transition
Edge Label: Events / Actions

A simple computer



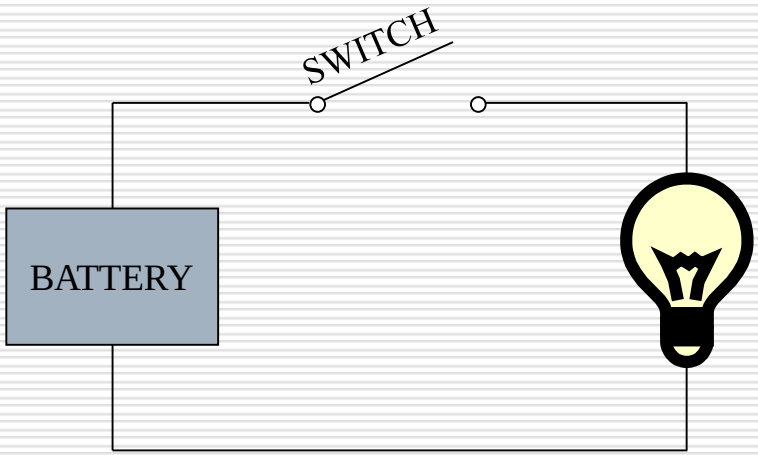
input: switch

output: light bulb

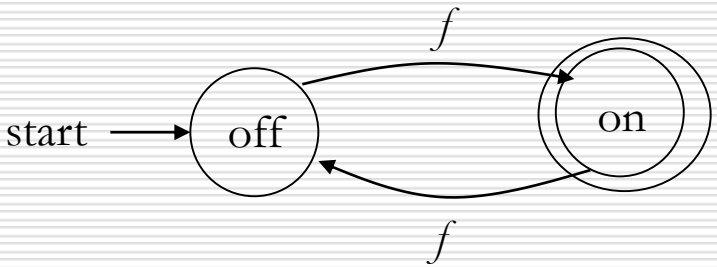
actions: flip switch

states: on, off

A simple "computer"



Abstract machine or Automaton



input: switch

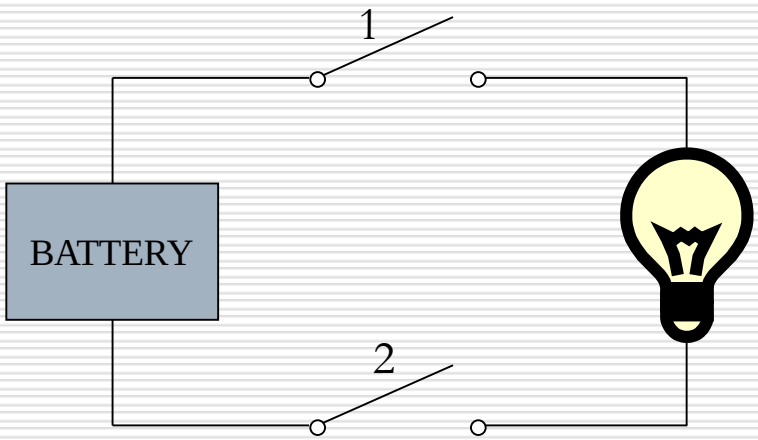
output: light bulb

actions: f for "flip switch"

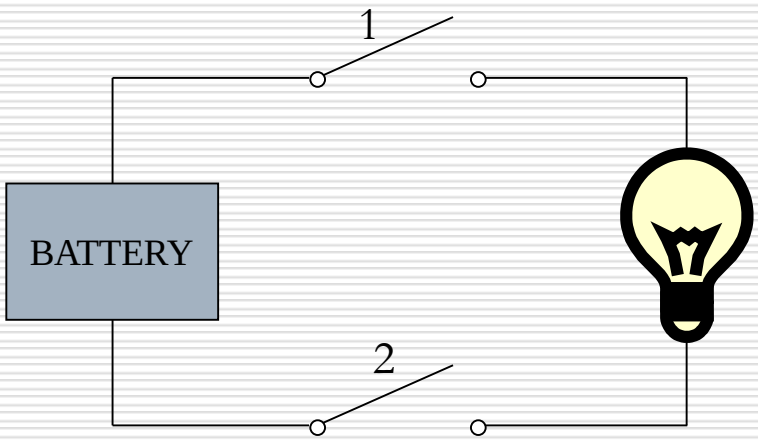
states: on, off

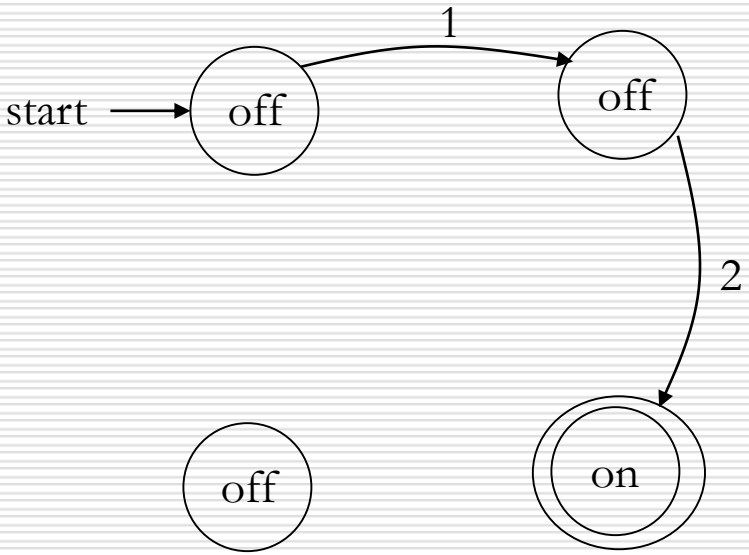
bulb is on if and only if there was an odd number of flips

Another “computer”



Another "computer"





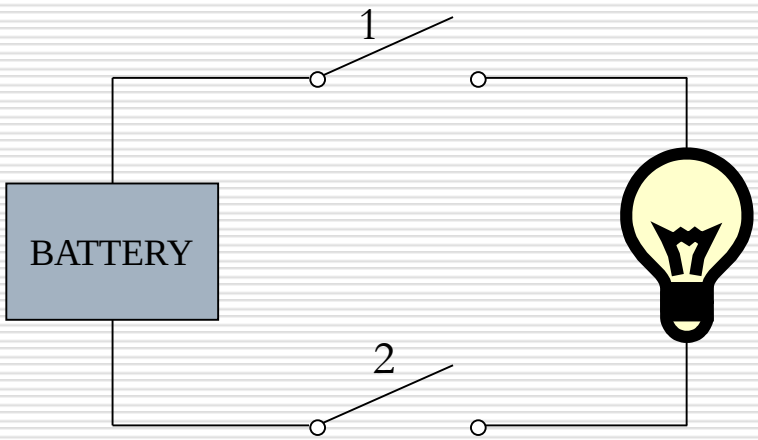
inputs: switches 1 and 2

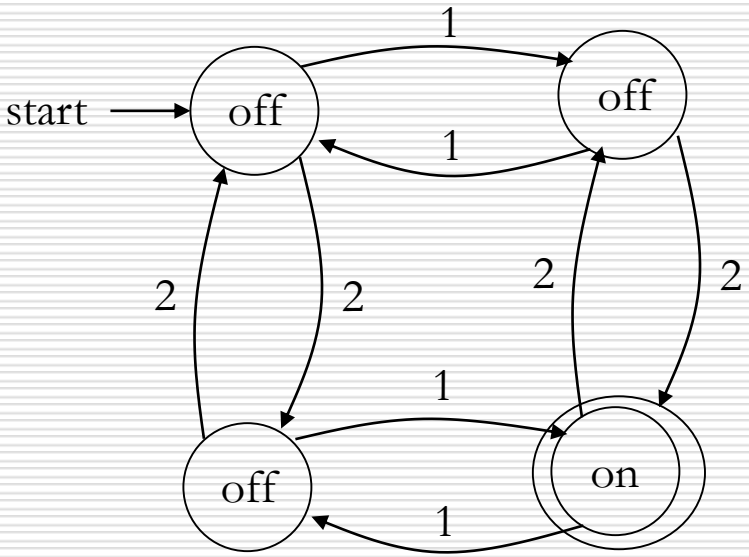
actions: 1 for "flip switch 1"
2 for "flip switch 2"

states: on, off

bulb is on if and only if both switches were flipped an odd number of times

Another "computer"





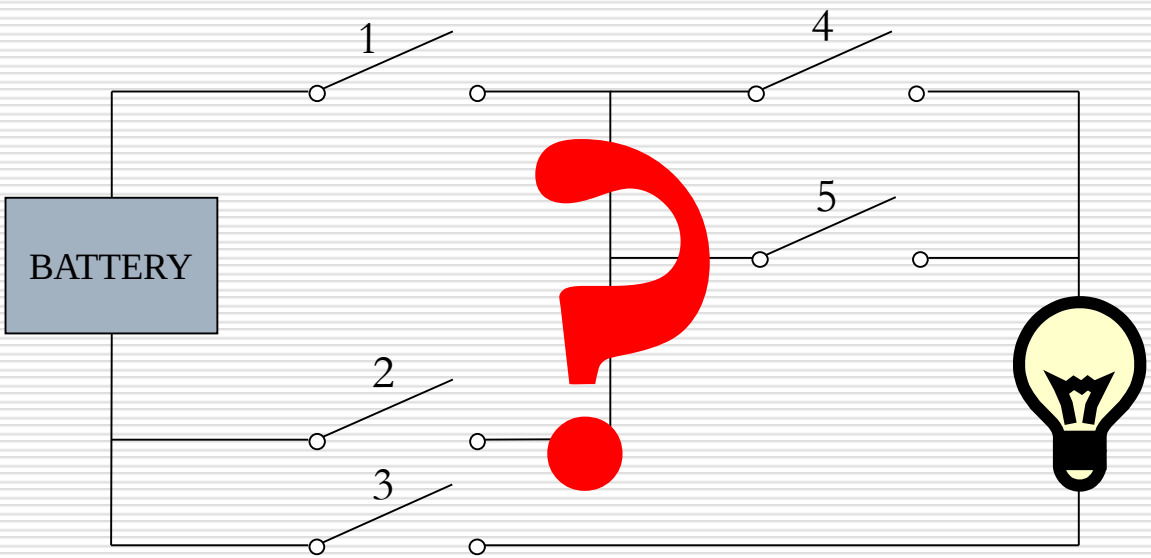
inputs: switches 1 and 2

actions: 1 for "flip switch 1"
2 for "flip switch 2"

states: on, off

bulb is on if and only if both switches were flipped an odd number of times

A design problem



Can you design a circuit where the light is on if and only if all the switches were flipped exactly the same number of times?

A design problem

- Such devices are difficult to reason about, because they can be designed in an infinite number of ways
- By representing them as abstract computational devices, or automata, we will learn how to answer such questions

Applications of Automata Theory

- **In Compiler:** Lexical analyzers and Parser generator
- **Word search** and Translation of Natural Languages
- Parity checkers, Vending machines, Communication protocols, Video games, Artificial Intelligence etc.

Different kinds of automata we will work on in this course

Finite Automata	Devices with a finite amount of memory. Used to model "small" computers.
Push-down Automata	Devices with infinite memory that can be accessed in a restricted way. Used to model Parsers, etc.
Turing Machines	Devices with infinite memory. Used to model any computer.

The **Applications** of these Automata are given as follows:

1. Finite Automata (FA) –

For the designing of lexical analysis of a compiler.

For recognizing the pattern using regular expressions.

For the designing of the combination and sequential circuits using Mealy and Moore Machines.

Used in text editors.

For the implementation of spell checkers.

2. Push Down Automata (PDA) –

For designing the parsing phase of a compiler (Syntax Analysis).

For implementation of stack applications.

For evaluating the arithmetic expressions.

For solving the Tower of Hanoi Problem.

3. Turing Machine (TM) –

For solving any recursively enumerable problem.

For understanding complexity theory.

For implementation of neural networks.

For implementation of Robotics Applications.

For implementation of artificial intelligence.

Unit-1

Central Concepts of Automata Theory

- Alphabets
- Strings
- Languages

Central Concepts

- Symbol
  - A symbol is a single object and is an abstract entity that has no meaning by itself.
  - It can be alphabet, character or special character

Central Concepts

Alphabet

- An ***alphabet*** is a finite, nonempty set of symbols, Σ (sigma) is used for an alphabet.
- For example

  {0, 1} is an alphabet with two symbols

  {a, b} is another alphabet with two symbols, and

  English alphabet is also an alphabet
  - Σ = {0,1}, or Σ = the binary alphabet
  - Σ = {a,b,…..z}, or Σ=the set of all lower-case letters

Central Concepts: **String**

- A ***string*** is a finite sequence of symbols chosen from some alphabet. e.g. 01101 is a string from the binary alphabet $\Sigma = \{0,1\}$.
- The number of symbols in a *string* is called the ***length of a string.*** e.g. 011101 has length 5 and the number of symbols are 2. The standard notation for the length of a string *w* is $|w|$. e.g. $|011| = 3$

Central Concepts: **String**

- The ***empty string*** (also called null string) is the string with length zero. That is, it has no symbols. It is denoted with ε (epsilon), that may be chosen from any alphabet whatsoever. Thus | ε | = 0.

Central Concepts: **String**

Powers of an Alphabet: If Σ is an alphabet, we define Σ^k to be the set of strings of length k, each of whose symbols is in Σ. e.g. $\Sigma^0 = \{\varepsilon\}$.

If $\Sigma = \{0,1\}$, then

$\Sigma^1 = \{0,1\}$

$\Sigma^2 = \{00,01,10,11\}$

$\Sigma^3 = \{000,001,010,011,100,101,110,111\}$

The set of all strings over an alphabet is denoted as Σ^*. For instance, $\{0,1\}^* = \{\varepsilon,0,1,00,01,10,11,000,\ldots\}$. Put another way,

$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \ldots$

$\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \ldots$

Central Concepts

Strings

- ***Concatenation of Strings***: Let x and y be strings. Then xy denotes the string obtained by concatenating x with y, that is, xy is the string obtained by appending the sequence os symbols of y to that of x. e.g. if $x = aab$ and $y = bbab$, then $xy = aabbbab$. Note that $xy \neq yx$.

Central Concepts

Languages

- A *language* is a set of strings over an alphabet. Thus *{a, ab, baa}* is a language (over alphabet *{a, b}*).
- A set of strings all of which are chosen from some Σ^*, where Σ is a particular alphabet, is called a *language*. If Σ is an alphabet, and $L \subseteq \Sigma^*$, then L is a *language* over Σ.
  - An example is English, where the collection of legal English words is a set of strings over the alphabet that consists of all the letters.

Question

Consider the alphabet set as $\Sigma = \{0,1\}$. Write down possible strings of the Language, L, which is defined as follows:

$L=\{w \mid w$ consists of equal number of 0's and 1's$\}$

Question

Consider the alphabet set as $\Sigma = \{0,1\}$. Write down possible strings of the Language, L, which is defined as follows:

L={w | w consists of equal number of 0's and 1's}

Answer:

L={ε, 01, 10, 1100, 0011, 0101, 1010, 0011010,……..}

Examples of Languages

- The language of all strings consisting of *n* 0's followed by *n* 1's, for some n ≥ 0:
  - {ε, 01, 0011, 000111,…..}
- The set of strings of 0's and 1's with an equal number of each:
  - {ε, 01,10,0011,0101,1001,…..}
- The set of binary numbers whose value is a prime:
  - {10, 11, 101,111,1011…..}

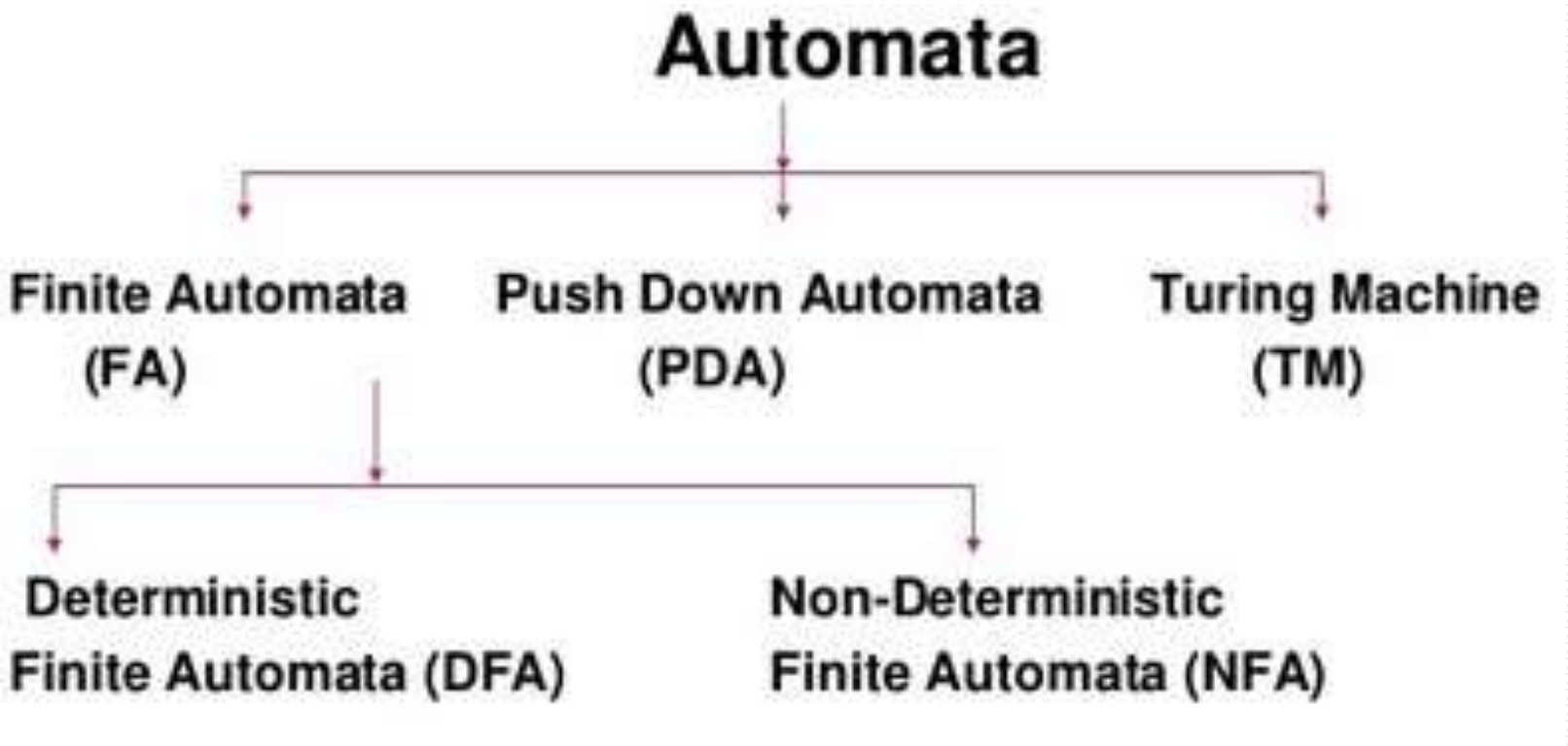
Note: The only important constraint on what can be a language is that all alphabets are finite, although they can have an infinite number of strings.

Summarizing

Central Concepts

1. Alphabet: A set of Symbols
2. Strings: A list of symbols from an alphabet
3. Language: A set of strings from the same alphabet.

Types of Finite Automata



Formal Definition of Deterministic Finite Automata

A Deterministic Finite Automaton (DFA) is a 5-tuple system

M= $(Q, \Sigma, \delta, q_0, F)$ where

- Q is a finite set of states
- Σ is an alphabet set or input symbols
- $\delta: Q \times \Sigma \rightarrow Q$ is a transition function
- $q_0 \in Q$ is the initial (or start)state $\rightarrow q_0$
- $F \subseteq Q$ is a set of accepting states (or final states). q

Question

Construct a Deterministic Finite Automata (DFA) to recognize all odd number a's given the alphabet set as $\Sigma = \{a\}$

Note: For DFA construction specific steps to be followed cannot be generalized for all computational problems.

But to get started, we will follow some steps. This steps may not be applicable to all DFA'. But, it helps to get started.

Construct a Deterministic Finite Automata (DFA) to recognize all odd number a's given the alphabet set as Σ ={a}

Step1: Write possible strings of the Language that will be accepted for the given DFA

L={a, aaa, aaaaa,……}

Construct a Deterministic Finite Automata (DFA) to recognize all odd number a's given the alphabet set as Σ ={a}

Step1: Write possible strings of the Language that will be accepted for the given DFA

L={a, aaa, aaaaa,……}

Step2: Write possible strings of the Language that should **not** be accepted for the given DFA

$\bar{L}$ ={ε, aa, aaaa,…….}

Construct a Deterministic Finite Automata (DFA) to recognize all odd number a's given the alphabet set as Σ ={a}

Step1: Write possible strings of the Language that will be accepted for the given DFA

L={a, aaa, aaaaa,……}

Step2: Write possible strings of the Language that should **not** be accepted for the given DFA

$\overline{L}$ ={ε, aa, aaaa,…….}

Step3: Write down the minimum string that will be accepted for the given DFA

'a'

Construct a Deterministic Finite Automata (DFA) to recognize all odd number a's given the alphabet set as Σ ={a}

Step1: Write possible strings of the Language that will be accepted for the given DFA

L={a, aaa, aaaaa,……}

Step2: Write possible strings of the Language that should **not** be accepted for the given DFA

$\overline{L}$ ={ε, aa, aaaa,…….}

Step3: Write down the minimum string that will be accepted for the given DFA

'a'

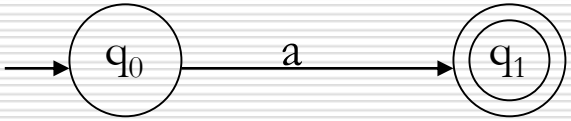
Step4: Find the length of the minimum string, n=|a|=1

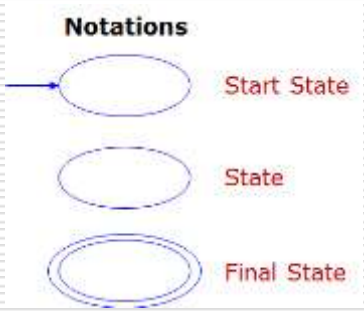
Step5: Find number of States required to construct this DFA

Number of States=n+1=1+1=2

Construct a Deterministic Finite Automata (DFA) to recognize all odd number a's given the alphabet set as Σ ={a}

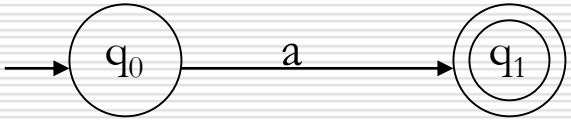
Step6: Draw the transition diagram of the DFA that accepts minimum string. Note: Make sure that there in one Start State and atleast one final state

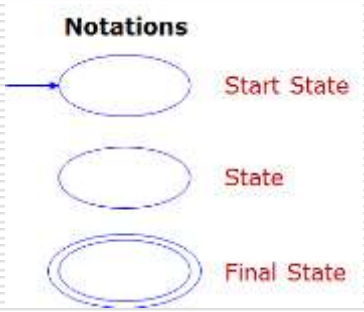




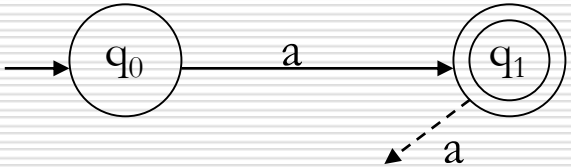
Construct a Deterministic Finite Automata (DFA) to recognize all odd number a's given the alphabet set as Σ ={a}

Step6: Draw the transition diagram of the DFA that accepts minimum string. Note: Make sure that there in one Start State and atleast one final state



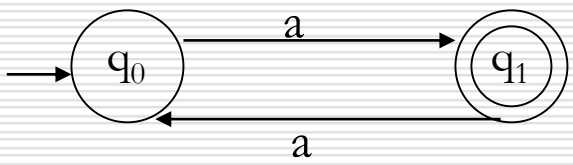


Step7: For the basic DFA, defined in Step6, consider one state at a time and obtain the various missing transitions on each input symbol



Construct a Deterministic Finite Automata (DFA) to recognize all odd number a's given the alphabet set as Σ ={a}

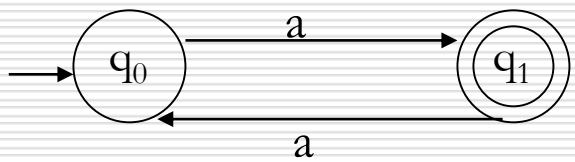
Step8: For the missing transitions identified in step7, connect this transitions to appropriate state so that the string stated in the problem is accepted.



Transition Diagram

Construct a Deterministic Finite Automata (DFA) to recognize all odd number a's given the alphabet set as Σ ={a}

Step8: For the missing transitions identified in step7, connect this transitions to appropriate state so that the string stated in the problem is accepted.



Transition Diagram

Step9: Write the formal definition of DFA

M= $(Q, \Sigma, \delta, q_0, F)$ where

- $Q = \{q_0, q_1\}$ Finite set of states
- $\Sigma = \{a\}$ Alphabet set or input symbols
- $q_0 = q_0$ Initial (or start) state
- $F = \{q_1\}$ Accepting states (or Final states).
- $\delta: Q \times \Sigma \rightarrow Q$ is a transition function

Transition Table

δ	**a**
q_0	q_1
q_1	q_0

Construct a Deterministic Finite Automata (DFA) to recognize all odd number a's given the alphabet set as $\Sigma = \{a\}$

Step10: Consider any one string which will be accepted and one string which should not accepted by this DFA defined in the step9. Show the transition function movements for these strings.

$\delta(q_0, aaa) = \delta(q_1, aa) = \delta(q_0, a) = q_1 \in F$, Final state

Hence string aaa is accepted

Transition Table

δ	**a**
q_0	q_1
q_1	q_0

Construct a Deterministic Finite Automata (DFA) to recognize all odd number a's given the alphabet set as $\Sigma = \{a\}$

Step10: Consider any one string which will be accepted and one string which should not accepted by this DFA defined in the step9. Show the transition function movements for these strings.

$\delta(q_0, aaa) = \delta(q_1, aa) = \delta(q_0, a) = q_1 \in F$, Final state

Hence string aaa is accepted

$\delta(q_0, aa) = \delta(q_1, a) = q_0 \notin F$, Final state

Hence string aa is not accepted

Transition Table

δ	**a**
q_0	q_1
q_1	q_0

Question

Construct a Deterministic Finite Automata (DFA) that accepts set of all strings having aa as a substring. Consider the alphabet set as $\Sigma = \{a,b\}$

Construct a Deterministic Finite Automata (DFA) that accepts set of all strings having aa as a substring. Consider the alphabet set as Σ ={a,b}

Step1: Write possible strings of the Language that will be accepted for the given DFA

L={aa, aaa, baa, aab, bbaa, aabb, baab, baaab, baabba,……}

Step2: Write possible strings of the Language that should **not** be accepted for the given DFA

$\overline{L}$ ={ε, a, ab, ba, aba, baba, abab…….}

Step3: Write down the minimum string that will be accepted for the given DFA

'aa'

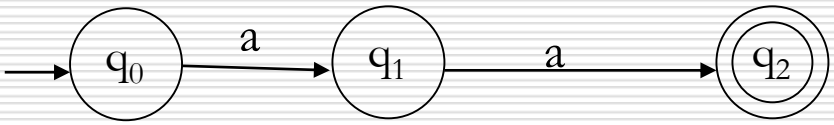
Step4: Find the length of the minimum string, n=|aa|=2

Step5: Find number of States required to construct this DFA

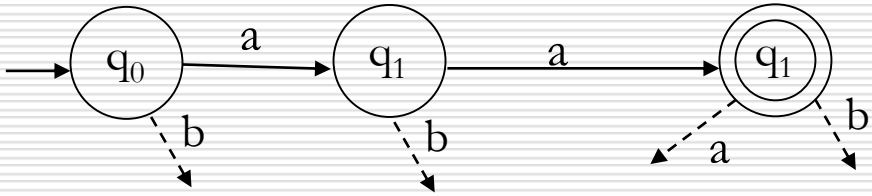
Number of States=n+1=2+1=3

Construct a Deterministic Finite Automata (DFA) that accepts set of all strings having aa as a substring. Consider the alphabet set as Σ ={a,b}

Step6: Draw the transition diagram of the DFA that accepts minimum string. Note: Make sure that there in one Start State and atleast one final state

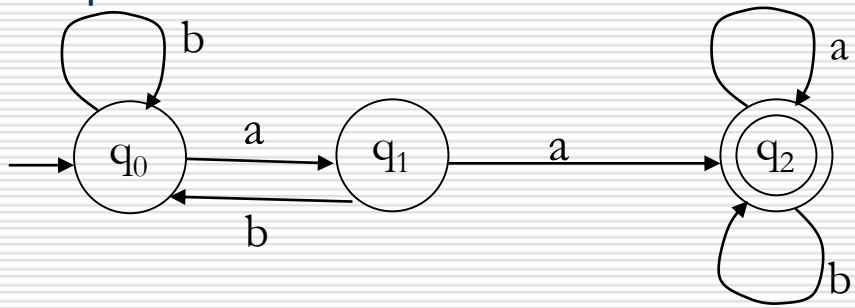


Step7: For the basic DFA, defined in Step6, consider one state at a time and obtain the various missing transitions on each input symbol



Construct a Deterministic Finite Automata (DFA) that accepts set of all strings having aa as a substring. Consider the alphabet set as Σ ={a,b}

Step8: For the missing transitions identified in step7, connect this transitions to appropriate state so that the string stated in the problem is accepted.



Transition Diagram

Step9: Write the formal definition of DFA

M= $(Q, \Sigma, \delta, q_0, F)$ where

- $Q = \{q_0, q_1, q_2\}$ Finite set of states
- $\Sigma = \{a, b\}$ Alphabet set or input symbols
- $q_0 = q_0$ Initial (or start) state
- $F = \{q_2\}$ Accepting states (or Final states).
- $\delta: Q \times \Sigma \rightarrow Q$ is a transition function

Transition Table

δ	**a**	**b**
q_0	q_1	q_0
q_1	q_2	q_0
q_2	q_2	q_2

Construct a Deterministic Finite Automata (DFA) that accepts set of all strings having aa as a substring. Consider the alphabet set as $\Sigma = \{a,b\}$

Step10: Consider any one string which will be accepted and one string which should not accepted by this DFA defined in the step9. Show the transition function movements for these strings.

$\delta(q_0, baa)=\delta(q_0, aa)=\delta(q_1, a)= q_2 \in F$, Final state

Hence string baa is accepted

$\delta(q_0, aba)=\delta(q_1, ba)=\delta(q_0, a)=q_1 \notin F$, Final state

Hence string aba is not accepted

Transition Table

δ	**a**	**b**
q_0	q_1	q_0
q_1	q_2	q_0
q_2	q_2	q_2

Questions

Construct DFA's accepting the following strings over the alphabet set $\Sigma = \{0,1\}$

i. The set of all strings ending with 00
ii. The set of all strings with three consecutive 0's atleast once
iii. Set of all strings beginning with 00

Rough Slide to explain

Construct DFA's accepting the following strings over the alphabet set $\Sigma = \{0,1\}$

i. The set of all strings ending with 00

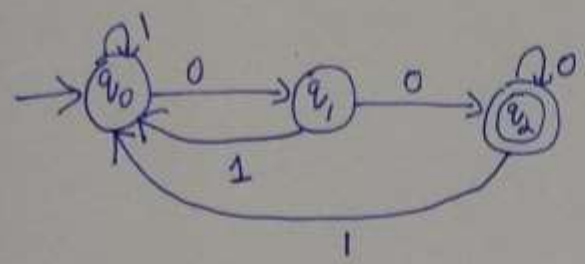
Construct DFA's accepting the following strings over the alphabet set Σ ={0,1}

i. The set of all strings ending with 00

The set of strings ending with 00

$L = \{00, 100, 000, 000100, 010100, \ldots\ldots\}$

$\Sigma = \{\epsilon, 001, 0, 01, 10, 0001, \ldots\ldots\}$



$M = (Q, \Sigma, \delta, q_0, F)$

$Q = \{q_0, q_1, q_2\}$

$\Sigma = \{0, 1\}$

$q_0 = q_0$

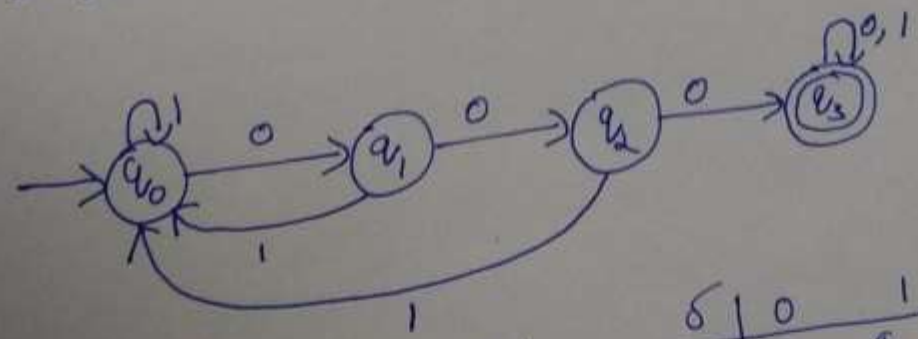
$F = \{q_2\}$

δ	0	1
$\rightarrow q_0$	q_1	q_0
q_1	q_2	q_0
$*q_2$	q_2	q_0

The set of all strings with three consecutive 0's

$L = \{000, 0001, 1000, 00011, 11000, 01000, 001000, \ldots\}$

$\Sigma = \{\epsilon, 0, 00, 01010, \ldots\}$



$M = (Q, \Sigma, \delta, q_0, F)$

$Q = \{q_0, q_1, q_2, q_3\}$

$\Sigma = \{0, 1\}$

$q_0 = q_0$

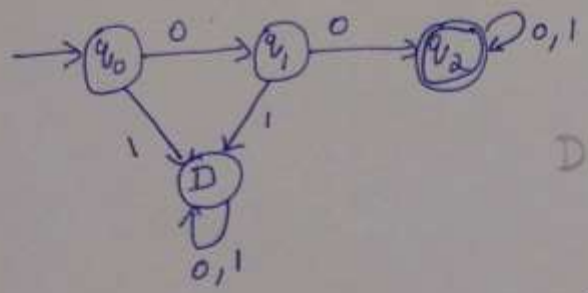
$F = \{q_3\}$

δ	0	1
$\rightarrow q_0$	q_1	q_0
q_1	q_2	q_0
q_2	q_3	q_0
$*q_3$	q_3	q_3

The set of all string beginning with 00

$L = \{00, 001, 0010, 0011, \ldots\}$

$\bar{L} = \{\epsilon, 1, 10, 01, 100, 0100, 1100, \ldots\}$



D: Dead State or Trap State

$M = (Q, \Sigma, \delta, q_0, F)$

$Q = \{q_0, q_1, q_2, D\}$

$\Sigma = \{0, 1\}$

$q_0 = q_0$

$F = \{q_2\}$

δ	0	1
$\rightarrow q_0$	q_1	D
q_1	q_2	D
$* q_2$	q_2	q_2
D	D	D

Questions

Construct DFA's accepting the following strings over the alphabet set Σ ={a,b}

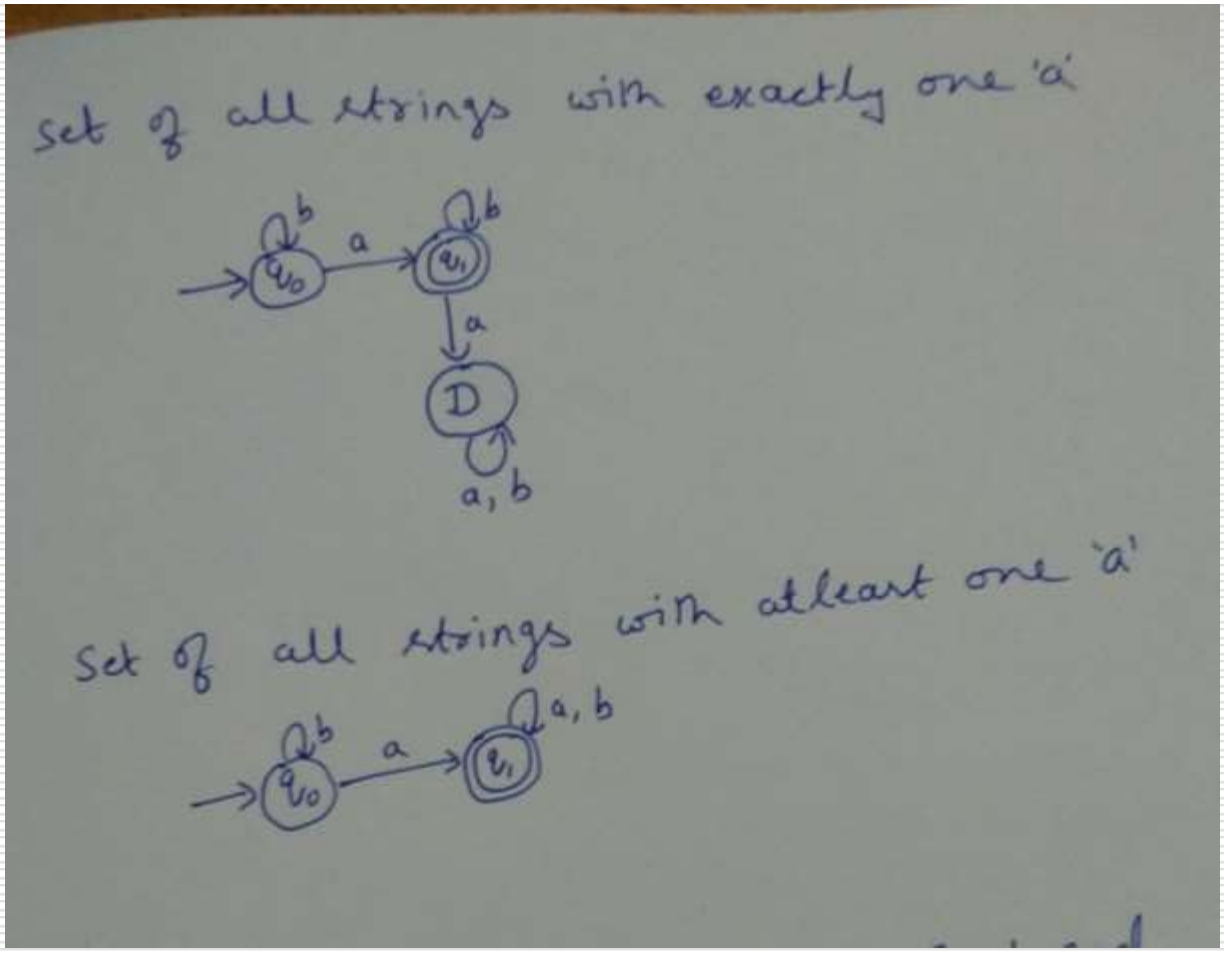
i. The set of all strings with exactly one 'a'
ii. The set of all strings with at least one 'a'
iii. Not more than three a's in the strings
iv. The set of all strings except those which end with 'abb'
v. Set of all strings not containing the substring 'abb'
vi. The set of all strings either begins or ends or both with substring 'ab'
vii. Set of strings ending with ab or ba
viii. Set of all strings with at least one 'a' and exactly two b's

Questions

Construct DFA's accepting the following strings over the alphabet set Σ ={a,b}

i. The set of all strings with exactly one 'a'

ii. The set of all strings with at least one 'a'



Note: For the DFA's,

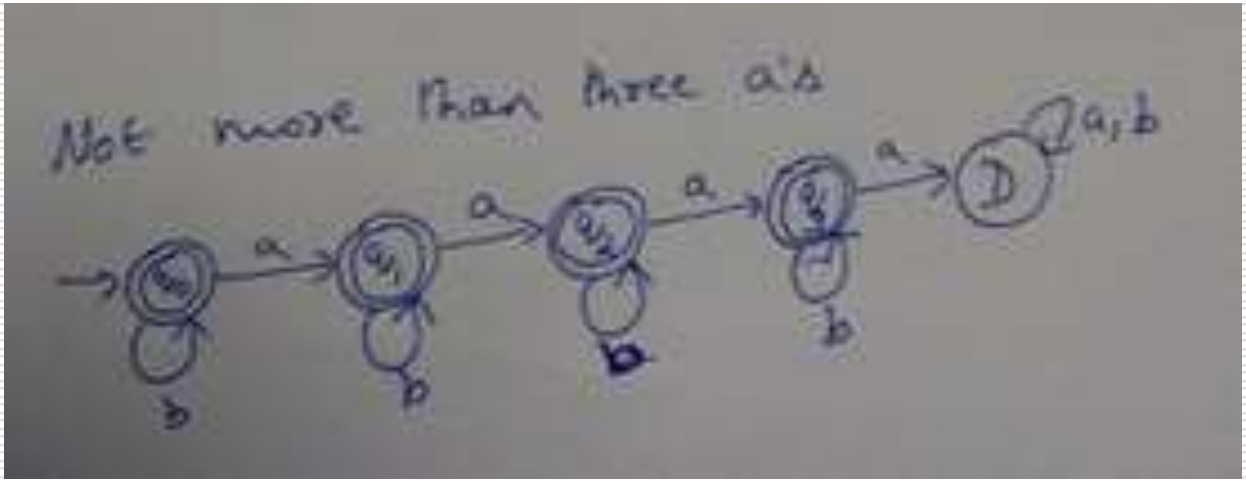
a. Transition diagram
b. Formal definition
c. Transition Table

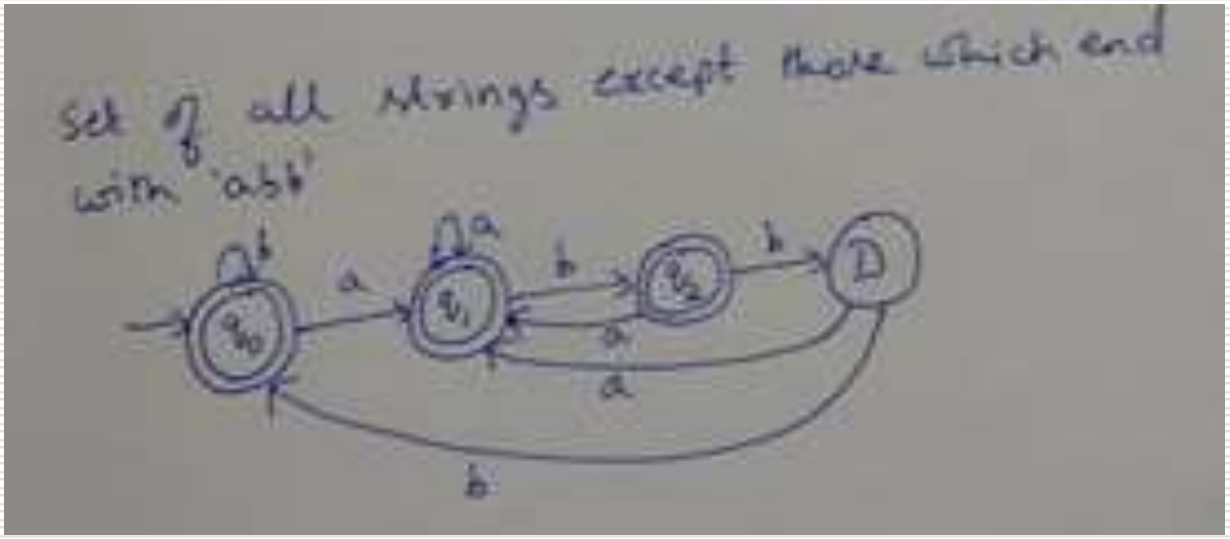
Should written

Questions

Construct DFA's accepting the following strings over the alphabet set Σ ={a,b}

i. The set of all strings with exactly one 'a'
ii. The set of all strings with at least one 'a'
iii. Not more than three a's in the strings
iv. The set of all strings except those which end with 'abb'





Note: For the DFA's,

a. Transition diagram
b. Formal definition
c. Transition Table

Should written

Questions

Construct DFA's accepting the following strings over the alphabet set Σ ={a,b}

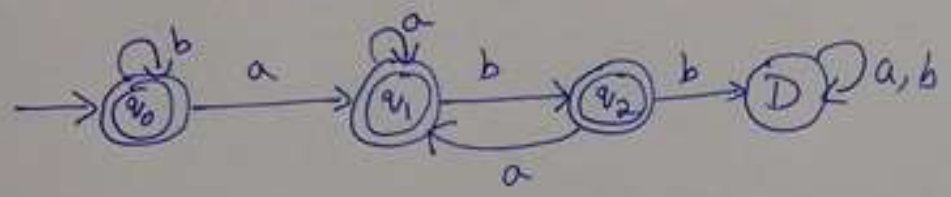
i. The set of all strings with exactly one 'a'
ii. The set of all strings with at least one 'a'
iii. Not more than three a's in the strings
iv. The set of all strings except those which end with 'abb'
v. Set of all strings not containing the substring 'abb'
vi. The set of all strings either begins or ends or both with substring 'ab'

Rough slide to explain

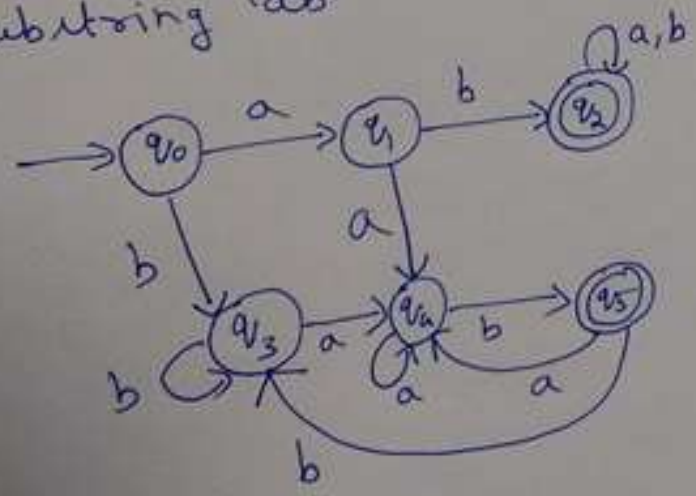
Construct DFA's accepting the following strings over the alphabet set Σ ={a,b}

Set of all strings not containing the substring 'abb'

Set of all strings not containing the substring 'abb'



Either begins or ends or both with substring 'ab'

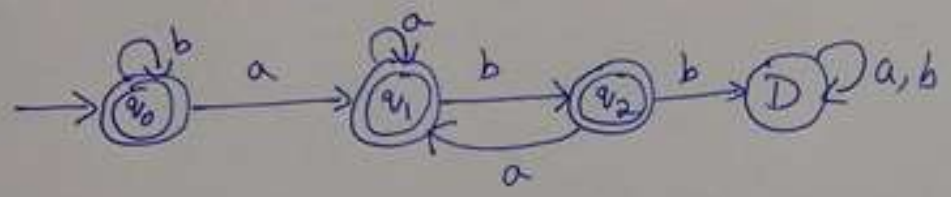


Rough slide to explain

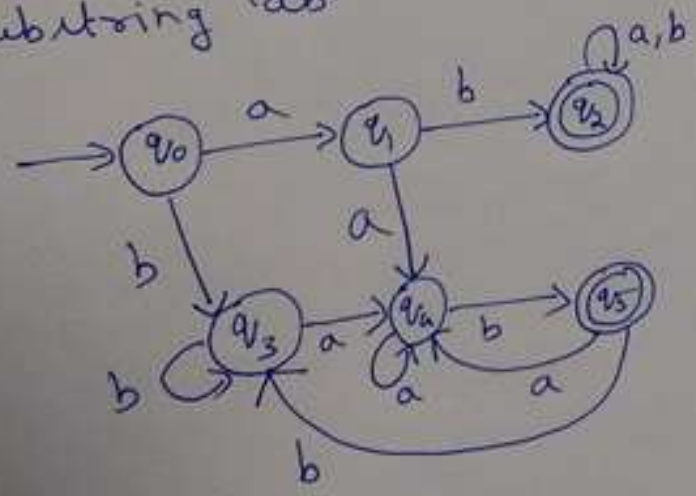
Construct DFA's accepting the following strings over the alphabet set Σ ={a,b}

The set of all strings either begins or ends or both with substring 'ab'

Set of all strings not containing the substring 'abb'



Either begins or ends or both with substring 'ab'



Questions

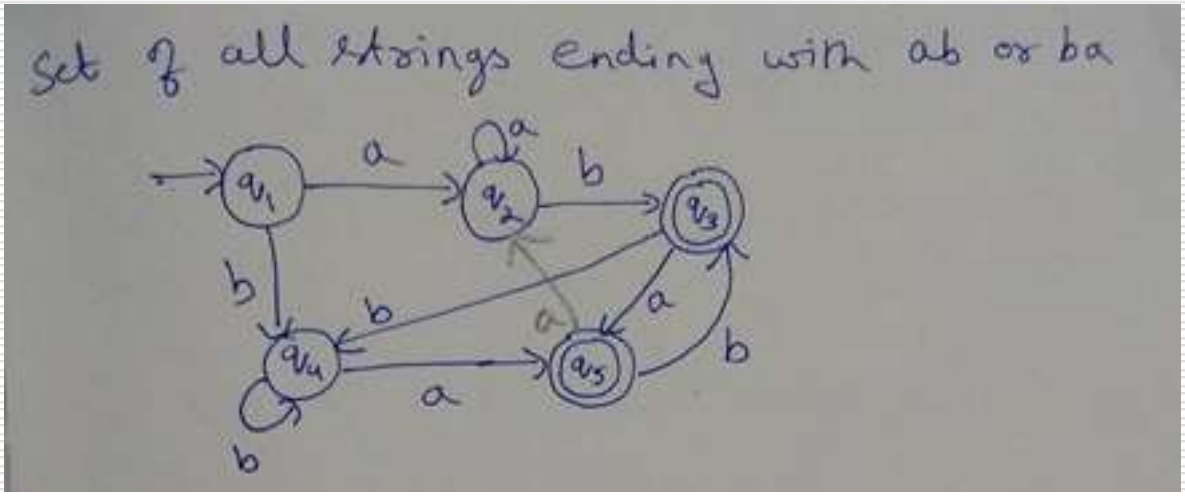
Construct DFA's accepting the following strings over the alphabet set Σ ={a,b}

i. The set of all strings with exactly one 'a'
ii. The set of all strings with at least one 'a'
iii. Not more than three a's in the strings
iv. The set of all strings except those which end with 'abb'
v. Set of all strings not containing the substring 'abb'
vi. The set of all strings either begins or ends or both with substring 'ab'
vii. Set of strings ending with ab or ba
viii. Set of all strings with at least one 'a' and exactly two b's

Rough Slide to explain

Construct DFA's accepting the following strings over the alphabet set Σ ={a,b}

Set of strings ending with ab or ba



Note: For the DFA's,

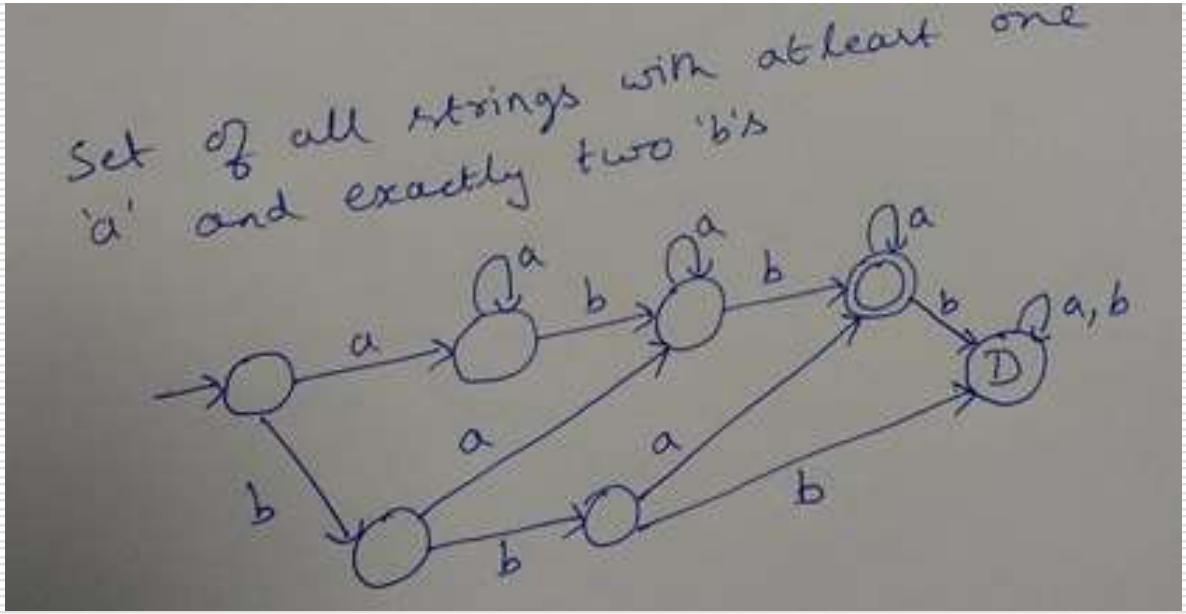
a. Transition diagram
b. Formal definition
c. Transition Table

Should written

Questions

Construct DFA's accepting the following strings over the alphabet set Σ ={a,b}

Set of all strings with at least one 'a' and exactly two b's



Note: For the DFA's,

a. Transition diagram
b. Formal definition
c. Transition Table

Should written

Question

The number of states in the minimal deterministic finite automaton corresponding to the regular expression

(0 + 1)*(10) is _____

(A) 2
(B) 3
(C) 4
(D) 5

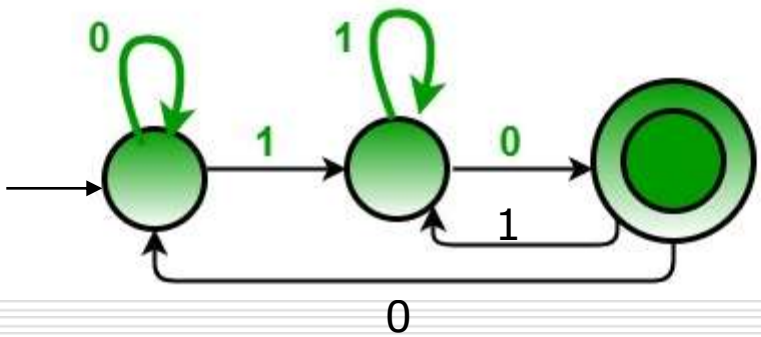
Question

The number of states in the minimal deterministic finite automaton corresponding to the regular expression (0 + 1)*(10) is _____

(A) 2
(B) 3
(C) 4
(D) 5

Answer: (B)

below DFA for the same

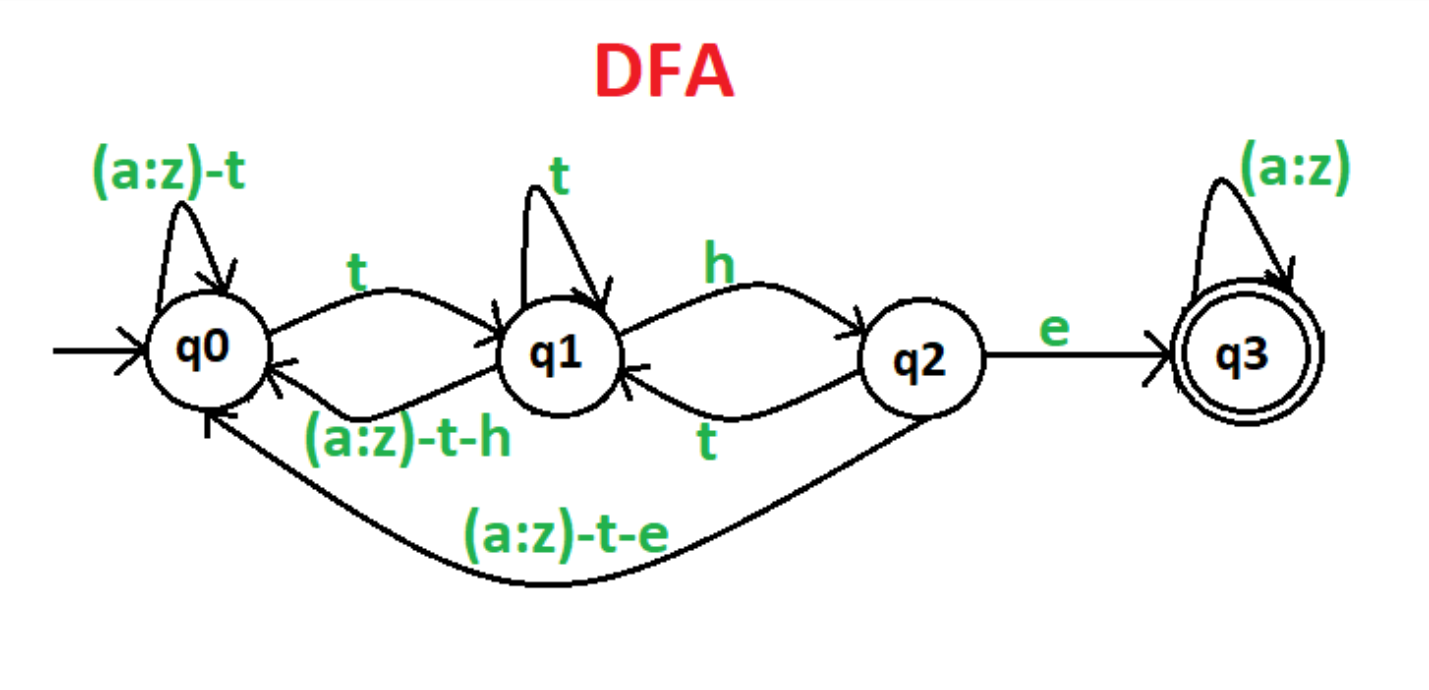


Question

Draw a deterministic finite automata which accept a string containing “the” anywhere in a string of $\Sigma = \{a\text{-}z\}$, e.g., “there” but not “those”.

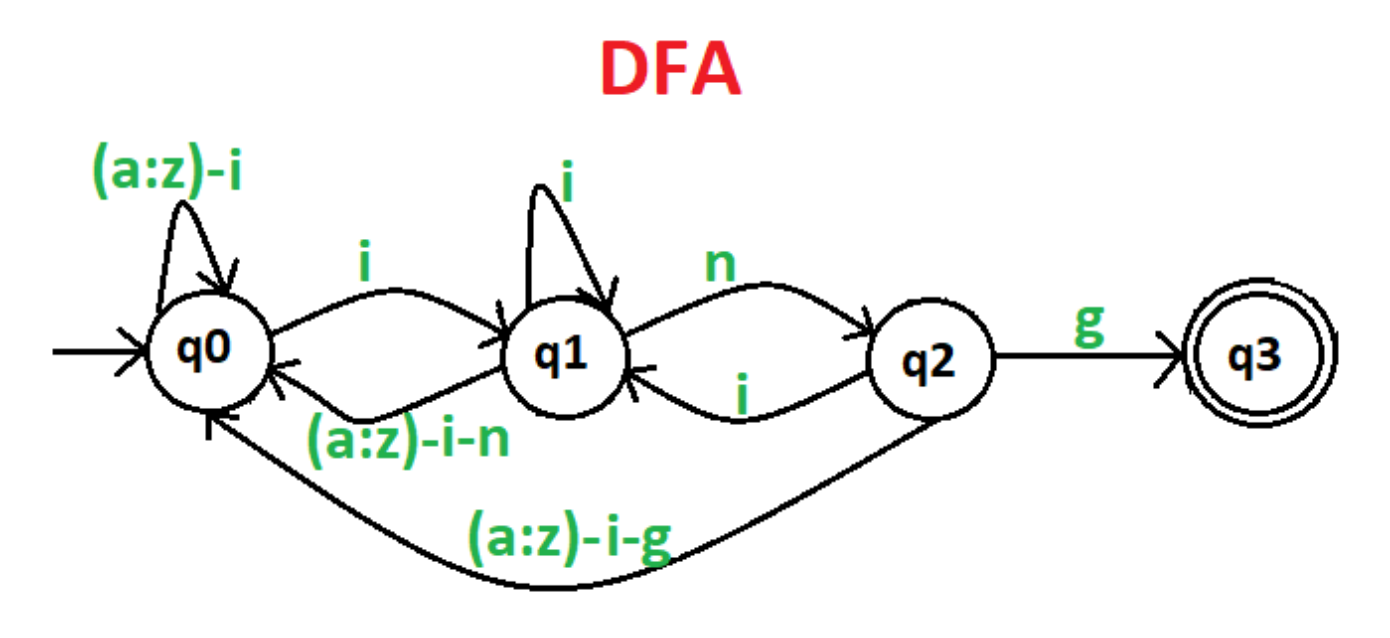
Question

Draw a deterministic finite automata which accept a string containing “the” anywhere in a string of {a-z}, e.g., “there” but not “those”.



Question

Draw a deterministic and non-deterministic finite automata which accept a string containing “ing” at the end of a string in a string of {a-z}, e.g., “anything” but not “anywhere”.



Questions

Construct DFA for the following languages over the alphabet set $\Sigma = \{a, b\}$

i. $L=\{awa \ / \ w \in (a+b)^* \}$

ii. $L=\{w \ / \ n_a(w) \bmod 3 == 0 \}$

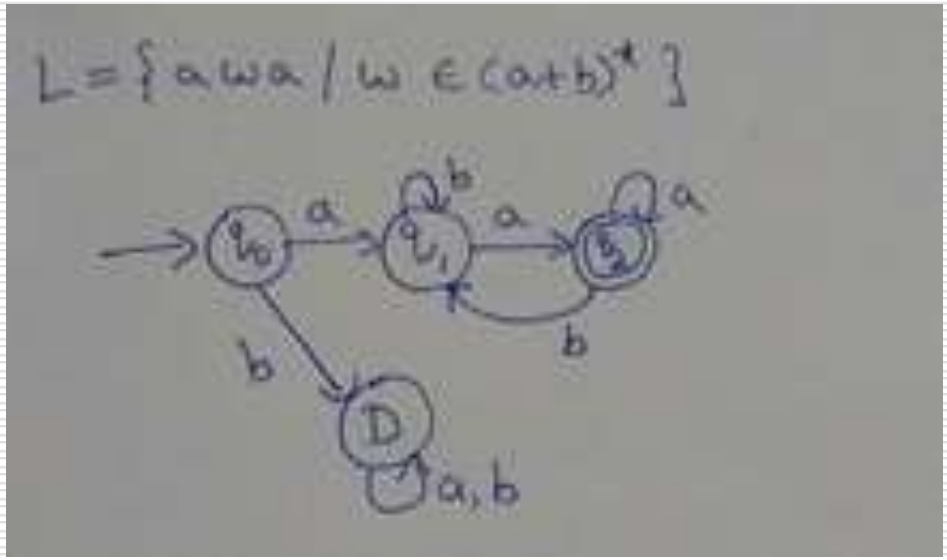
iii. $L=\{w \ / \ |w| \bmod 3 == 0 \}$

iv. $L=\{w \ / \ |w| \bmod 5 \neq 0 \}$

Questions

Construct DFA for the following languages over the alphabet set $\Sigma = \{a,b\}$

i. $L=\{awa \ / \ w \in (a+b)^* \}$



Note: For the DFA's,

a. Transition diagram
b. Formal definition
c. Transition Table

Should written

Rough Slide to explain

Construct DFA for the following languages over the alphabet set $\Sigma = \{a, b\}$

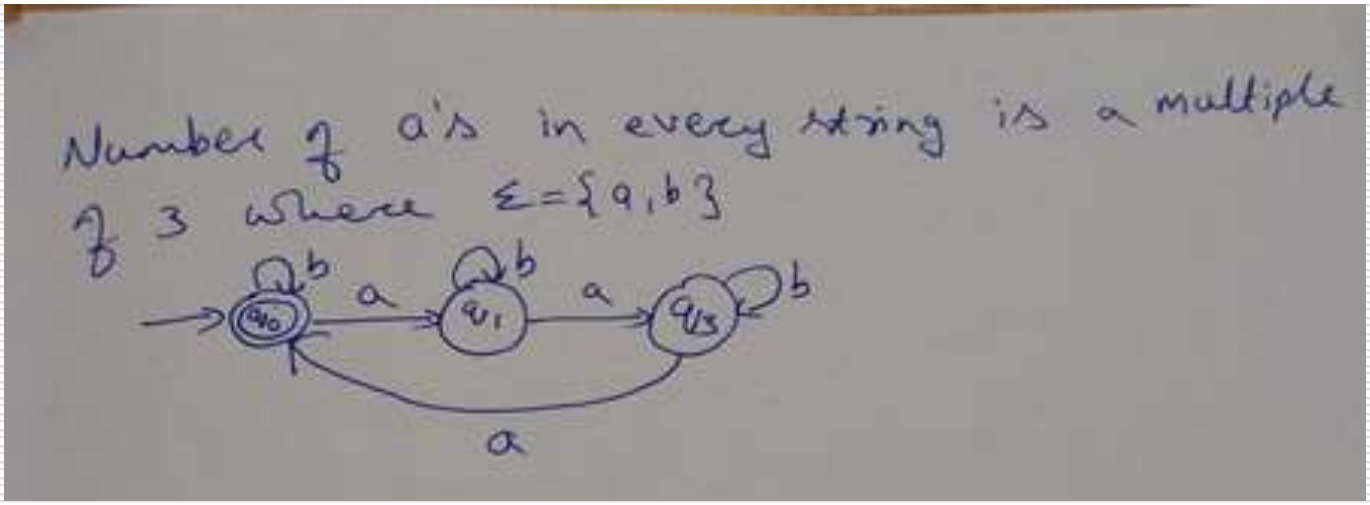
$L=\{w \ / \ n_a(w) \bmod 3 == 0 \}$

Questions

Construct DFA for the following languages over the alphabet set Σ ={a, b}

i. L={awa / w ∈ $(a+b)^*$ }

ii. L={w / $n_a(w)$ mod 3 ==0 }



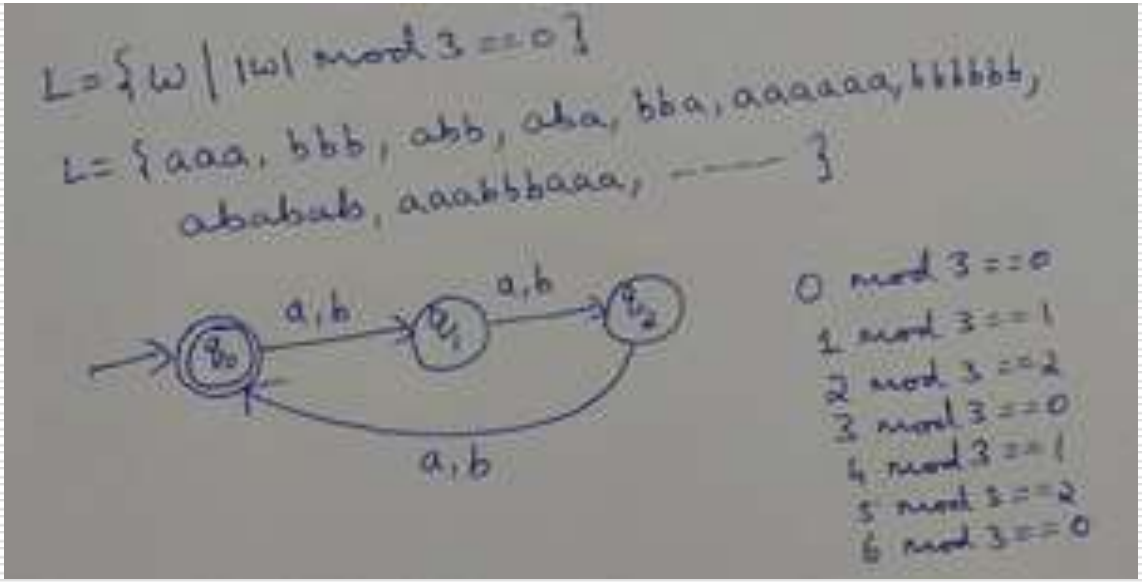
Questions

Construct DFA for the following languages over the alphabet set $\Sigma = \{a, b\}$

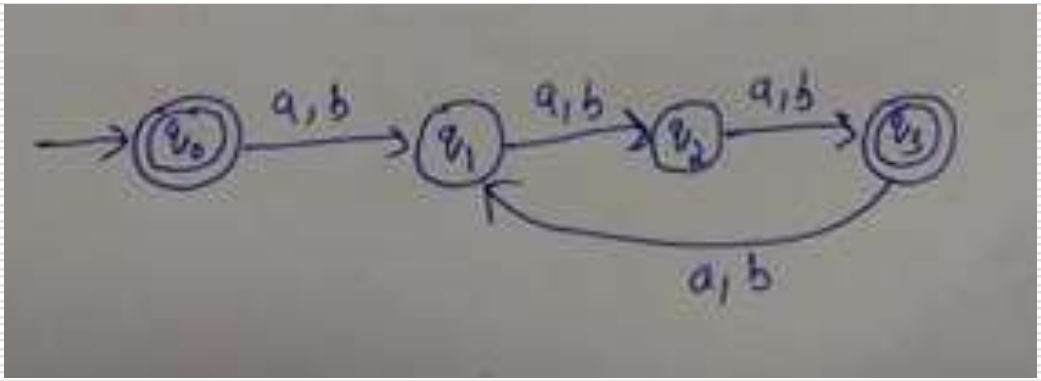
i. $L=\{awa \ / \ w \in (a+b)^* \}$

ii. $L=\{w \ / \ n_a(w) \bmod 3 == 0 \}$

iii. $L=\{w \ / \ |w| \bmod 3 == 0 \}$



OR



Note: For the DFA's,

a. Transition diagram
b. Formal definition
c. Transition Table

Should written

Questions

Construct DFA for the following languages over the alphabet set $\Sigma = \{a, b\}$

i. $L=\{awa \ / \ w \in (a+b)^* \}$

ii. $L=\{w \ / \ n_a(w) \bmod 3 == 0 \}$

iii. $L=\{w \ / \ |w| \bmod 3 == 0 \}$

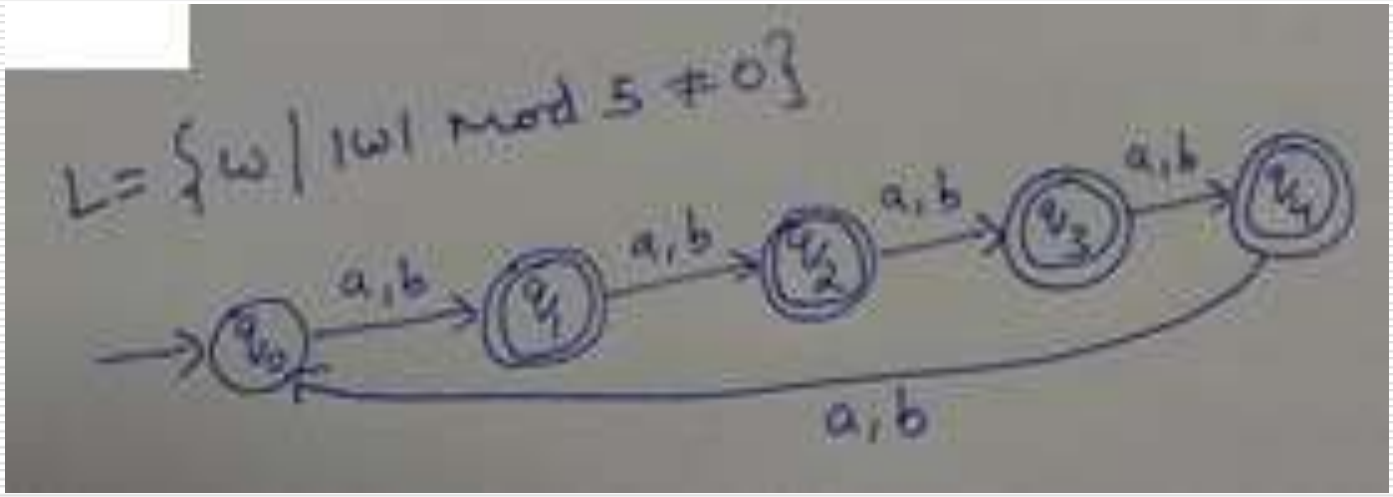
iv. $L=\{w \ / \ |w| \bmod 5 \neq 0 \}$

Rough Slide to explain

Construct DFA for the following languages over the alphabet set $\Sigma = \{a, b\}$

$L=\{w \ / \ |w| \bmod 5 \neq 0 \}$

$L=\{w \ / \ |w| \bmod 5 == 0 \}$



Note: For the DFA's,

a. Transition diagram
b. Formal definition
c. Transition Table

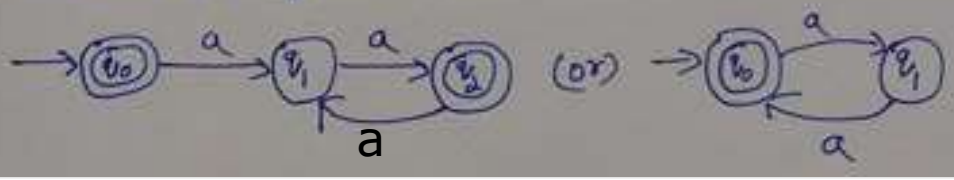
Should written

Question

1. Construct DFA to accept **even** number of a's over the alphabet set Σ ={a}
2. Construct DFA to accept **odd** number of a's over the alphabet set Σ ={a}

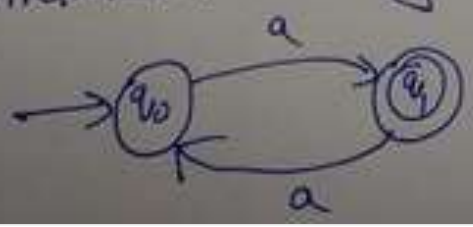
DFA to accept even no of a's over $\Sigma=\{a\}$

Minimum string is aa



DFA to accept odd no of a's over $\Sigma=\{a\}$

Minimum string to accept 'a'



Questions

Construct DFA for the following language over the alphabet set $\Sigma = \{a, b\}$

$L=\{w \,/\, w \in (a+b)^*, n_a(w) \bmod 2 == 0 \text{ and } n_b(w) \bmod 2 == 0\}$

OR

$L=\{w \,/\, w \in (a+b)^*, N_a(w)$ as well as $N_b(w)$ is even $\}$

OR

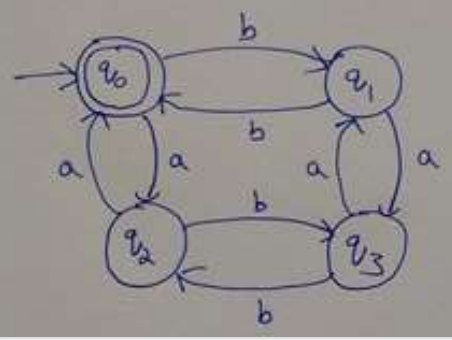
To accept strings of a's and b's with even number of a's and b's

Rough slide to explain

Construct DFA for the following language over the alphabet set $\Sigma = \{a, b\}$

$L=\{w \ / \ w \in (a+b)^*, \ n_a(w) \bmod 2 == 0 \text{ and } n_b(w) \bmod 2 == 0 \}$

DFA to accept even number of a's and even number of b's over $\Sigma = \{a, b\}$

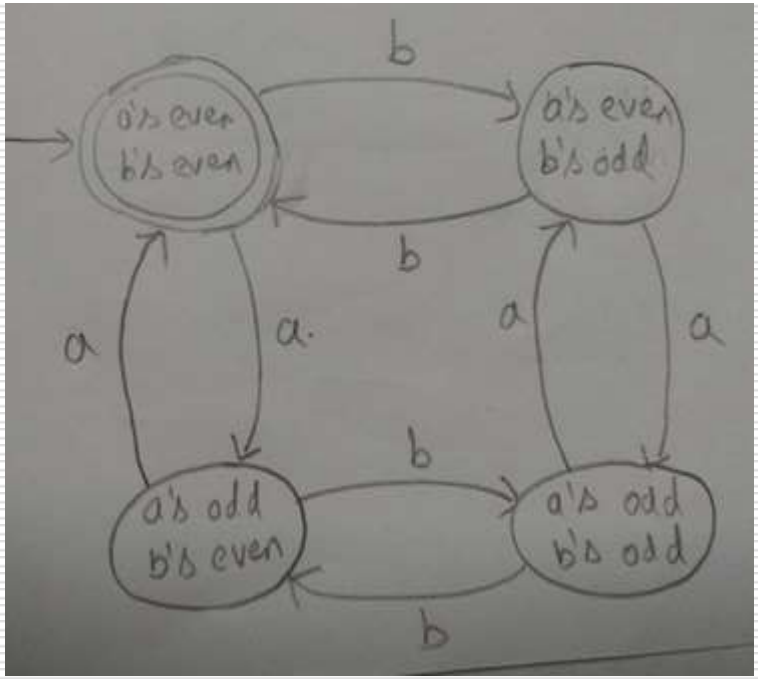


Note:

(i) To accept odd number of a's and odd number of b's, only state q_3 should be final state.

(ii) To accept even number of a's and odd number of b's, only state q_1 should be final state.

(iii) To accept odd number of a's and even number of b's, only state q_2 should be final state.



Question

Consider the DFA with the following transition table:

	0	1
$\rightarrow A$	A	B
$*B$	B	A

Informally describe the language accepted by this DFA

Question

Consider the DFA with the following transition table:

	0	1
$\rightarrow A$	A	B
$*B$	B	A

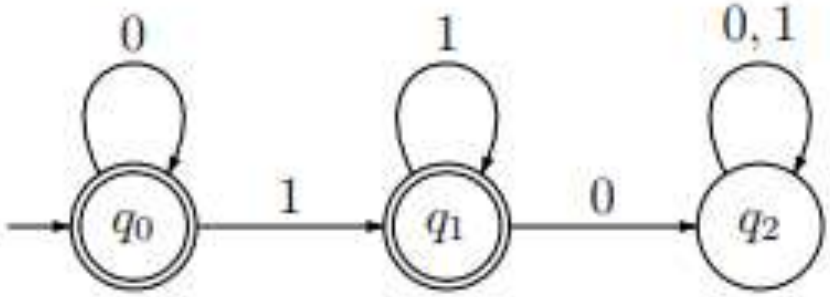
Informally describe the language accepted by this DFA

Answer

- You should draw the transition graph before reading this answer; it will help with your understanding.
- We claim that the language of the DFA is {w | w contains an odd number of 1's}

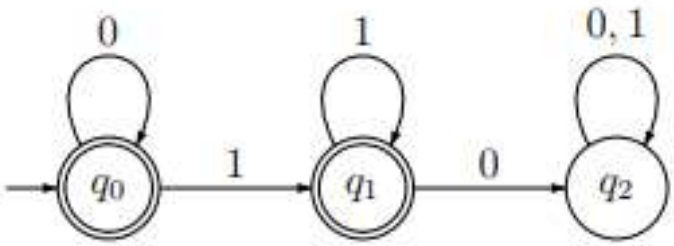
Question

What is the language recognized by the following DFA:



Question

What is the language recognized by the following DFA:



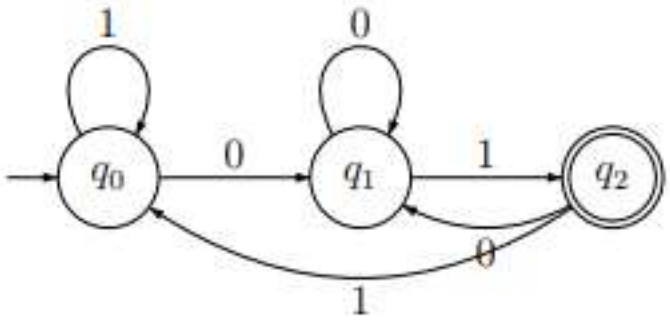
Solution: $L = \{0^m 1^n \mid m \geq 0, n \geq 0\}$.
Meaning of the states:
The meaning of the states:

- q_0 - the string the DFA read so far is a sequence of 0's
- q_1 - the string the DFA read so far is a sequence of 0's followed by the sequence of 1's
- q_2 - trap state

Alternative answer: $L_2 = \{w \mid w$ does not contain 10 as a subsequence $\}$. One of the ways to convince yourself that L_2 is the language recognized by this DFA, is to construct a DFA for $\overline{L_2} = \{w \mid w$ contains 10 as a subsequence $\}$, and then switch the accepting/rejecting states

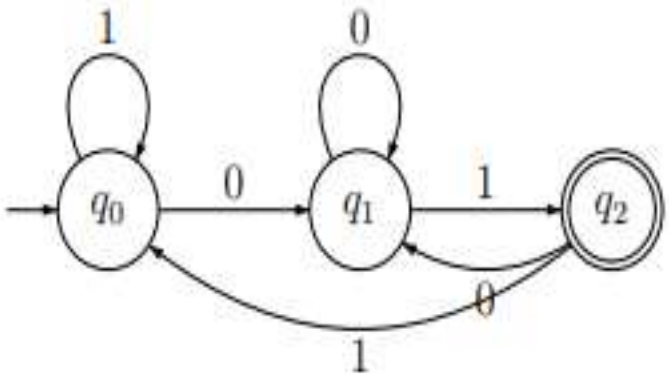
Question

What is the language recognized by the following DFA:



Question

What is the language recognized by the following DFA:



Solution: $L = \Sigma^*\{01\}$, that is all binary strings that end with 01.
To see this, notice that we get to q_2 only if we read a sequence 01. At which point, if no symbols are read we accept, otherwise we continue waiting for 01.

Question

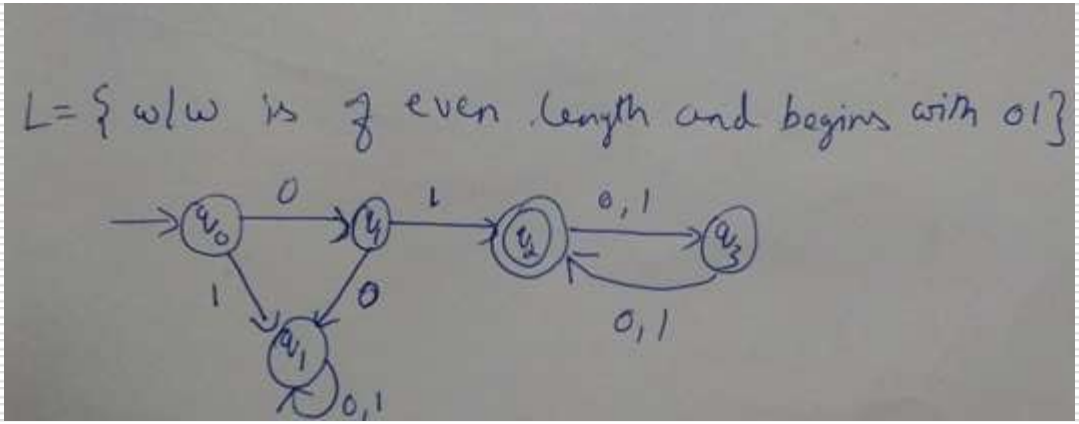
Construct DFA for the following language over the alphabet set Σ ={0, 1}

L={w / w is even length and begins with '01'

Question

Construct DFA for the following language over the alphabet set Σ ={0, 1}

L={w / w is even length and begins with '01'



Question

Obtain DFA to accept strings of a's and b's such that

$L=\{w \,/\, w \in (a+b)^* \text{ such that}$

$N_a(w) \bmod 3 == 0 \text{ and } N_b(w) \bmod 2==0\}$

Rough Slide to explain

Obtain DFA to accept strings of a's and b's such that

$L=\{w \,/\, w \in (a+b)^* \text{ such that}$

$N_a(w) \bmod 3 == 0 \text{ and } N_b(w) \bmod 2==0\}$

Possible Remainders

$N_a(w) \bmod 3$: { 0, 1, 2}

$N_b(w) \bmod 2$: { 0, 1}

So possible states of DFA

is given by cross product

Of {0,1,2} x {0,1}

$N_a(w) \bmod 3$ can be 0, 1, 2

$N_b(w) \bmod 2$ can be 0, 1

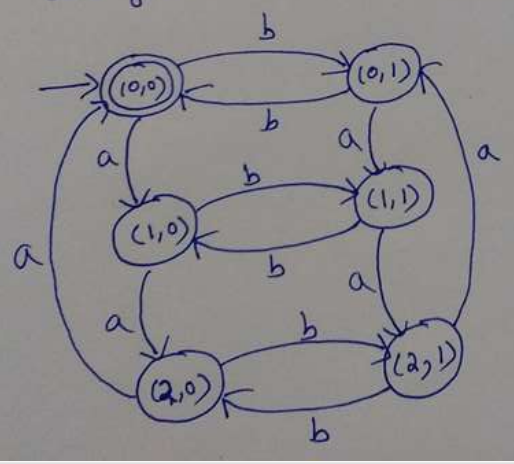
So the possible states of DFA is given by

$N_a(w) \bmod 3 \times N_b(w) \bmod 2$,

$N_a(w) \bmod 3 = \{0, 1, 2\}$

$N_b(w) \bmod 2 = \{0, 1\}$

cross product $= \{(0,0), (0,1), (1,0), (1,1), (2,0), (2,1)\}$ are the possible states of the DFA with (0,0) as the start state and (0,0) as final state.



Question

Obtain DFA to accept strings of a's and b's such that

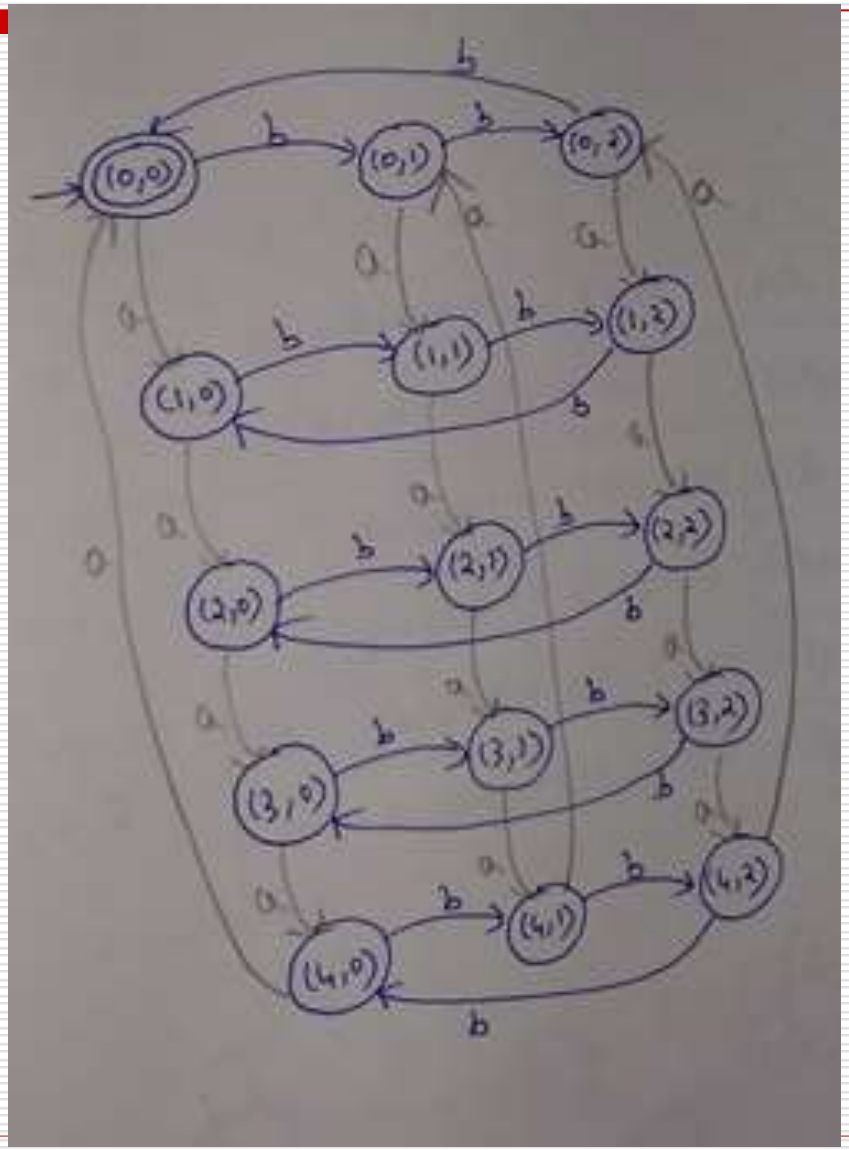
$L=\{w \ / \ w \in (a+b)^*$ such that

$N_a(w) \bmod 5 == 0$ and $N_b(w) \bmod 3 == 0\}$

$n_a(w) \bmod 5 = \{0, 1, 2, 3, 4\}$

$n_b(w) \bmod 3 = \{0, 1, 2\}$

$\{(0,0)\ (0,1)\ (0,2),$
$(1,0), (1,1), (1,2),$
$(2,0), (2,1), (2,2)$
$(3,0), (3,1), (3,2)$
$(4,0), (4,1), (4,2)\}$



Question

Construct a DFA which accepts strings of 0's and 1's where the values of each string is represented as a binary number only. Only the strings representing zero modulo three should be accepted. For example 0, 11, 011, 110, 1001, ……

OR

Design a DFA to accept the binary numbers which are divisible by 3

Design a DFA to accept the binary numbers which are divisible by 3

r is radix, For binary r=2

i is remainder obtained after dividing by k

d represents digits

k is divisor

r=2

i={0,1,2}

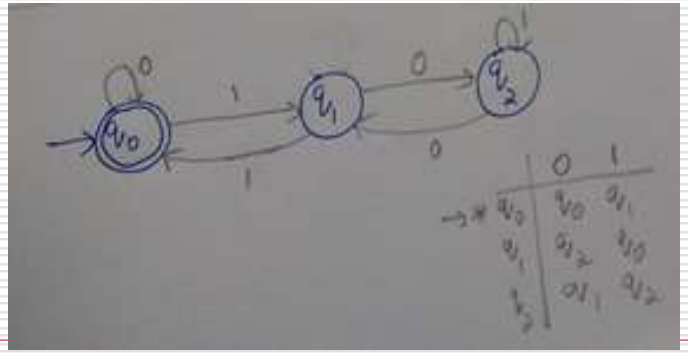
d={0,1}

k=3

We will use the relation, $\delta(q_i , a)=q_j$ where $j=(r*i+d) \bmod k$

Design a DFA to accept the binary numbers which are divisible by 3

Remai -nder	d	**j=(r*i+d) mod k**	**$\delta(q_i , a)=q_j$**
i=0	0	j=(2*0+0) mod 3=0	$\delta(q_0 , 0)=q_0$
	1	j=(2*0+1) mod 3=1	$\delta(q_0 , 1)=q_1$
i=1	0	j=(2*1+0) mod 3=2	$\delta(q_1 , 0)=q_2$
	1	j=(2*1+1) mod 3=0	$\delta(q_1 , 1)=q_0$
i=2	0	j=(2*2+0) mod 3=1	$\delta(q_2 , 0)=q_1$
	1	j=(2*2+1) mod 3=2	$\delta(q_2 , 1)=q_2$

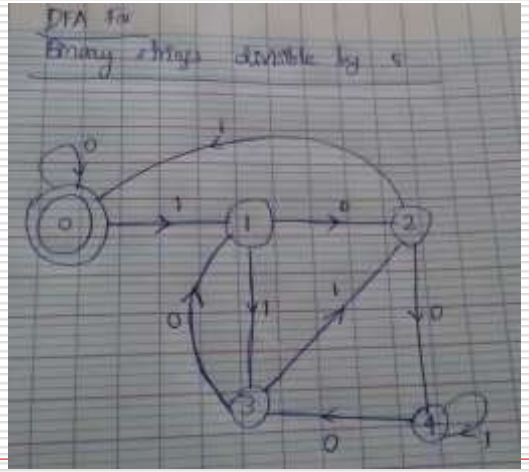


Question

Construct a DFA which accepts strings of 0's and 1's where the values of each string is represented as a binary number only. Only the strings representing zero modulo five should be accepted. For example 0, 101, 0101, 1010, 1111, 1101,……

OR

Design a DFA to accept the binary numbers which are divisible by 5



Question

The password to the admins account=”administrator”. The total number of states required to make a password-pass system using DFA would be _____

a) 14 states
b) 13 states
c) 12 states
d) A password pass system cannot be created using DFA

Question

The password to the admins account=”administrator”. The total number of states required to make a password-pass system using DFA would be _____

a) 14 states
b) 13 states
c) 12 states
d) A password pass system cannot be created using DFA

Answer: a

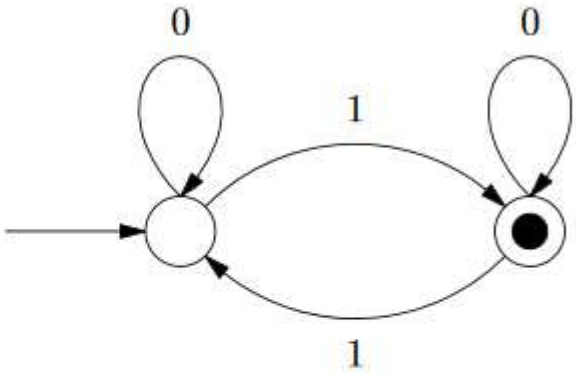
Explanation: For a string of n characters with no repetitive substrings, the number of states required to pass the string is n+1.

DFA Exercises

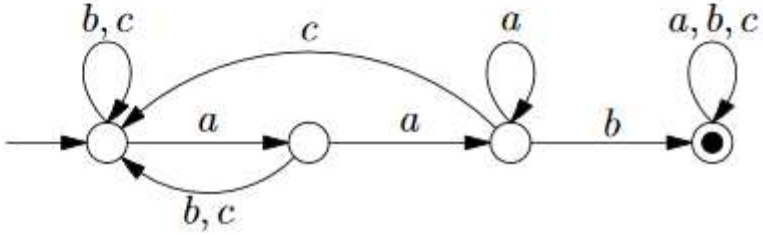
1. Give a DFA for $\Sigma = \{0, 1\}$ and strings that have an odd number of 1's and any number of 0's.
2. Give a DFA for $\Sigma = \{a, b, c\}$ that accepts any string with aab as a substring.
3. Give a DFA for $\Sigma = \{a, b\}$ that accepts any string with $aababb$ as a substring.

Solutions

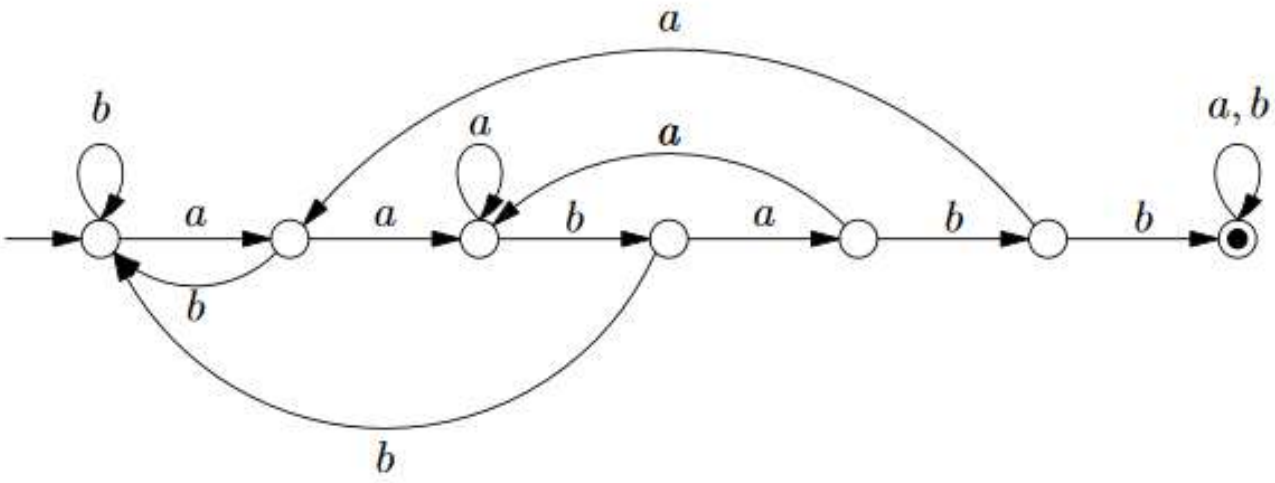
1.



2.



3.

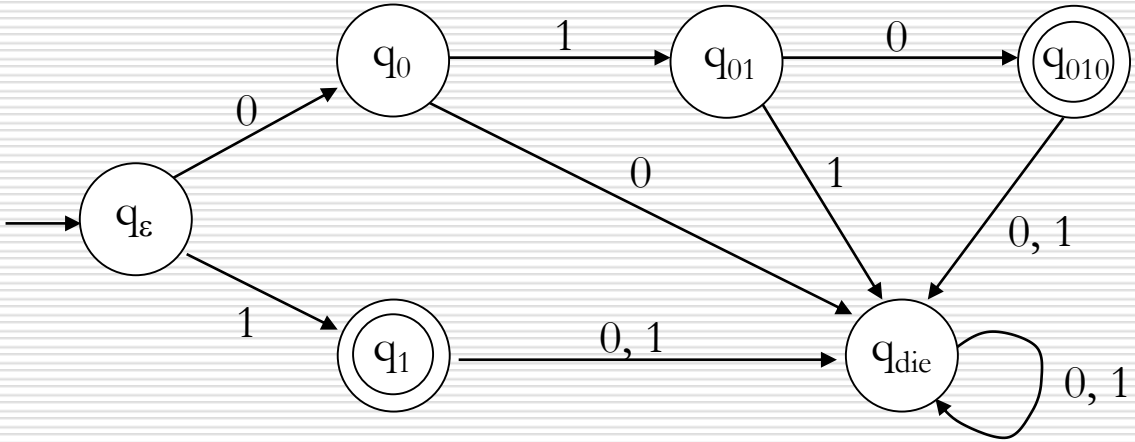


Examples

- Construct a DFA that accepts the language

$$L = \{010, 1\} \qquad (\Sigma = \{0, 1\})$$

- Answer



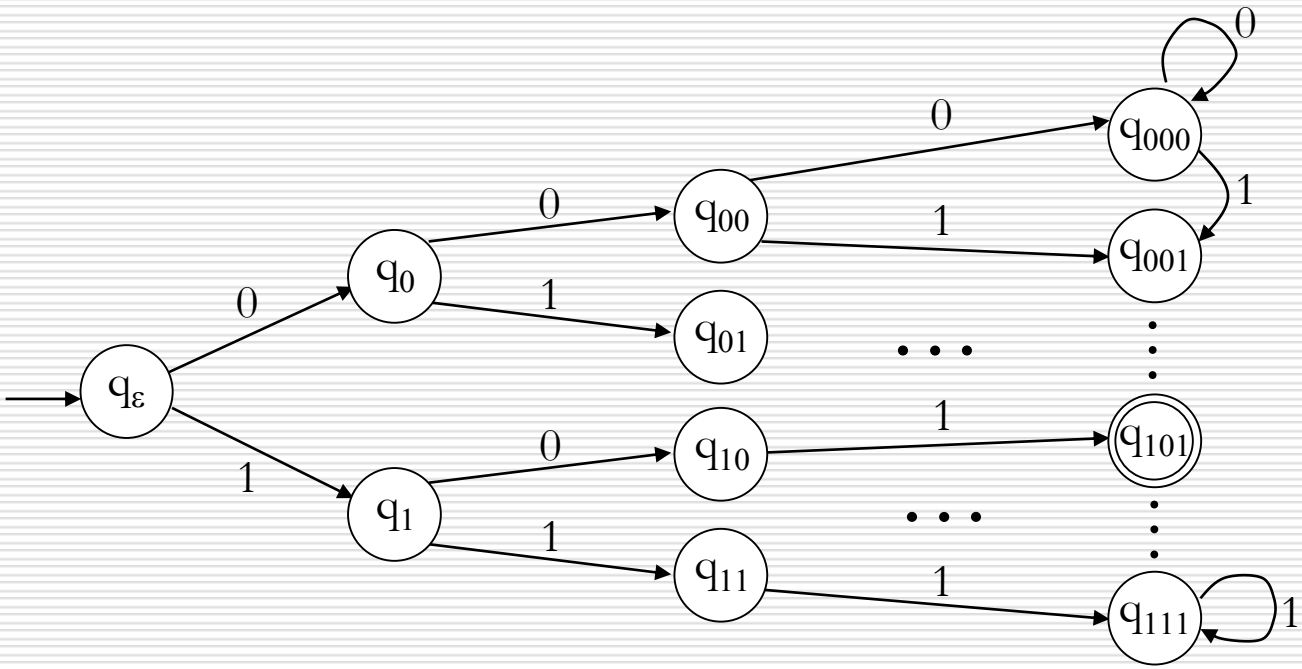
Examples

- Construct a DFA over alphabet $\{0, 1\}$ that accepts all strings that end in 101

- Hint: The DFA must "remember" the last 3 bits of the string it is reading

Examples

- Construct a DFA over alphabet $\{0, 1\}$ that accepts all strings that end in 101
- Sketch of answer:

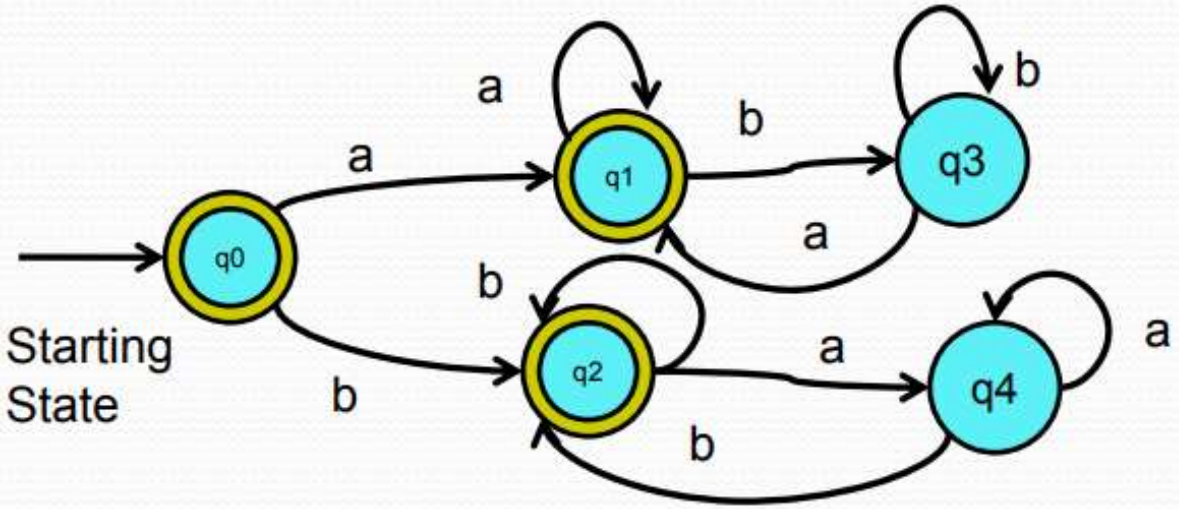


Problem :-

Input :-Language over $\sum=\{a,b\}^*$,such that every string starts and ends with the same symbol.

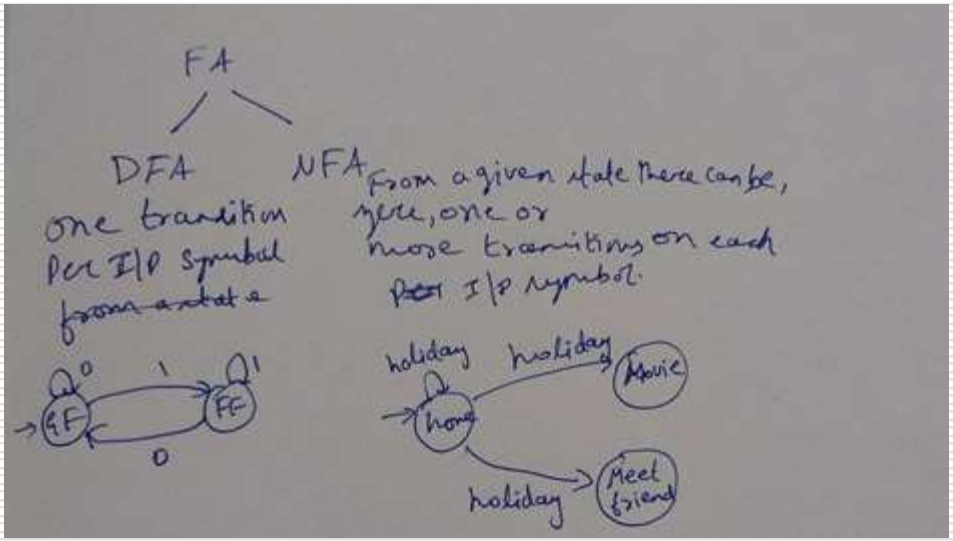
Solution:-

DFA representing the above problem:-



Unit-1

Nondeterministic Finite Automata (NFA)



Rough slide to explain Nondeterministic Finite Automata (NFA)

DFA: Set of all strings ending with **ab** over Σ ={a, b}

NFA: Set of all strings ending with **ab** over Σ ={a, b}

Nondeterministic Finite Automata

- Non determinism gives a machine multiple options for its moves.

In a ***nondeterministic*** finite automaton (NFA), for each state there can be zero, one, two, or more transitions corresponding to a particular symbol.

If NFA gets to state with more than one possible transition corresponding to the input symbol, we say it ***branches***.

If NFA gets to a state where there is no valid transition, then that branch ***dies***.

Nondeterministic Finite Automata

Suppose in life,

- Whenever you had a choice, you could try both possibilities and live your life.
- At the end, you would go back and choose the one that worked out the

This is the idea of nondeterminism for finite automata. It's not practical to build devices like this, but it does help in applications of finite automata to make use of this concept.

- In a deterministic automaton, in a state s with an input a there is one and only one state t that it can go to in the next time instant.
- In a nondeterministic automaton, there can be many states $t_1, t_2, \ldots, t_n$ that it can go to, or possibly no states at all.
- Also, nondeterministic automata have the possibility to go from one state to another without reading any input, if there is an ϵ transition between the two states.

Formal Definition of an NFA

An NFA can be represented by a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where –

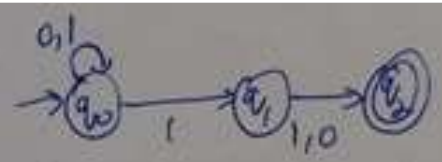
- **Q** is a finite set of states.
- **Σ** is a finite set of symbols called the alphabets.
- **δ** is the transition function where $\delta: Q \times \Sigma \rightarrow 2^Q$

  (Here the power set of Q (2^Q) has been taken because in case of NFA, from a state, transition can occur to any combination of Q states)
- **q_0** is the initial state from where any input is processed ($q_0 \in Q$).
- **F** is a set of final state/states of Q ($F \subseteq Q$).

Rough Slide to explain NFA formal definition

NFA: Set of all strings ending with ab over $\Sigma = \{a,b\}$

Example of NFA



$M = (Q, \Sigma, \delta_{NFA}, q_0, F)$

$Q = \{q_0, q_1, q_2\}$

$\Sigma = \{0, 1\}$

$q_0 = q_0$

$F = \{q_2\}$

$\delta_{NFA} : Q \times \Sigma \to 2^Q$

δ_{NFA}	0	1
q_0	$\{q_0\}$	$\{q_0, q_1\}$
q_1	$\{q_2\}$	$\{q_2\}$
q_2	$\emptyset$	$\emptyset$

2^Q Power set

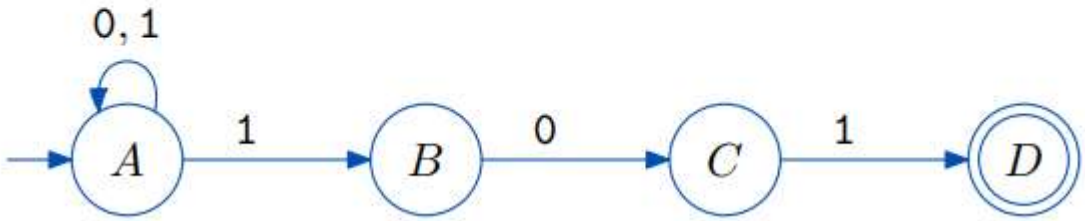
$Q = \{q_0, q_1\}$

$2^Q = \{\emptyset, \{q_0\}, \{q_1\}, \{q_0, q_1\}\}$

$\delta_{DFA} : Q \times \Sigma \to Q$

NFA

An NFA that accepts all binary strings that end with 101.



Unit-1

Equivalence of NFA and DFA (or) Converting NFA to DFA

Converting NFA to DFA using subset construction method

Subset Construction Procedure

Given an NFA $M_{NFA} = (Q_{nfa}, \Sigma, \delta_{nfa}, q_0, F_{nfa})$ which accepts the language L, find an equivalent DFA $M_{DFA} = (Q_{dfa}, \Sigma, \delta_{dfa}, q_0, F_{dfa})$ such that

$L(M_{NFA}) = L(M_{DFA})$

Step-1: Identify the start state of the DFA . Since q_o is the start state of NFA, $\{q_0\}$ is the start state of DFA

Step-2: Identify the alphabets of DFA. The input alphabets of NFA are the input alphabets of DFA

Step-3: Identify the Q_{DFA} , which are the states of DFA

The set of subsets of Q_{NFA} will be the states of DFA Q_{DFA}

Example:

$Q_{NFA} = \{q_0 , q_1 , q_2 \}$

$Q_{DFA} = \{\emptyset , \{q_0\} , \{q_1\} , \{q_2\}, \{q_0 , q_1 \} , \{q_0 , q_2 \} \ldots\ldots\}$

Step-4: Identify the final states of DFA

If $\{q_i , q_j, \ldots q_k\}$ is the state of DFA then $\{q_i , q_j, \ldots q_k\}$ will be final state of DFA provided one of $q_i , q_j, \ldots q_k$ is the final state of NFA

Step-5: Identifying transitions of DFA

For each state $\{q_i , q_j, \ldots q_k\}$ in QDFA and for each input symbol 'a' in Σ , the transition can be obtained as shown below

$\delta_{DFA} (\{q_i , q_j, \ldots q_k\} , a) = \{\delta_{NFA} (q_i, a) \cup \delta_{NFA} (q_j, a) \ldots\ldots\ldots\}$

Converting NFA to DFA using subset construction method

Subset Construction Procedure

Given an NFA $M_{NFA} = (Q_{NFA}, \Sigma, \delta_{NFA}, q_0, F_{NFA})$ which accepts language L, Find an equivalent DFA $M_{DFA} = (Q_{DFA}, \Sigma, \delta_{DFA}, \{q_0\}, F_{DFA})$ such that $L(M_{NFA}) = L(M_{DFA})$

Step1: Identify the start state of DFA

Since q_0 is the start state of NFA, $\{q_0\}$ is the start state of DFA

Step2: Identify the alphabets of DFA

The I/P alphabets of DFA are the I/P characters of NFA

Step3: Identify Q_{DFA}, which are the states of DFA

The set of subsets of Q_{NFA} will be the states of DFA Q_{DFA}

Ex: $Q_{NFA} = \{q_0, q_1, q_2\}$

$Q_{DFA} = \{\phi, \{q_0\}, \{q_1\}, \{q_2\}, \{q_0, q_1\} \ldots\}$

Step4: Identify final states of DFA

If $\{q_i, q_j, \ldots q_k\}$ is a state in Q_{DFA}, then $\{q_i, q_j, \ldots q_k\}$ will be the final states of DFA provided one of $q_i, q_j, \ldots q_k$ is the final state of NFA

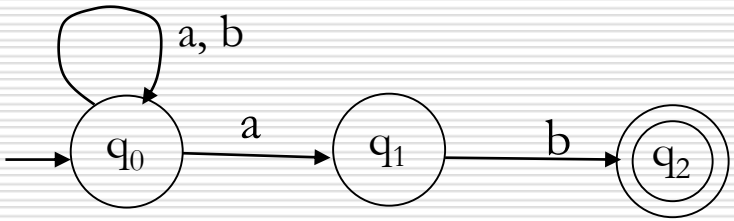
Step5: Identifying transitions of DFA

For each state $\{q_i, q_j \ldots q_k\}$ in Q_{DFA} and for each I/P symbol 'a' in Σ, the transition can be obtained as shown below

$\delta_{DFA}(\{q_i, q_j \ldots q_k\}, a) = \{\delta_{NFA}(q_i, a) \cup \delta_{NFA}(q_j, a) \ldots\}$

Rough Slide to explain: Obtain the NFA to accept strings of a's and b's ending with **ab** and convert into equivalent DFA

NFA to accept strings ending with **ab**



$M_{NFA} = (Q_{nfa}, \Sigma, \delta_{nfa}, q_0, F_{nfa})$

$Q_{NFA} = \{q_0, q_1, q_2\}$

$\Sigma = \{a, b\}$, $q_0 = q_0$, $F_{NFA} = \{q_2\}$

$\delta_{NFA}: Q \times \Sigma \rightarrow 2^Q$

Transition Table

δ_{NFA}	**a**	**b**
q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
q_2	$\emptyset$	$\emptyset$

Converting NFA to DFA

$M_{DFA} = (Q_{dfa}, \Sigma, \delta_{dfa}, q_0, F_{dfa})$

$\Sigma = \{a, b\}$, $q_0 = \{q_0\}$

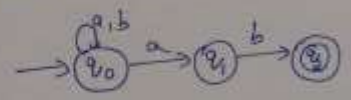
$\delta_{DFA}(\{q_i, q_j, \ldots q_k\}, a) = \{\delta_{NFA}(q_i, a) \cup \delta_{NFA}(q_j, a) \ldots\ldots\}$

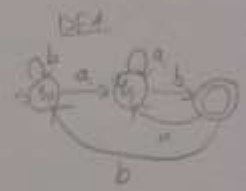
δ_{DFA}	**a**	**b**

Obtain an NFA to accept strings of a's an b's ending with ab and convert into equivalent DFA

Solution:

NFA to accept strings of a's and b's ending with ab is shown below





$M_{NFA} = (Q_{NFA}, \Sigma, \delta_{NFA}, q_0, F_{NFA})$

$Q_{NFA} = \{q_0, q_1, q_2\}$

$\Sigma = \{a, b\}$

$q_0 = \{q_0\}$

$F = \{q_2\}$

δ_{NFA}	a	b
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	ϕ	$\{q_2\}$
q_2	ϕ	ϕ

$M_{DFA} = (Q_{DFA}, \Sigma, \delta_{DFA}, q_0, F_{DFA})$

$\Sigma = \{a, b\}$

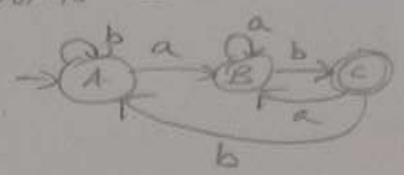
$q_0 = \{q_0\}$

$\delta_{DFA}(\{q_i, q_j \ldots q_k\}, a) = \{\delta_{NFA}(q_i, a) \cup \delta_{NFA}(q_j, a) \ldots \delta_{NFA}(q_k, a)\}$

δ_{DFA}	a	b
$\rightarrow \{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$* \{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$

$\{q_0\} = A$ $\{q_0, q_2\} = B$

$\{q_0, q_1\} = B$



$\delta_{DFA}(\{q_0\}, a) = \{\delta_{NFA}(q_0, a)\}$
$= \{q_0, q_1\}$

$\delta_{DFA}(\{q_0\}, b) = \{\delta_{NFA}(q_0, b)\}$
$= \{q_0\}$

$\delta_{DFA}(\{q_0, q_1\}, a) = \{\delta_{NFA}(q_0, a) \cup \delta_{NFA}(q_1, a)\}$
$= \{q_0, q_1\}$

$\delta_{DFA}(\{q_0, q_1\}, b) = \{\delta_{NFA}(q_0, b) \cup \delta_{NFA}(q_1, b)\}$
$= \cancel{\{q_0, q_1, q_2\}}$
$= \{q_0, q_2\}$

$\delta_{DFA}(\{q_0, q_2\}, a) = \{\delta_{NFA}(q_0, a) \cup \delta_{NFA}(q_2, a)\}$
$= \{q_0, q_1, \phi\}$
$= \{q_0, q_1\}$

$\delta_{DFA}(\{q_0, q_2\}, b) = \{\delta_{NFA}(q_0, b) \cup \delta_{NFA}(q_2, b)\}$
$= \{q_0, \phi\}$
$= \{q_0\}$

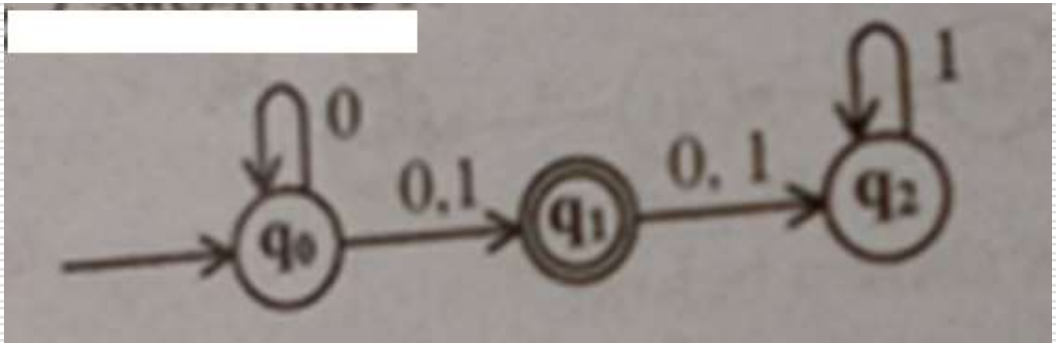
$Q_{DFA} = \{\{q_0\}, \{q_0, q_1\}, \{q_0, q_2\}\}$
$F_{DFA} = \{\{q_0, q_2\}\}$

δ_{NFA}	**a**	**b**
q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
q_2	$\emptyset$	$\emptyset$

$\delta_{DFA}(\{q_i, q_j, \ldots q_k\}, a) =$
$\{\delta_{NFA}(q_i, a) \cup$
$\delta_{NFA}(q_j, a) \ldots\ldots\}$

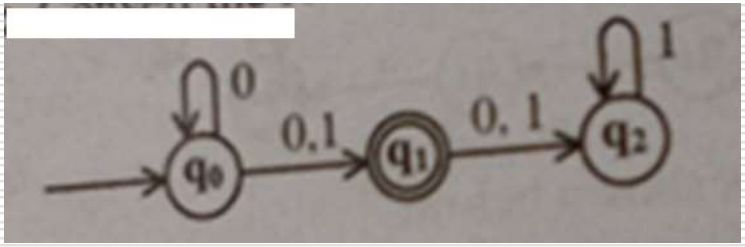
Question

Convert following NFA to its equivalent DFA

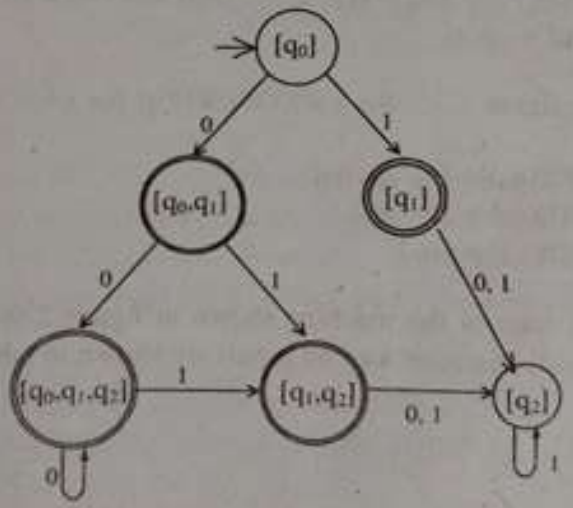


Answer

Convert following NFA to its equivalent DFA



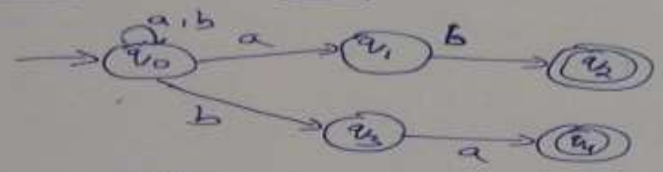
δ	0	1
$[q_0]$	$[q_0, q_1]$	$[q_1]$
$[q_0,q_1]$	$[q_0, q_1, q_2]$	$[q_1, q_2]$
$[q_1]$	$[q_2]$	$[q_2]$
$[q_0,q_1,q_2]$	$[q_0,q_1,q_2]$	$[q_1, q_2]$
$[q_1,q_2]$	$[q_2]$	$[q_2]$
$[q_2]$	ϕ	$[q_2]$



Obtain NFA to accept strings of a's and b's ending with ab or ba and convert into equivalent DFA

Obtain NFA to accept strings of a's and b's ending with ab or ba and convert into equivalent DFA

Solution: NFA



(or)

NFA

$M_{NFA} = (Q_{NFA}, \Sigma, \delta_{NFA}, q_0, F_{NFA})$

$\Sigma = \{a, b\}$

$Q_{NFA} = \{q_0, q_1, q_2, q_3, q_4\}$

$q_0 = q_0$

$F_{NFA} = \{q_2, q_4\}$

δ_{NFA}	a	b
→ q_0	$\{q_0, q_1\}$	$\{q_0, q_3\}$
q_1	ϕ	$\{q_2\}$
* q_2	ϕ	ϕ
q_3	$\{q_4\}$	ϕ
* q_4	ϕ	ϕ

Converting to DFA

$M_{DFA} = (Q_{DFA}, \Sigma, \delta_{DFA}, q_0, F_{DFA})$

$\Sigma = \{a, b\}$

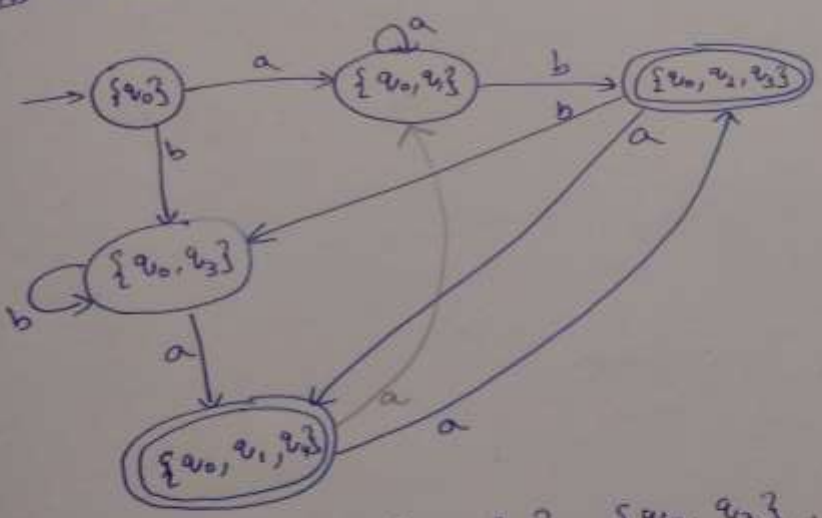
$q_0 = \{q_0\}$

In general,

$\delta_{DFA}(\{q_i, q_j, \ldots q_k\}, a) = \{\delta_{NFA}(q_i, a) \cup \delta_{NFA}(q_j, a) \ldots \delta_{NFA}(q_k, a)\}$

δ_{DFA}	a	b
$\rightarrow \{q_0\}$	$\{q_0, q_1\}$	$\{q_0, q_3\}$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2, q_3\}$
$\{q_0, q_3\}$	$\{q_0, q_1, q_4\}$	$\{q_0, q_3\}$
$*\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_4\}$	$\{q_0, q_3\}$
$*\{q_0, q_1, q_4\}$	$\{q_0, q_1\}$	$\{q_0, q_3, q_2\}$

DFA transition diagram



$Q_{DFA} = \{\{q_0\}, \{q_0, q_1\}, \{q_0, q_3\}, \{q_0, q_1, q_4\}, \{q_0, q_2, q_3\}\}$

$F_{DFA} = \{\{q_0, q_2, q_3\}, \{q_0, q_1, q_4\}\}$

Question

Convert the following NFA to a DFA

	0	1
→p	{p,q}	{p}
q	{r,s}	{t}
r	{p,r}	{t}
*s	∅	∅
*t	∅	∅

Question

Convert the following NFA to a DFA

	0	1
→p	{p,q}	{p}
q	{r,s}	{t}
r	{p,r}	{t}
*s	∅	∅
*t	∅	∅

Answer

DFA	0	1
→{p}	{p,q}	{p}
{p,q}	{p,q,r,s}	{p,t}
* {p,q,r,s}	{p,q,r,s}	{p,t}
*{p,t}	{p,q}	{p}

Question

Which of the following options is correct?
Statement 1: Initial State of NFA is Initial State of DFA.
Statement 2: The final state of DFA will be every combination of final state of NFA.

a) Statement 1 is true and Statement 2 is true
b) Statement 1 is true and Statement 2 is false
c) Statement 1 can be true and Statement 2 is true
d) Statement 1 is false and Statement 2 is also false

Question

Which of the following options is correct?
Statement 1: Initial State of NFA is Initial State of DFA.
Statement 2: The final state of DFA will be every combination of final state of NFA.
a) Statement 1 is true and Statement 2 is true
b) Statement 1 is true and Statement 2 is false
c) Statement 1 can be true and Statement 2 is true
d) Statement 1 is false and Statement 2 is also false

Answer: a
Explanation: Statement 1 and 2 always true for a given Language.

Question

Subset Construction method refers to:

a) Conversion of NFA to DFA

b) DFA minimization

c) Eliminating Null references

d) e-NFA to NFA

Question

Subset Construction method refers to:
a) Conversion of NFA to DFA
b) DFA minimization
c) Eliminating Null references
d) e-NFA to NFA

Answer: a
Explanation: The conversion of a non-deterministic automata into a deterministic one is a process we call subset construction or power set construction.

Questions

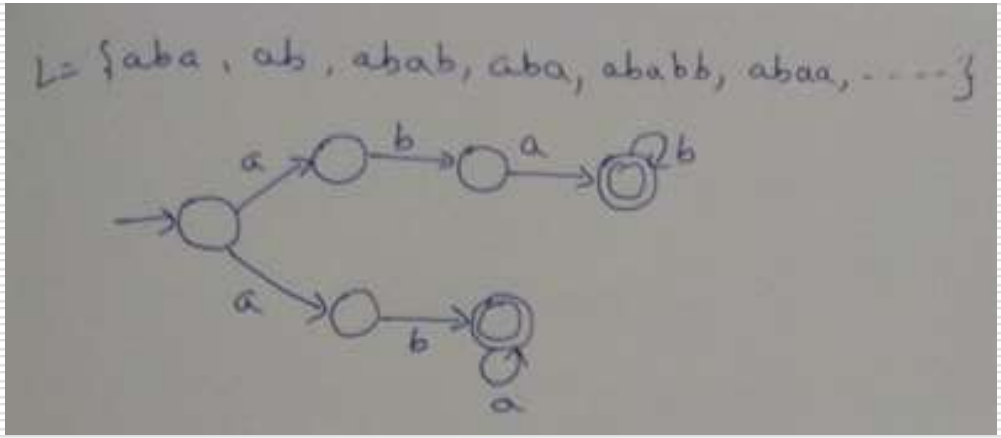
1. Obtain NFA for the language $L=\{w / w \in abab^n$ or aba^n $n>=0\}$
2. Design a NFA to accept the language which consists of string $w \in \{a,b\}*$ which has aab as substring
3. Design a NFA that accepts the language L(aa*(a+b))
4. Design a NFA that accepts the language $L=\{a^3 U a^{2n}, n>=1\}$
5. Construct NFA for accepting the set of all strings whose second last symbol is 1 over the alphabet set $\Sigma =\{0,1\}$
6. Write NFA accepting all strings of 0's and 1'ssuch that either second or third position from the end has 1.

Questions

1. Obtain NFA for the language $L=\{w \,/\, w \in abab^n \text{ or } aba^n \; n>=0\}$

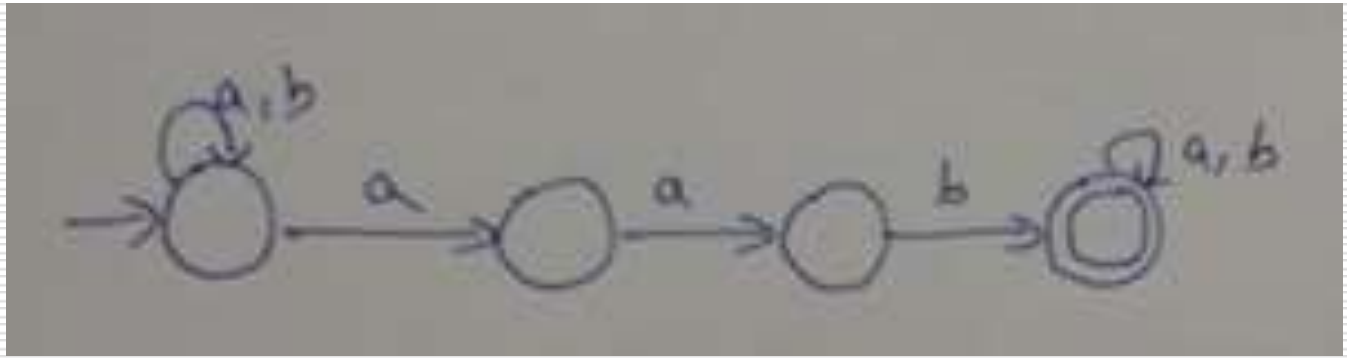
Questions

1. Obtain NFA for the language L={w / w $\in$ abab^n or aba^n n>=0}



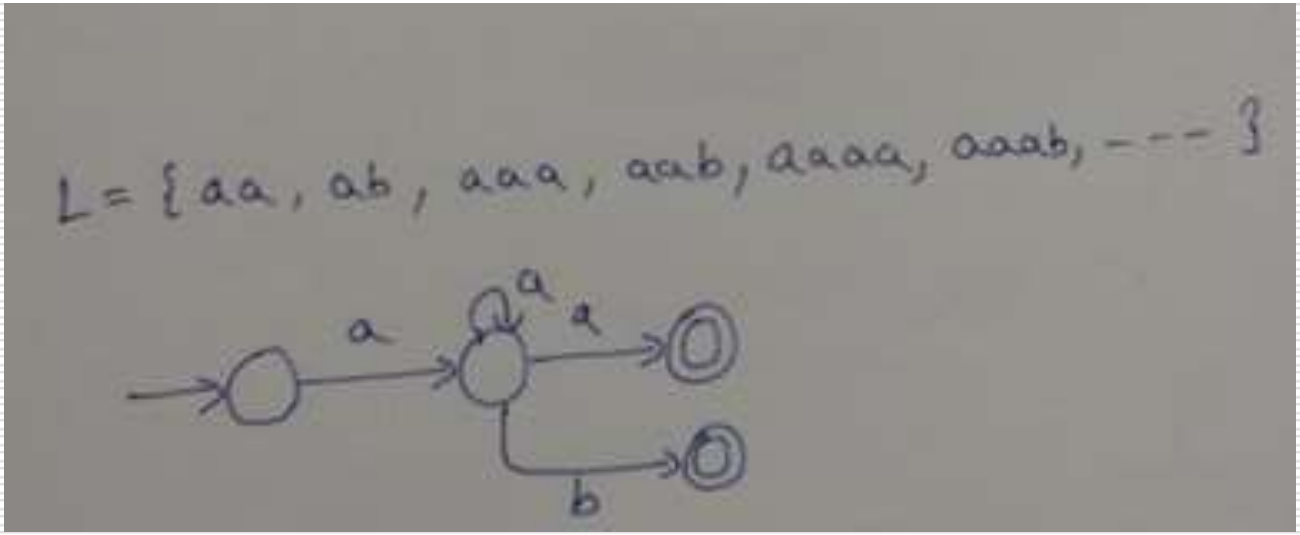
Questions

2. Design a NFA to accept the language which consists of string w $\in$ {a,b}$*$ which has aab as substring



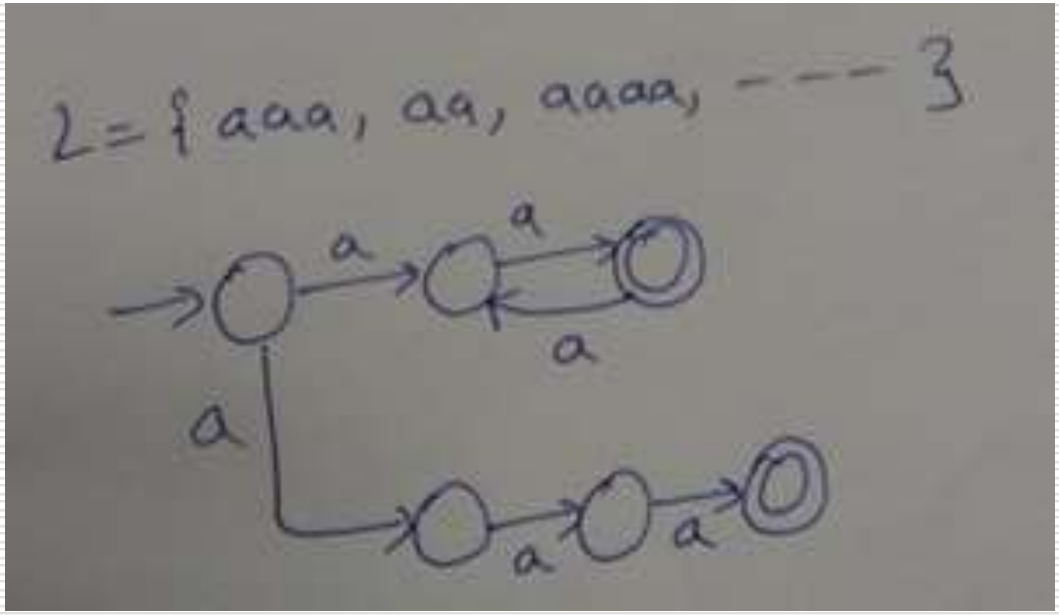
Questions

3. Design a NFA that accepts the language L(aa*(a+b))



Questions

4. Design a NFA that accepts the language $L=\{a^3 \cup a^{2n}, n>=1\}$



Questions

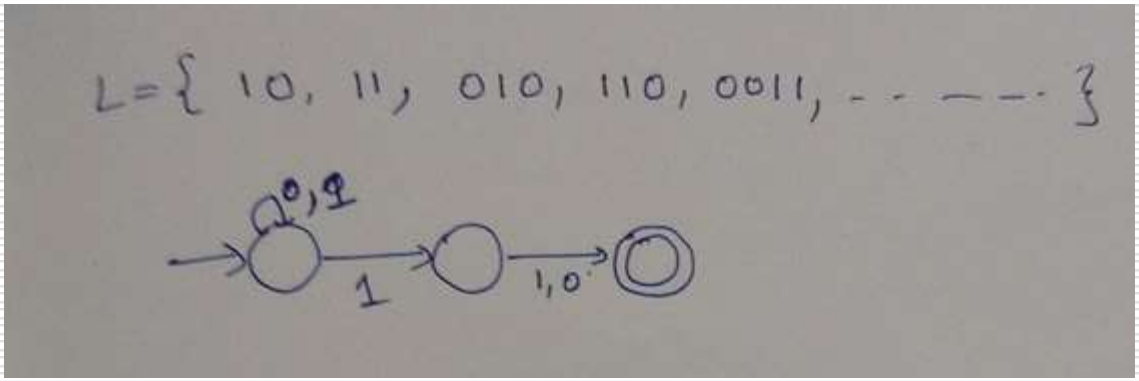
5. Construct NFA for accepting the set of all strings whose second last symbol is 1 over the alphabet set $\Sigma = \{0,1\}$

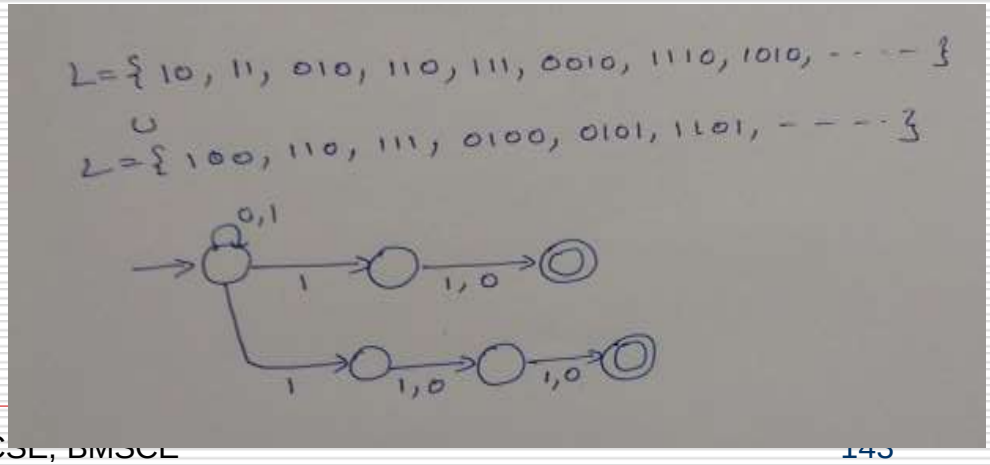
6. Write NFA accepting all strings of 0's and 1's such that either second or third position from the end has 1.

Questions

5. Construct NFA for accepting the set of all strings whose second last symbol is 1 over the alphabet set $\Sigma = \{0,1\}$

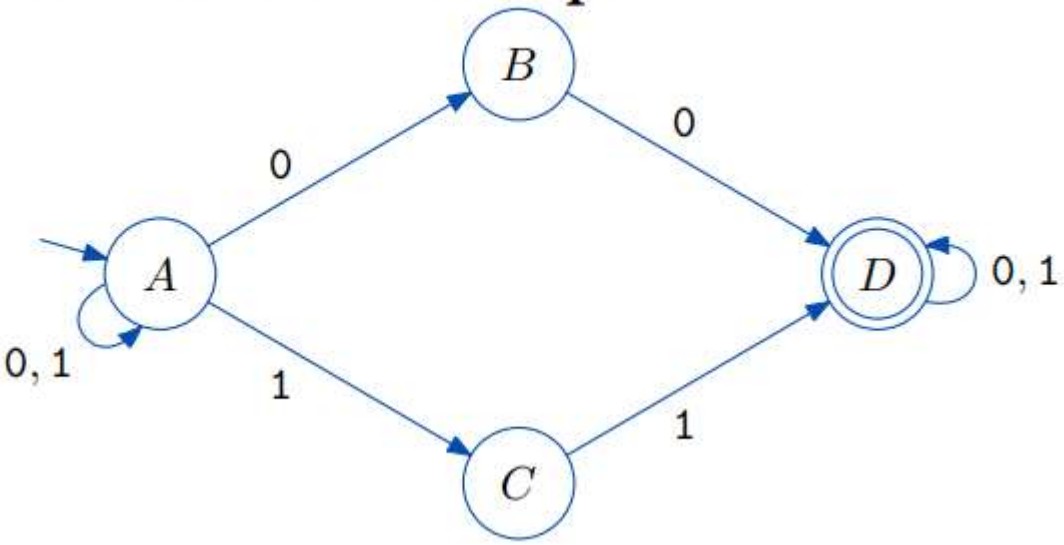
6. Write NFA accepting all strings of 0's and 1's such that either second or third position from the end has 1.





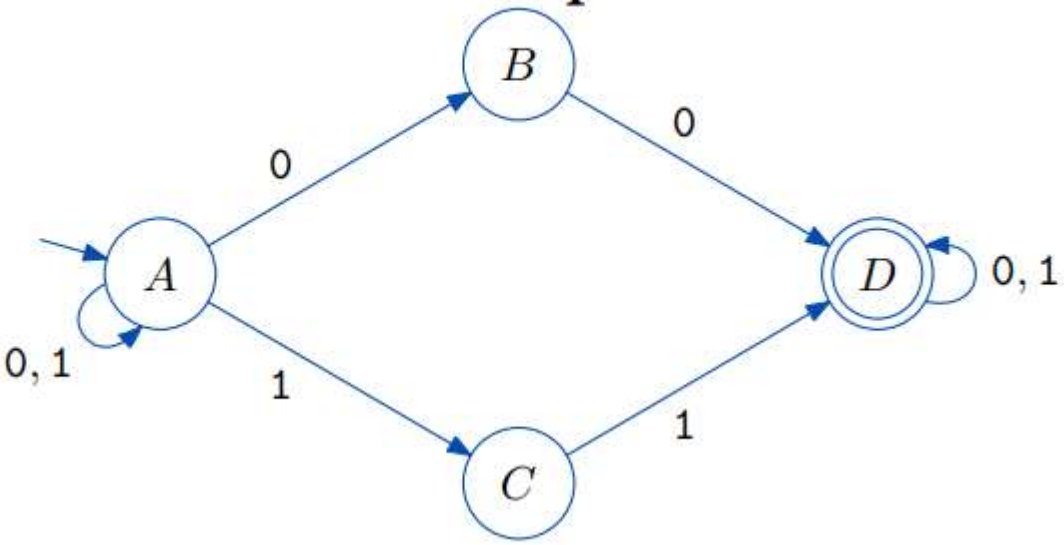
Question

What does this NFA accept?



Answer

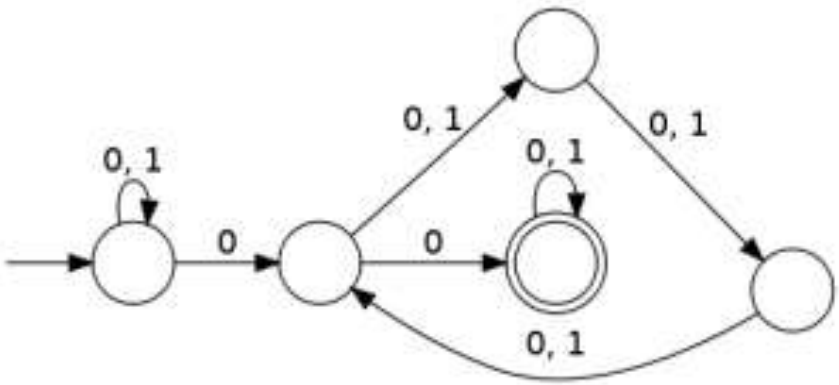
What does this NFA accept?



It accepts any binary string that contains 00 or 11 as a substring.

Question

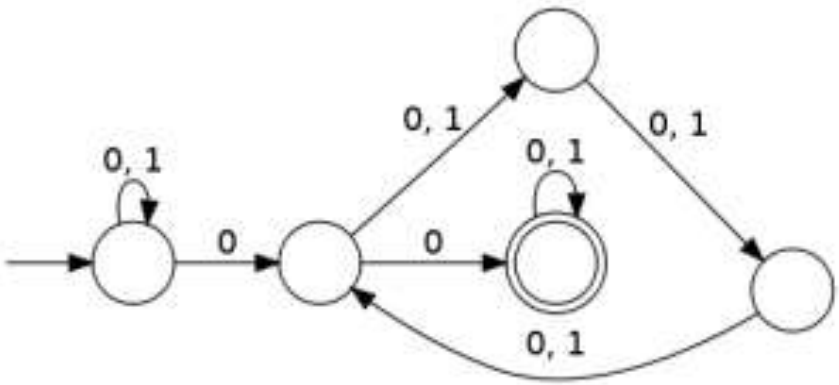
Consider the following non-deterministic finite automaton (NFA) over the alphabet $\Sigma = \{0, 1\}$.



Give a one-sentence description of the language recognized by the NFA. Write a regular expression for this language.

Answer

Consider the following non-deterministic finite automaton (NFA) over the alphabet $\Sigma = \{0, 1\}$.

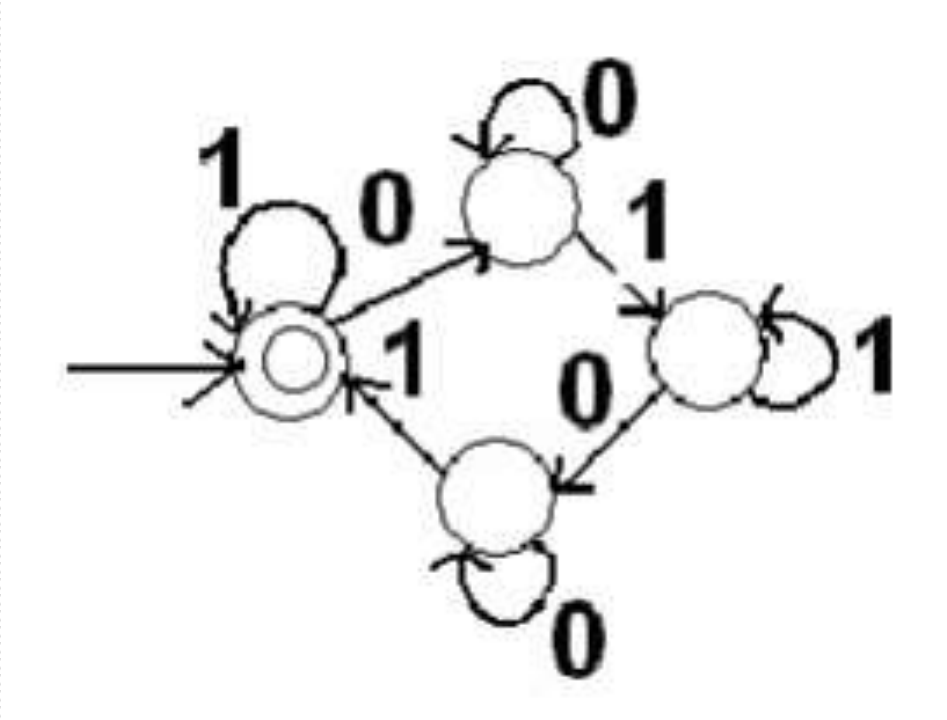


Give a one-sentence description of the language recognized by the NFA. Write a regular expression for this language.

- The NFA recognizes all strings that contain two 0's separated by a substring whose length is a multiple of 3.
- A regular expression for this language is $(0+1)^*0((0+1)(0+1)(0+1))^*0(0+1)^*$.

Homework Problem

- Give an NFA over {0,1} that accepts the set of strings that contain an even number of substrings 01.



Homework problem

Convert to a DFA the following NFA:

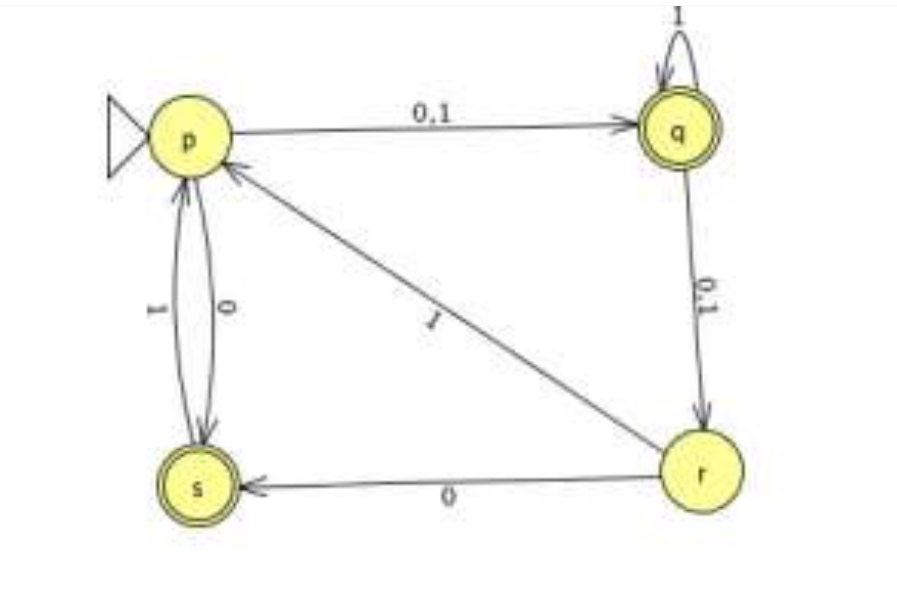
	0	1
$\rightarrow p$	$\{q, s\}$	$\{q\}$
$*q$	$\{r\}$	$\{q, r\}$
r	$\{s\}$	$\{p\}$
$*s$	$\emptyset$	$\{p\}$

Answer

Convert to a DFA the following NFA:

	0	1
$\rightarrow p$	$\{q, s\}$	$\{q\}$
$*q$	$\{r\}$	$\{q, r\}$
r	$\{s\}$	$\{p\}$
$*s$	$\emptyset$	$\{p\}$

NFA



Answer

Convert to a DFA the following NFA:

	0	1
$\rightarrow p$	$\{q, s\}$	$\{q\}$
$*q$	$\{r\}$	$\{q, r\}$
r	$\{s\}$	$\{p\}$
$*s$	$\emptyset$	$\{p\}$

The table for the equivalent DFA using the subset construction is:

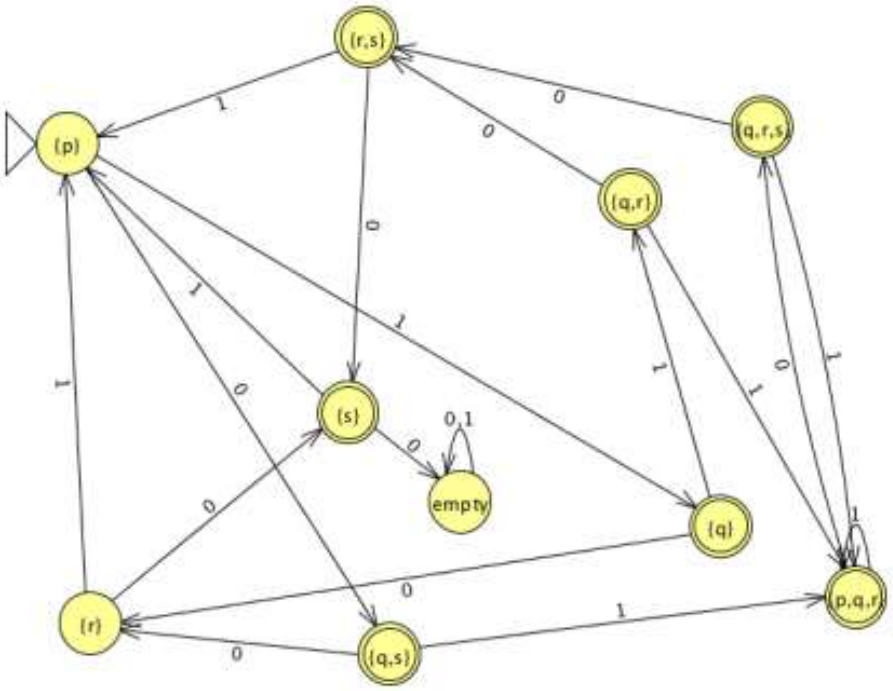
	0	1
$\rightarrow \{p\}$	$\{q, s\}$	$\{q\}$
$*\{q, s\}$	$\{r\}$	$\{p, q, r\}$
$*\{q\}$	$\{r\}$	$\{q, r\}$
$\{r\}$	$\{s\}$	$\{p\}$
$*\{p, q, r\}$	$\{q, r, s\}$	$\{p, q, r\}$
$*\{q, r\}$	$\{r, s\}$	$\{p, q, r\}$
$*\{s\}$	$\emptyset$	$\{p\}$
$*\{q, r, s\}$	$\{r, s\}$	$\{p, q, r\}$
$*\{r, s\}$	$\{s\}$	$\{p\}$
$\emptyset$	$\emptyset$	$\emptyset$

Answer

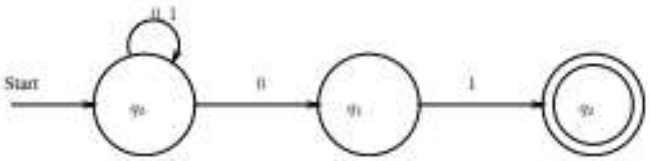
The table for the equivalent DFA using the subset construction is:

	0	1
$\rightarrow \{p\}$	$\{q,s\}$	$\{q\}$
$*\{q,s\}$	$\{r\}$	$\{p,q,r\}$
$*\{q\}$	$\{r\}$	$\{q,r\}$
$\{r\}$	$\{s\}$	$\{p\}$
$*\{p,q,r\}$	$\{q,r,s\}$	$\{p,q,r\}$
$*\{q,r\}$	$\{r,s\}$	$\{p,q,r\}$
$*\{s\}$	$\emptyset$	$\{p\}$
$*\{q,r,s\}$	$\{r,s\}$	$\{p,q,r\}$
$*\{r,s\}$	$\{s\}$	$\{p\}$
$\emptyset$	$\emptyset$	$\emptyset$

DFA



Converting NFA to DFA using subset construction method

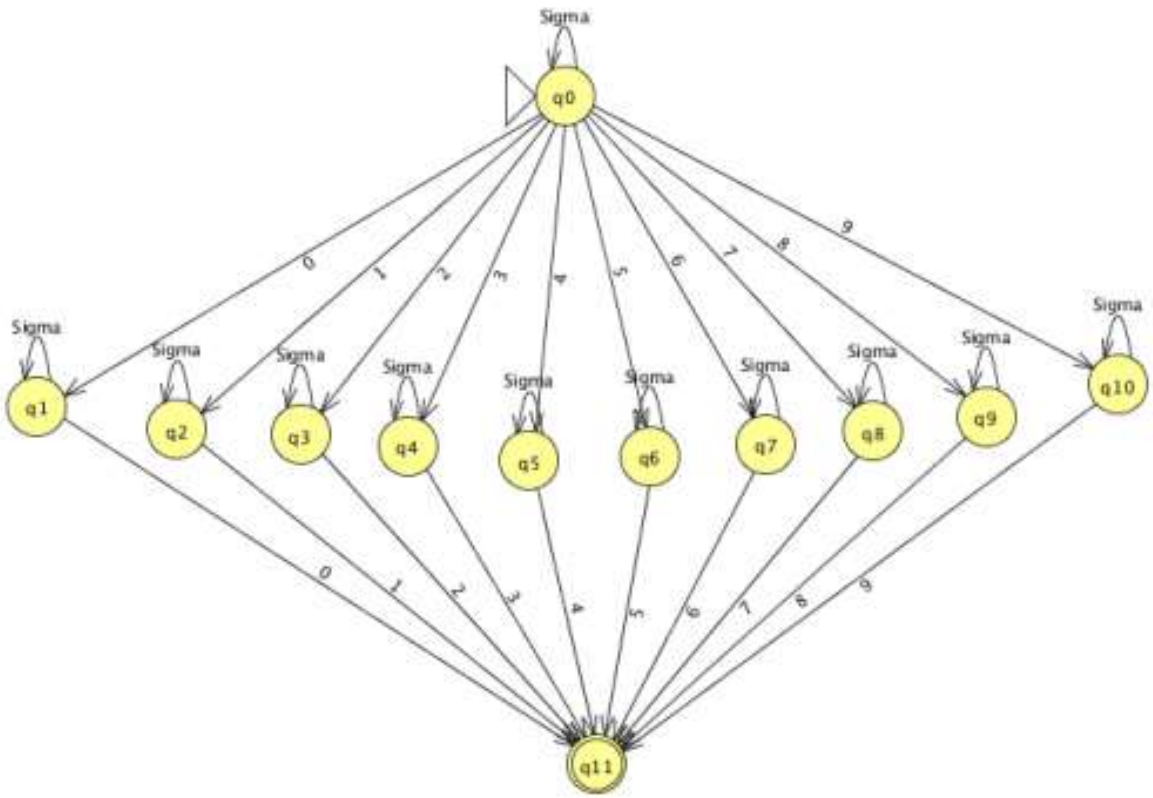


$Q_N = \{q_0, q_1, q_2\}$ then $Q_D = \{\emptyset, \{q_0\}, \{q_1\}, \{q_2\}, \{q_0, q_1\} \ldots\}$, *i.e.*, Q_D has 8 states (each one corresponding to a subset of Q_N)

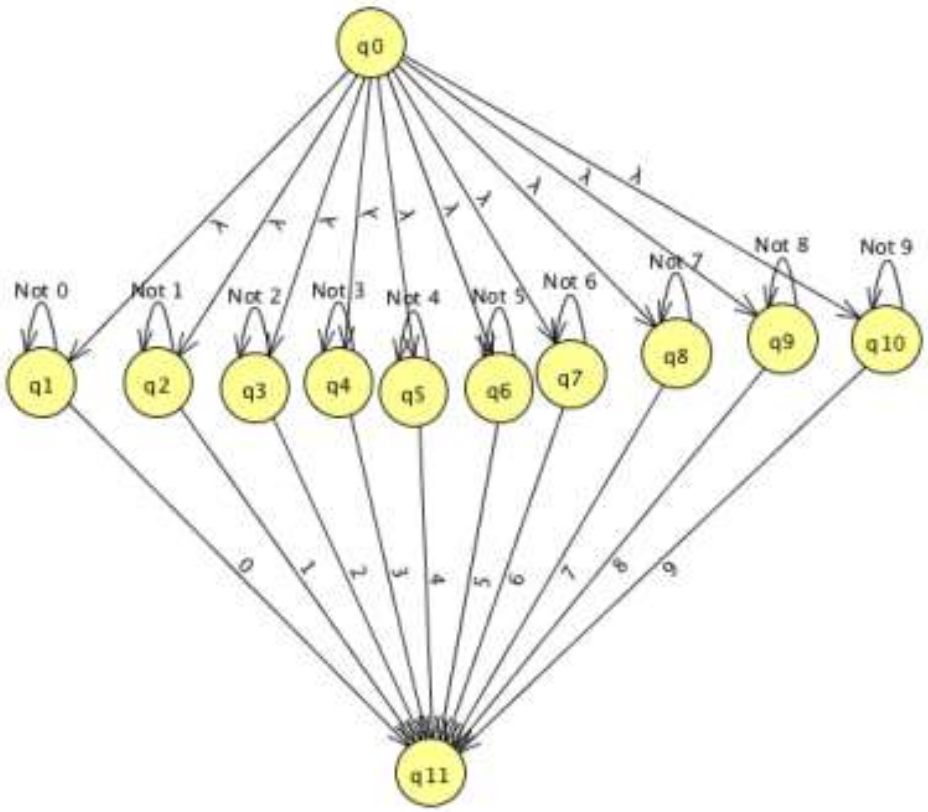
		0	1
	$\emptyset$	$\emptyset$	$\emptyset$
$\rightarrow$	$\{q_0\}$	$\{q_0\ q_1\}$	$\{q_0\}$
	$\{q_1\}$	$\emptyset$	$\{q_2\}$
$\star$	$\{q_2\}$	$\emptyset$	$\emptyset$
	$\{q_0, q_1\}$	$\{q_0\ q_1\}$	$\{q_0, q_2\}$
$\star$	$\{q_0, q_2\}$	$\{q_0\ q_1\}$	$\{q_0\}$
$\star$	$\{q_1, q_2\}$	$\emptyset$	$\{q_2\}$
$\star$	$\{q_0, q_1, q_2\}$	$\{q_0\ q_1\}$	$\{q_0, q_2\}$

Give nondeterministic finite automata to accept the following languages. Try to take advantage of nondeterminism as much as possible.

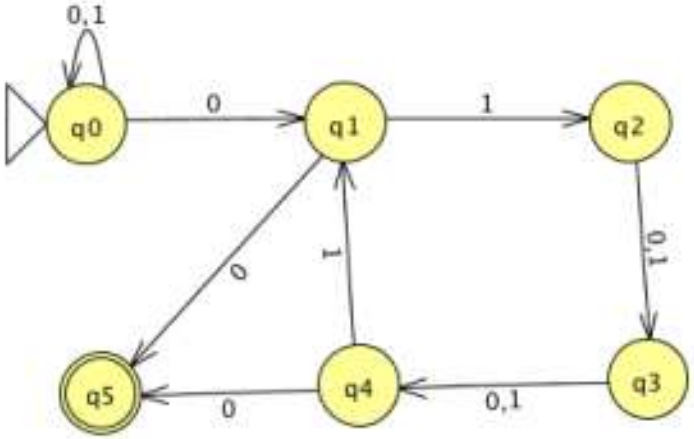
(a) The set of strings over alphabet $\{0, 1, \ldots, 9\}$ such that the final digit has appeared before.



(b) The set of strings over alphabet $\{0, 1, \ldots, 9\}$ such that the final digit has *not* appeared before.



(c) The set of strings of 0's and 1's such that there are two 0's separated by a number of positions that is a multiple of 4. Note that 0 is an allowable multiple of 4.



Homework Problem

Let A_2 be an automaton with the transition function given in the following table. The initial state of the automaton is s_0 and the only accepting state is s_f.

State	0	1
s_0	s_1	s_2
s_1	s_3	s_4
s_2	s_4	s_3
s_3	s_5	s_f
s_4	s_f	s_5
s_5	s_5	s_5
s_f	s_6	s_6
s_6	s_6	s_6

2.1 Understanding the Automaton

1. Draw the transition diagram for A_2.
2. Describe the language $L(A_2)$ accepted by A_2.

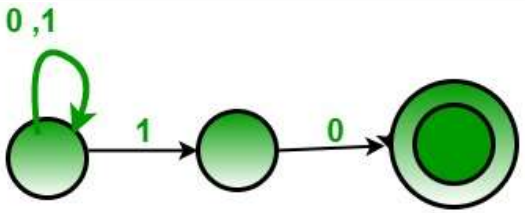
Question

- The number of states in the minimal deterministic finite automaton corresponding to the regular expression (0 + 1)*(10) is _____

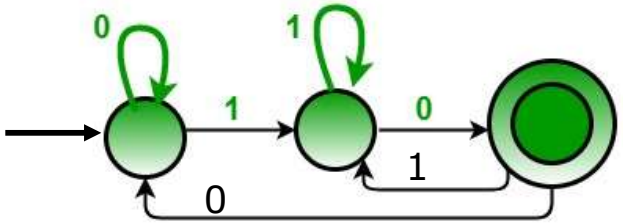
  (A) 2
  (B) 3
  (C) 4
  (D) 5

  Answer: (B)

NFA

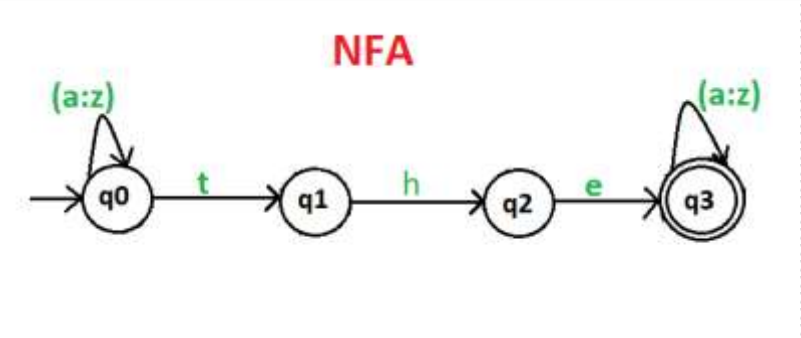


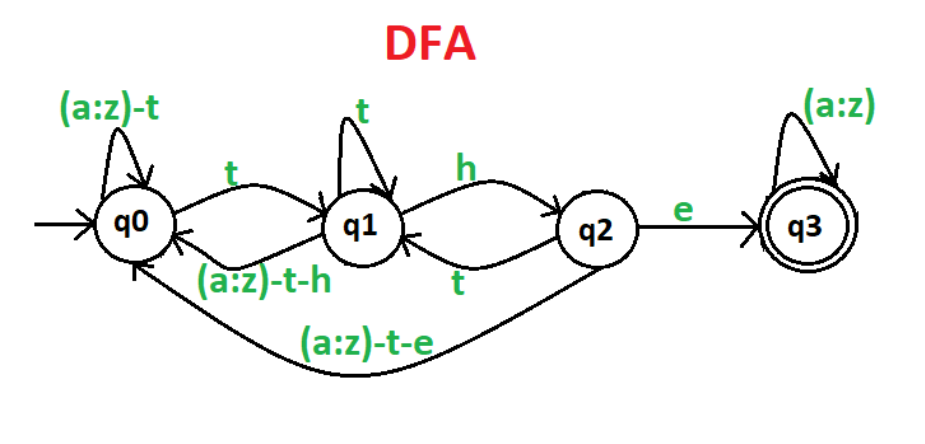
below DFA for the same



Question

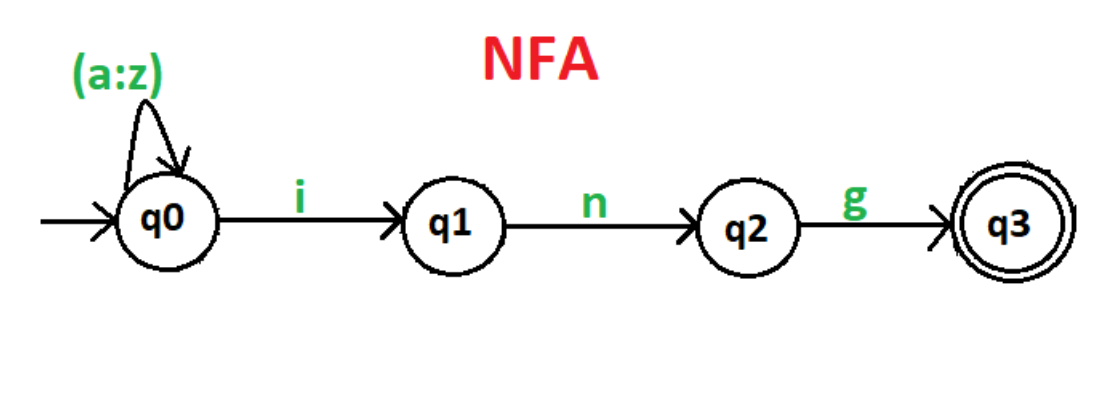
Draw a deterministic and non-deterministic finite automata which accept a string containing “the” anywhere in a string of {a-z}, e.g., “there” but not “those”.

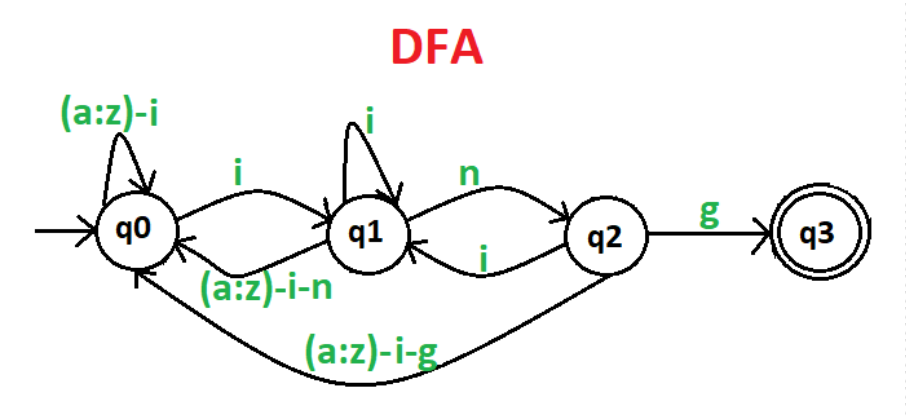




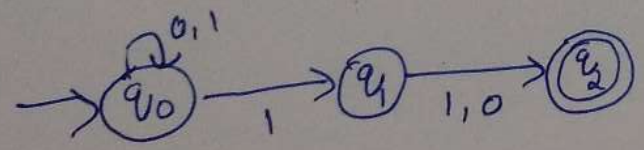
Question

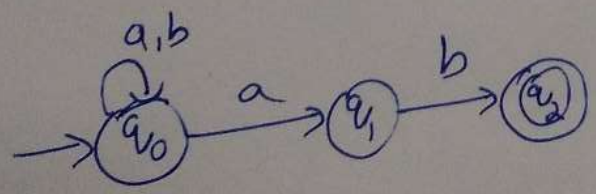
Draw a deterministic and non-deterministic finite automata which accept a string containing “ing” at the end of a string in a string of {a-z}, e.g., “anything” but not “anywhere”.

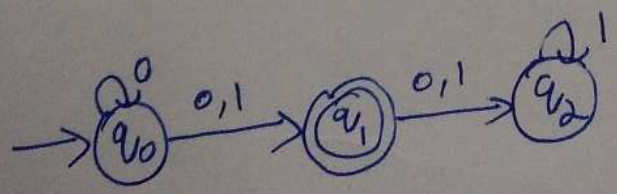




Convert the following NFA to DFA







Unit-1: Non-Deterministic Finite Automata (NFA) with ε-transitions

NFA with ε-transitions is a finite automata in which there are transitions from one state to other state even without taking input alphabet.

NFA with ε – Transitions: Formal Definition

NFA with ε –moves is a 5-tuple system

$M_E = (Q, \Sigma, \delta, q_0, F)$ where

- Q is a finite set of states
- Σ is an alphabet set or input symbols
- $q_0 \in Q$ is the initial (or start)state
- $F \subseteq Q$ is a set of accepting states (or final states).
- $\delta: Q \times (\Sigma \cup \varepsilon) \rightarrow 2^Q$ is a transition function

  δ (q , a) will consists of all states of p such that there is a transition labelled 'a' from q to p where 'a' is either ε or a symbol in Σ

Note:

- Transition in ε – NFA is allowed to make a transition spontaneously without receiving an input symbol.
- ε-NFA's are closely related to regular expressions.
- ε-NFA's are useful in proving the equivalence between the classes of language accepted by finite automate and by regular expressions

Rough slide to Explain example of ε-NFA

ε z-NFA that accepts decimal number consisting of

i. An optional + or - sign

ii. A string of digits

iii. A decimal point

iv. Another string of digits. Either this string of digits or the string can be empty, but at least one of the two strings of digits must be non empty.

Rough slide to Explain example of NFA without ε-moves

ε z-NFA that accepts decimal number consisting of

i. An optional + or - sign

ii. A string of digits

iii. A decimal point

iv. Another string of digits. Either this string of digits or the string can be empty, but at least one of the two strings of digits must be non empty.

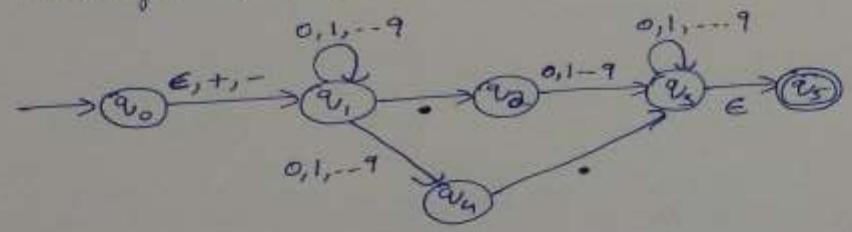
ε-NFA that accepts decimal number consisting of:

(a) An optional + or − sign

(b) A string of digits

(c) A decimal point

(d) Another string of digits. Either this string of digits, or the string can be empty, but at least one of the two strings of digits must be nonempty.



Ex: 2.5 , +2.5 , −2.5

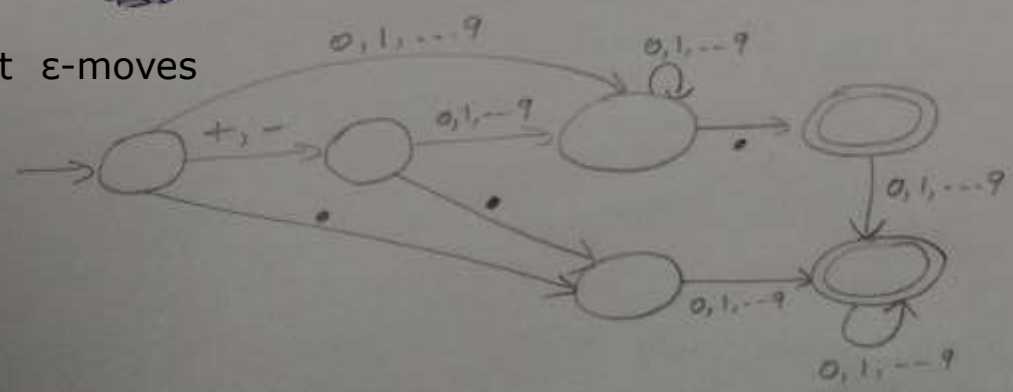
38.05 +38.05 −38.05

.05 +.05 −.05

+38. −38.

~~+38~~

NFA without ε-moves



$M = (Q, \Sigma, \delta, q_0, F)$

$Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$

$\Sigma = \{\bullet, +, -, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

$q_0 = q_0$

$F = \{q_5\}$

δ	ϵ	$+, -$	$\bullet$	$0, 1, \ldots 9$
q_0	$\{q_1\}$	$\{q_1\}$	ϕ	ϕ
q_1	ϕ	ϕ	$\{q_2\}$	$\{q_1, q_4\}$
q_2	ϕ	ϕ	ϕ	$\{q_3\}$
q_3	$\{q_5\}$	ϕ	ϕ	$\{q_3\}$
q_4	ϕ	ϕ	$\{q_3\}$	ϕ
q_5	ϕ	ϕ	ϕ	ϕ

ϵ-closure(q) : It is defined as set of all states which are reachable from q without giving any input.

ϵ-closure$(q) = q \cup$ set of all states that can be reached with ϵ

or

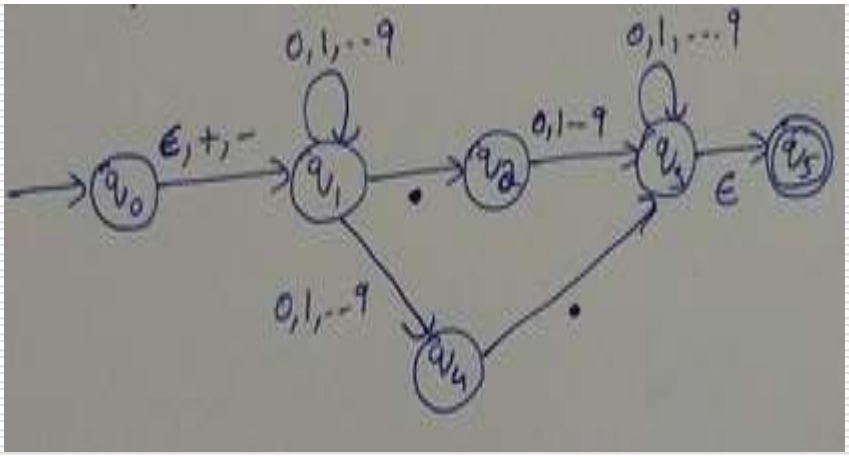
Empty closure of the state q

Union

Rough slide to explain:ε-Closure or Empty closure

ε-Closure of a state: It is a set of reachable states from a given state with ε-Transitions.

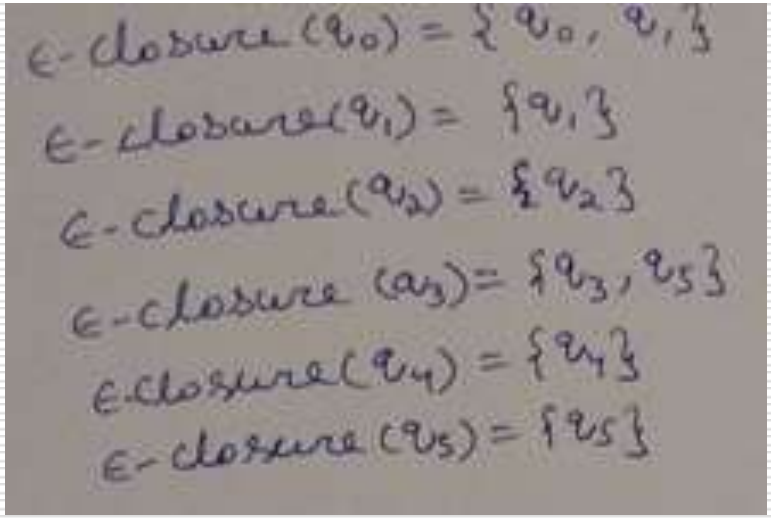
ε-Closure (q) = q U set of all states that can be reached with ε

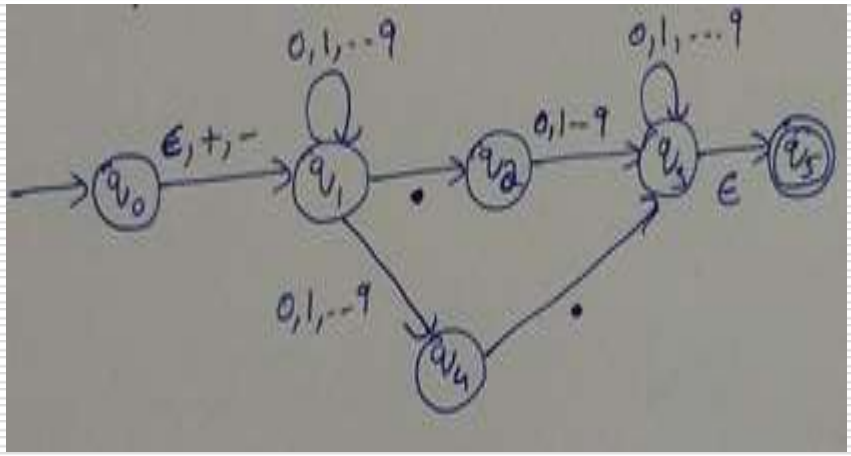


ε-Closure or Empty closure

ε-Closure of a state: It is a set of reachable states from a given state with ε-Transitions.

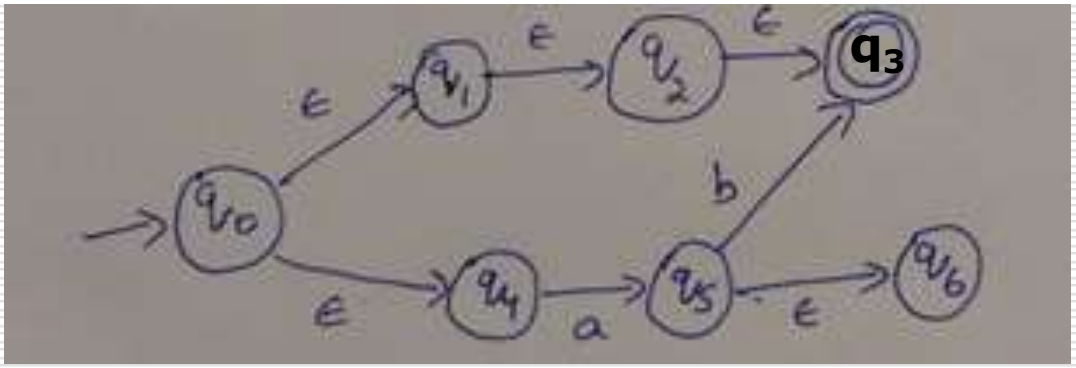
ε-Closure (q) = q U set of all states that can be reached with ε





Question

Considering the following ε–NFA , compute the ε-closure of the states q_0, q_4 , q_1

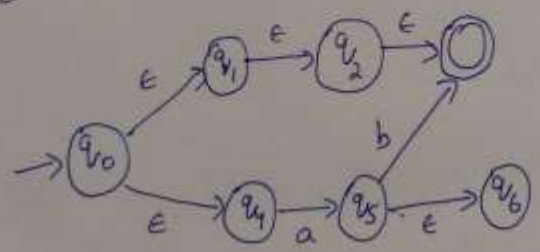


Considering the following, ε-NFA, Compute ε-closure of the states q_0, q_4, q_1

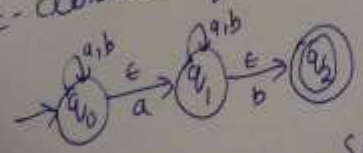
$\epsilon\text{-closure}(q_0) = \{q_0, q_1, q_4, q_2, q_3\}$

$\epsilon\text{-closure}(q_4) = \{q_4\}$

$\epsilon\text{-closure}(q_1) = \{q_1, q_2, q_3\}$



Considering the following ε-NFA, Compute ε-closure of the states q_0, q_1, q_2



$\epsilon\text{-closure}(q_0) = \{q_0, q_1, q_2\}$

$\epsilon\text{-closure}(q_1) = \{q_1, q_2\}$

$\epsilon\text{-closure}(q_2) = \{q_2\}$

Converting ε–NFA to DFA

Conversion procedure is very close to Subset construction procedure

Given an ε- NFA $M_E = (Q_e, \Sigma, \delta_e, q_0, F_e)$ which accepts the language L, find an equivalent DFA $M_D = (Q_d, \Sigma, \delta_d, q_d, F_d)$ such that $L(M_E) = L(M_D)$

Step-1: If q_0 is start state of ε–NFA then ε-Closure(q_0) is the start state of DFA i.e. , q_d = ε-Closure (q_0)

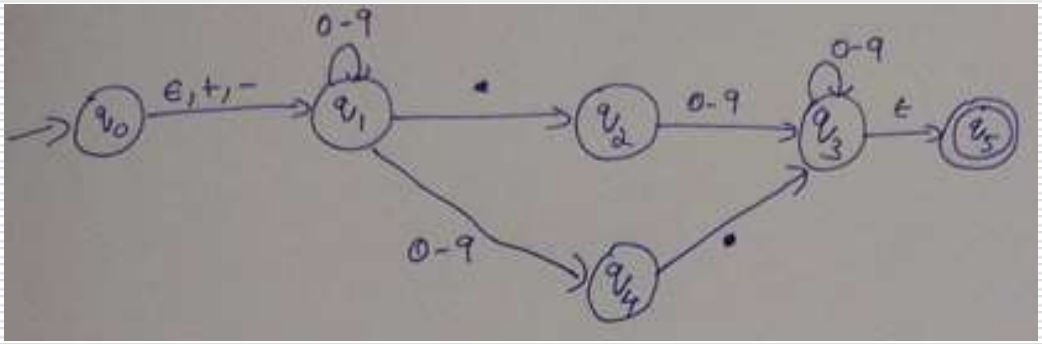
Step-2: Compute the transitions for DFA. Let $\{q_i , q_j, \ldots q_k\}$ be the state of DFA. Then, the transitions from this state on 'a' i.e. $\delta_d(\{q_i , q_j, \ldots q_k\} , a)$ is computed as shown below

- Let $\delta_e(\{q_i , q_j, \ldots q_k\} , a) = \{P_1, P_2, \ldots\ldots P_m\}$
- Then, take ε-Closure($\{P_1, P_2, \ldots\ldots P_m\}$)

Thus, $\delta_d(\{q_i , q_j, \ldots q_k\} , a)$ = ε-Closure($\delta_e(\{q_i , q_j, \ldots q_k\} , a)$)

Step-3: If $\{q_i , q_j, \ldots q_k\}$ is a state in DFA and if this set contains at least one final state of ε -NFA , then $\{q_i , q_j, \ldots q_k\}$ is the final state of DFA

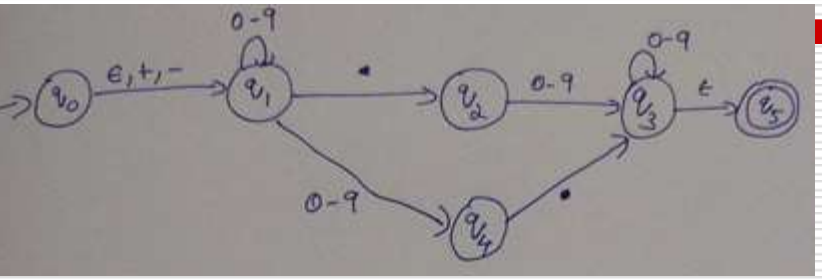
Rough Slide to explain Converting ε-NFA to DFA



δ_e	+ , -	.	0-9	ε
q_0	$\{q_1\}$	∅	∅	$\{q_1\}$
q_1	∅	$\{q_2\}$	$\{q_1, q_4\}$	∅
q_2	∅	∅	$\{q_3\}$	∅
q_3	∅	∅	$\{q_3\}$	$\{q_5\}$
q_4	∅	$\{q_3\}$	∅	∅
q_5	∅	∅	∅	∅

ε-closure(q_0) = $\{q_0, q_1\}$
ε-closure(q_1) = $\{q_1\}$
ε-closure(q_2) = $\{q_2\}$
ε-closure(q_3) = $\{q_3, q_5\}$
ε-closure(q_4) = $\{q_4\}$
ε-closure(q_5) = $\{q_5\}$

Rough Slide to explain Converting ε-NFA to DFA



δ_e	+ , -	.	0-9	ε
q_0	$\{q_1\}$	∅	∅	$\{q_1\}$
q_1	∅	$\{q_2\}$	$\{q_1, q_4\}$	∅
q_2	∅	∅	$\{q_3\}$	∅
q_3	∅	∅	$\{q_3\}$	$\{q_5\}$
q_4	∅	$\{q_3\}$	∅	∅
q_5	∅	∅	∅	∅

ε-closure(q_0) = {q_0, q_1
ε-closure(q_1) = {q_1}
ε-closure(q_2) = {q_2}
ε-closure (q_3) = {q_3, q_5
ε-closure(q_4) = {q_4}
ε-closure (q_5) = {q_5}

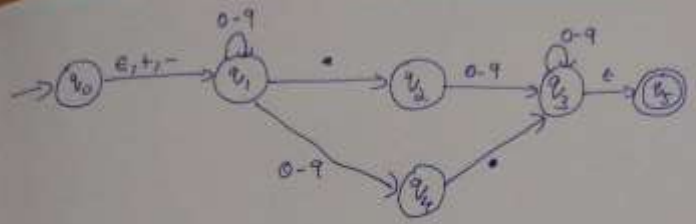
$M_D = (Q_d, \Sigma, \delta_d, q_d, F_d)$

$\Sigma = \{., +, -, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

q_d = ε -Closure(q_0)

δ_d	+ , -	.	0-9

Converting ε-NFA to DFA



ε-closure(q_0) = {q_0, q_1}

ε-closure(q_1) = {q_1}

ε-closure(q_2) = {q_2}

ε-closure(q_3) = {q_3, q_5}

ε-closure(q_4) = {q_4}

ε-closure(q_5) = {q_5}

δ_ε	+, −	•	0-9	ε
q_0	{q_1}	∅	∅	{q_1}
q_1	∅	{q_2}	{q_1, q_4}	∅
q_2	∅	∅	{q_3}	∅
q_3	∅	∅	{q_3}	∅
q_4	∅	{q_3}	∅	∅
q_5	∅	∅	∅	∅

Converting ε-NFA to DFA (Contd…)

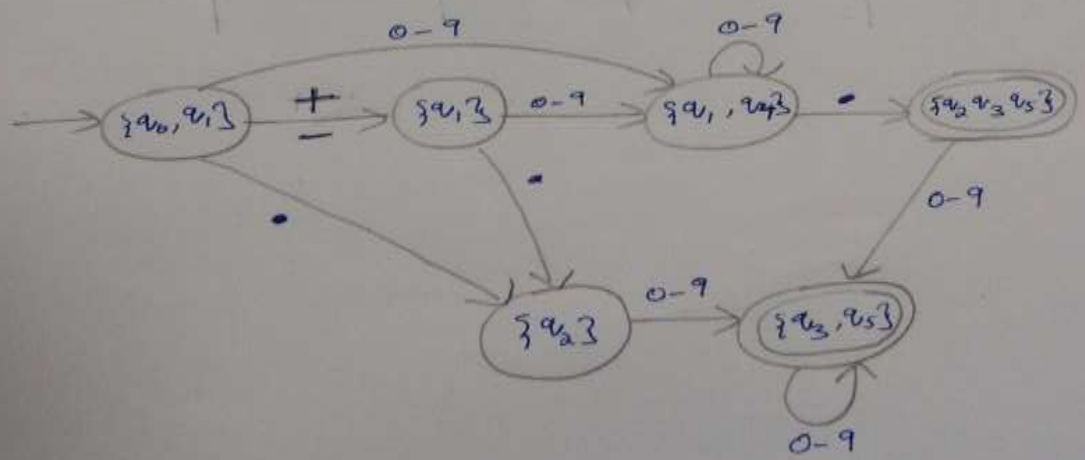
$M_D = (Q_D, \Sigma, \delta_D, q_D, F_D)$

$\Sigma = \{+, -, \cdot, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

$q_D = \epsilon\text{-closure}(q_0)$

$= \{q_0, q_1\}$

δ_D	$+, -$	$\cdot$	$0-9$
$\{q_0, q_1\}$	$\{q_1\}$	$\{q_2\}$	$\{q_1, q_4\}$
$\{q_1\}$	ϕ	$\{q_2\}$	$\{q_1, q_4\}$
$\{q_2\}$	ϕ	ϕ	$\{q_3, q_5\}$
$\{q_1, q_4\}$	ϕ	$\{q_2, q_3, q_5\}$	$\{q_1, q_4\}$
$\{q_3, q_5\}$	ϕ	ϕ	$\{q_3, q_5\}$
$\{q_2\ q_3\ q_5\}$	ϕ	ϕ	$\{q_3, q_5\}$



Converting ε-NFA- to DFA

$M_E = (Q_E, \Sigma, \delta_E, q_0, F_E)$

$Q_E = \{q_0, q_1, q_2\}$

$\Sigma = \{\epsilon, 1, 2, 3\}$

$q_0 = q_0$

$F_E = \{q_2\}$

δ_E	ϵ	1	2	3
q_0	q_1	q_0	ϕ	ϕ
q_1	q_2	ϕ	q_1	ϕ
q_2	ϕ	ϕ	ϕ	q_2

ϵ-Closure$(q_0) = \{q_0\ q_1\ q_2\}$

ϵ-Closure$(q_1) = \{q_1\ q_2\}$

ϵ-closure$(q_2) = \{q_2\}$

$M_D = (Q_D, \Sigma, \delta_D, q_D, F_D)$

$\Sigma = \{1, 2, 3\}$

$q_D = \epsilon$-closure(q_0)

$= \{q_0\ q_1\ q_2\}$

δ_D	1	2	3
A $\{q_0\ q_1\ q_2\}$	$\{q_0\ q_1\ q_2\}$	$\{q_1\ q_2\}$	$\{q_2\}$
B $\{q_1\ q_2\}$	ϕ	$\{q_1\ q_2\}$	$\{q_2\}$
C $\{q_2\}$	ϕ	ϕ	$\{q_2\}$

$\delta_D(\{q_0\ q_1\ q_2\}, 1) = \epsilon\text{-Closure}(\delta_E(\{q_0\ q_1\ q_2\}, 1))$

$= \epsilon\text{-Closure}(\delta_E(q_0, 1) \cup \delta_E(q_1, 1) \cup \delta_E(q_2, 1))$

$= \epsilon\text{-Closure}(q_0)$

$= \{q_0\ q_1\ q_2\}$ A

$\delta_D(\{q_0, q_1\ q_2\}, 2) = \epsilon\text{-Closure}(\delta_E(\{q_0\ q_1\ q_2\}, 2))$

$= \epsilon\text{-Closure}(q_1)$

$= \{q_1\ q_2\}$ B

$\delta_D(\{q_0\ q_1\ q_2\}, 3) = \epsilon\text{-Closure}(\delta_E(\{q_0\ q_1\ q_2\}, 3))$

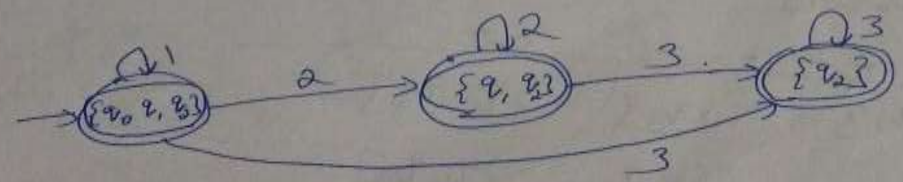
$= \epsilon\text{-Closure}(q_2)$

$= \{q_2\}$ C

$\delta_D(\{q_1\ q_2\}, 1) = \epsilon\text{-Closure}(\delta_E(\{q_1\ q_2\}, 1))$

$= \epsilon\text{-Closure}(\phi)$

$= \phi$

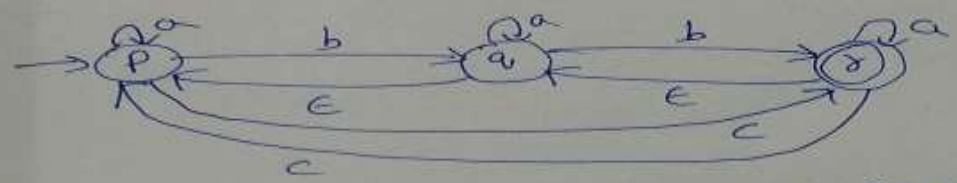


δ_D	1	2	3
A	A	B	C
B	ϕ	B	C
C	ϕ	ϕ	C

$Q_D = \{\{q_0\ q_1\ q_2\}, \{q_1\ q_2\}, \{q_2\}\}$

$F_D = \{\{q_0\ q_1\ q_2\}, \{q_1\ q_2\}\ \{q_2\}\}$

δ_E	ϵ	a	b	c
→ p	ϕ	$\{p\}$	$\{q\}$	$\{r\}$
q	$\{p\}$	$\{q\}$	$\{r\}$	ϕ
* r	$\{q\}$	$\{r\}$	ϕ	$\{p\}$



(i) Complete ε- closure of each state

(ii) Give all strings of length three or less accepted by the automaton

(iii) Convert the automaton to a DFA

$M_E = (Q_E, \Sigma, \delta_E, q_0, F_E)$ ε-NFA

$Q_E = \{p, q, r\}$

$\Sigma = \{\epsilon, a, b, c\}$

$q_0 = p$

$F_E = \{r\}$

(i) ε-closure $(p) = \{p\}$

(ii) ε-closure $(q) = \{p, q\}$

(iii) ε-closure $(r) = \{p, q, r\}$

(ii) Strings of 3 or less accepted by the ε-NFA

$\{c, bb, abb, bab, bba, ca, caa,$

(iii) Converting ϵ-NFA to DFA

$M_D = (Q_D, \Sigma, \delta_D, q_D, F_D)$

$\Sigma = \{a, b, c\}$

$q_D = \epsilon\text{-closure}(p)$

$= \{p\}$

δ_D	a	b	c
$\rightarrow \{p\}$	$\{p\}$	$\{p, q\}$	$\{p, q, r\}$
$\{p, q\}$	$\{p, q\}$	$\{p, q, r\}$	$\{p, q, r\}$
$*\{p, q, r\}$	$\{p, q, r\}$	$\{p, q, r\}$	$\{p, q, r\}$

$\delta_D(\{p\}, a) = \epsilon\text{-closure}(\delta_\epsilon(p, a))$

$= \epsilon\text{-closure}(p)$

$= \{p\}$

$\delta_D(\{p, q\}, a) = \epsilon\text{-closure}(\delta_\epsilon(p, a) \cup \delta_\epsilon(q, a))$

$= \epsilon\text{-closure}(\{p, q\})$

$= \{p, q\}$

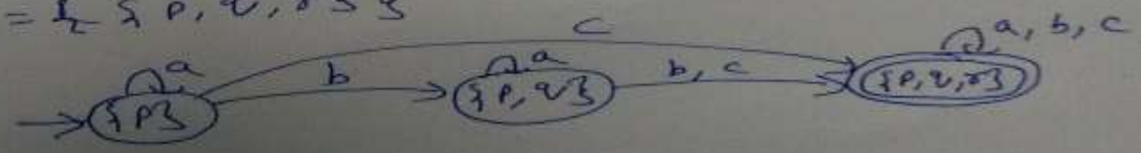
$\delta_D(\{p, q\}, b) = \epsilon\text{-closure}(\delta_\epsilon(p, b) \cup \delta_\epsilon(q, b))$

$= \epsilon\text{-closure}(\{q, r\})$

$= \{p, q, r\}$

$Q_D = \{\{p\}, \{p, q\}, \{p, q, r\}\}$

$F_D = \{\{p, q, r\}\}$



Difference between DFA, NFA, ε-NFA

DFA	NFA	**ε**-NFA
M= $(Q, \Sigma, \delta, q_0, F)$ where Q is a finite set of states Σ is an alphabet set or input symbols $\delta: Q \times \Sigma \rightarrow Q$ is a transition function $q_0 \in Q$ is the initial (or start)state $F \subseteq Q$ is a set of accepting states	M_{NFA}= $(Q, \Sigma, \delta, q_0, F)$ where Q is a finite set of states Σ is an alphabet set or input symbols $q_0 \in Q$ is the initial (or start)state $F \subseteq Q$ is a set of accepting states (or final states). $\delta: Q \times \Sigma \rightarrow 2^Q$ is a transition function	M_E= $(Q, \Sigma, \delta, q_0, F)$ where Q is a finite set of states Σ is an alphabet set or input symbols $q_0 \in Q$ is the initial (or start)state $F \subseteq Q$ is a set of accepting states (or final states). $\delta: Q \times (\Sigma \cup \varepsilon) \rightarrow 2^Q$ is a transition function
If a string w is accepted by DFA from state q, there will be exactly one path labelled w starting at q. Therefore, to determine if a string is accepted, it is sufficient to check for this one path. Following this path, if the machine is in final state, we say that the string w is accepted. Otherwise, the string is rejected	If a string w is accepted by NFA from state q, there will be many paths labelled w. Therefore, to determine if a string is accepted, it is necessary to check all these paths to see whether on or more terminate at final state. Following these path, if we reach a final state, the string is accepted else the string is rejected.	For ε-NFA, there may be many paths labelled w along with ε and all must be checked to see whether one or more terminate at final state. If the final state is reached, the given string w is accepted by the machine else it is rejected by the machine.
Difficult to construct	Easy to construct	Easy to construct using regular expression

Text Search using Automata

2.4 An Application: Text Search

In this section, we shall see that the abstract study of the previous section, where we considered the "problem" of deciding whether a sequence of bits ends in 01, is actually an excellent model for several real problems that appear in applications such as Web search and extraction of information from text.

2.4.1 Finding Strings in Text

A common problem in the age of the Web and other on-line text repositories is the following. Given a set of words, find all documents that contain one (or all) of those words. A search engine is a popular example of this process. The search engine uses a particular technology, called *inverted indexes*, where for each word appearing on the Web (there are 100,000,000 different words), a list of all the places where that word occurs is stored. Machines with very large amounts of main memory keep the most common of these lists available, allowing many people to search for documents at once.

Inverted-index techniques do not make use of finite automata, but they also take very large amounts of time for crawlers to copy the Web and set up the indexes. There are a number of related applications that are unsuited for inverted indexes, but are good applications for automaton-based techniques. The characteristics that make an application suitable for searches that use automata are:

1. The repository on which the search is conducted is rapidly changing. For example:

   (a) Every day, news analysts want to search the day's on-line news articles for relevant topics. For example, a financial analyst might search for certain stock ticker symbols or names of companies.

(b) A "shopping robot" wants to search for the current prices charged for the items that its clients request. The robot will retrieve current catalog pages from the Web and then search those pages for words that suggest a price for a particular item.

2. The documents to be searched cannot be cataloged. For example, Amazon.com does not make it easy for crawlers to find all the pages for all the books that the company sells. Rather, these pages are generated "on the fly" in response to queries. However, we could send a query for books on a certain topic, say "finite automata," and then search the pages retrieved for certain words, e.g., "excellent" in a review portion.

2.4.2 Nondeterministic Finite Automata for Text Search

Suppose we are given a set of words, which we shall call the *keywords*, and we want to find occurrences of any of these words. In applications such as these, a useful way to proceed is to design a nondeterministic finite automaton, which signals, by entering an accepting state, that it has seen one of the keywords. The text of a document is fed, one character at a time to this NFA, which then recognizes occurrences of the keywords in this text. There is a simple form to an NFA that recognizes a set of keywords.

1. There is a start state with a transition to itself on every input symbol, e.g. every printable ASCII character if we are examining text. Intuitively, the start state represents a "guess" that we have not yet begun to see one of the keywords, even if we have seen some letters of one of these words.

2. For each keyword $a_1a_2\cdots a_k$, there are k states, say $q_1, q_2, \ldots, q_k$. There is a transition from the start state to q_1 on symbol a_1, a transition from q_1 to q_2 on symbol a_2, and so on. The state q_k is an accepting state and indicates that the keyword $a_1a_2\cdots a_k$ has been found.

Example 2.14: Suppose we want to design an NFA to recognize occurrences of the words `web` and `ebay`. The transition diagram for the NFA designed using the rules above is in Fig. 2.16. State 1 is the start state, and we use Σ to stand for the set of all printable ASCII characters. States 2 through 4 have the job of recognizing `web`, while states 5 through 8 recognize `ebay`. □

Of course the NFA is not a program. We have two major choices for an implementation of this NFA.

1. Write a program that simulates this NFA by computing the set of states it is in after reading each input symbol. The simulation was suggested in Fig. 2.10.

2. Convert the NFA to an equivalent DFA using the subset construction. Then simulate the DFA directly.

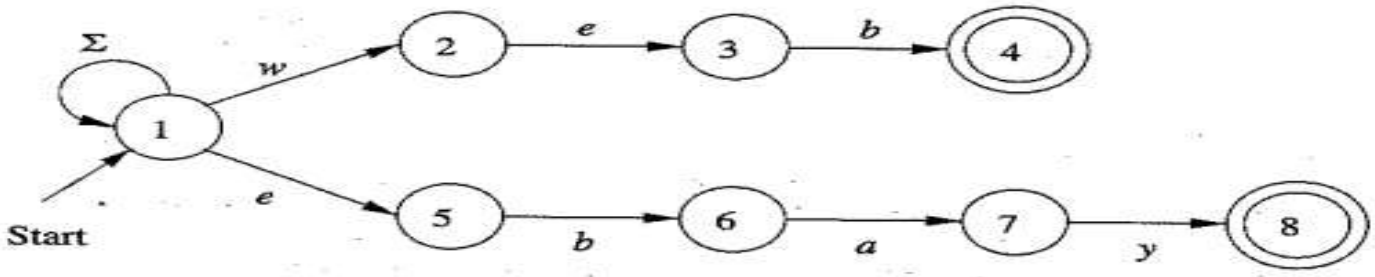


Figure 2.16: An NFA that searches for the words web and ebay

Some text-processing programs, such as advanced forms of the UNIX grep command (egrep and fgrep) actually use a mixture of these two approaches. However, for our purposes, conversion to a DFA is easy and is guaranteed not to increase the number of states.

2.4.3 A DFA to Recognize a Set of Keywords

We can apply the subset construction to any NFA. However, when we apply that construction to an NFA that was designed from a set of keywords, according to the strategy of Section 2.4.2, we find that the number of states of the DFA is never greater than the number of states of the NFA. Since in the worst case the number of states exponentiates as we go to the DFA, this observation is good news and explains why the method of designing an NFA for keywords and then constructing a DFA from it is used frequently. The rules for constructing the set of DFA states is as follows.

a) If q_0 is the start state of the NFA, then $\{q_0\}$ is one of the states of the DFA.

b) Suppose p is one of the NFA states, and it is reached from the start state along a path whose symbols are $a_1 a_2 \cdots a_m$. Then one of the DFA states is the set of NFA states consisting of:

   1. q_0.
   2. p.
   3. Every other state of the NFA that is reachable from q_0 by following a path whose labels are a suffix of $a_1 a_2 \cdots a_m$, that is, any sequence of symbols of the form $a_j a_{j+1} \cdots a_m$.

Note that in general, there will be one DFA state for each NFA state p. However, in step (b), two states may actually yield the same set of NFA states, and thus become one state of the DFA. For example, if two of the keywords begin with the same letter, say a, then the two NFA states that are reached from q_0 by an

arc labeled a will yield the same set of NFA states and thus get merged in the DFA.

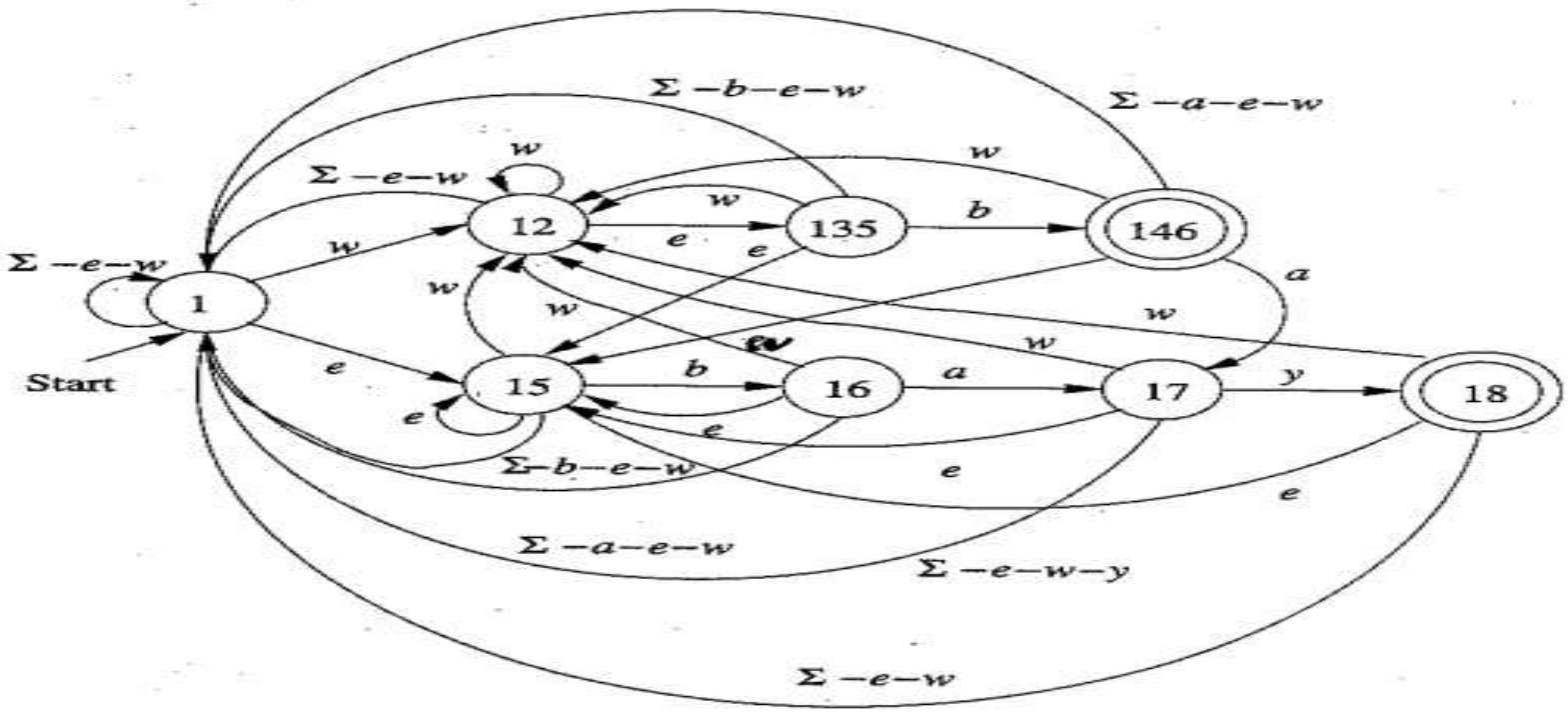


Figure 2.17: Conversion of the NFA from Fig. 2.16 to a DFA

Example 2.15: The construction of a DFA from the NFA of Fig. 2.16 is shown in Fig. 2.17. Each of the states of the DFA is located in the same position as the state p from which it is derived using rule (b) above. For example, consider the state 135, which is our shorthand for $\{1, 3, 5\}$. This state was constructed from state 3. It includes the start state, 1, because every set of the DFA states does. It also includes state 5 because that state is reached from state 1 by a suffix, e, of the string we that reaches state 3 in Fig. 2.16.

The transitions for each of the DFA states may be calculated according to the subset construction. However, the rule is simple. From any set of states that includes the start state q_0 and some other states $\{p_1, p_2, \ldots, p_n\}$, determine, for each symbol x, where the p_i's go in the NFA, and let this DFA state have a transition labeled x to the DFA state consisting of q_0 and all the targets of the

p_i's on symbol x. On all symbols x such that there are no transitions out of any of the p_i's on symbol x, let this DFA state have a transition on x to that state of the DFA consisting of q_0 and all states that are reached from q_0 in the NFA following an arc labeled x.

For instance, consider state 135 of Fig. 2.17. The NFA of Fig. 2.16 has transitions on symbol b from states 3 and 5 to states 4 and 6, respectively. Therefore, on symbol b, 135 goes to 146. On symbol e, there are no transitions of the NFA out of 3 or 5, but there is a transition from 1 to 5. Thus, in the DFA, 135 goes to 15 on input e. Similarly, on input w, 135 goes to 12.

On every other symbol x, there are no transitions out or 3 or 5, and state 1 goes only to itself. Thus, there are transitions from 135 to 1 on every symbol in Σ other than b, e, and w. We use the notation $\Sigma - b - e - w$ to represent this set, and use similar representations of other sets in which a few symbols are removed from Σ. □

Thanks for Listening

Look for updates at Course Webpage:
https://sites.google.com/site/drvumadevi/teaching/tfc-jan-may-2020

No need to remember the webpage URL
Do Google Search -☺ – "Umadevi BMS", you will find teaching webpage

Homework Problems

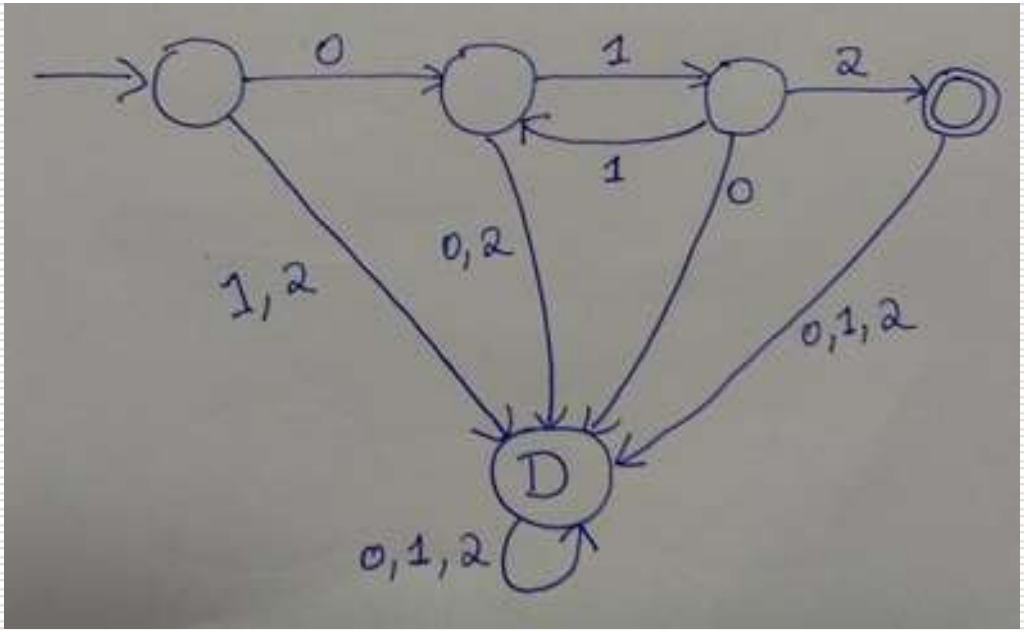
1. Obtain a DFA to accept strings of 0's , 1's and 2's beginning with a '0' followed by odd number of 1's and ending with '2'

2. Construct DFA for the following language over the alphabet set Σ ={0, 1}

L={w / w is even length and begins with '01'

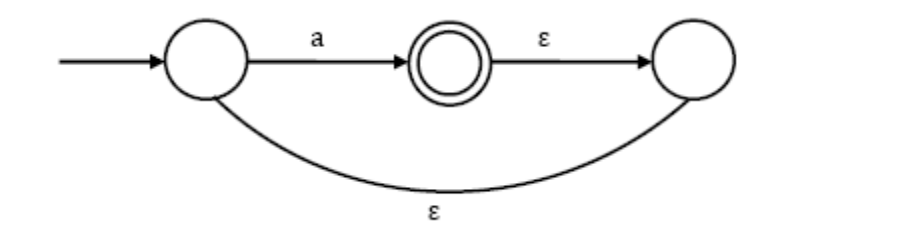
Question

Obtain a DFA to accept strings of 0's , 1's and 2's beginning with a '0' followed by odd number of 1's and ending with '2'



Question

- What is the complement of the language accepted by the NFA shown below? Assume ∑ = {a} and ε is the empty string



- (A) Φ
  (B) ε
  (C) a
  (D) {a, ε}
- Answer (B)
  The given alphabet ∑ contains only one symbol {a} and the given NFA accepts all strings with any number of occurrences of 'a'. In other words, the NFA accepts a+. Therefore complement of the language accepted by automata is empty string.